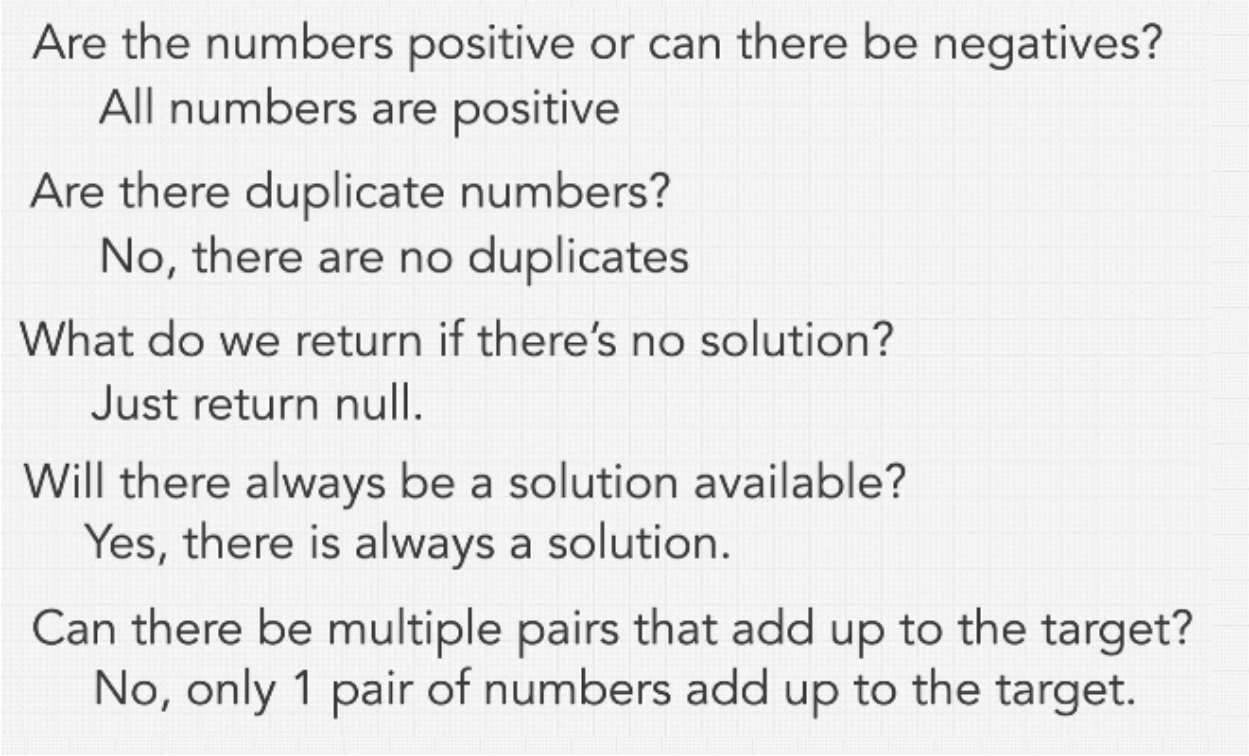
To see Theory and methods watch the google doc called “Theory and methods”

Arrays

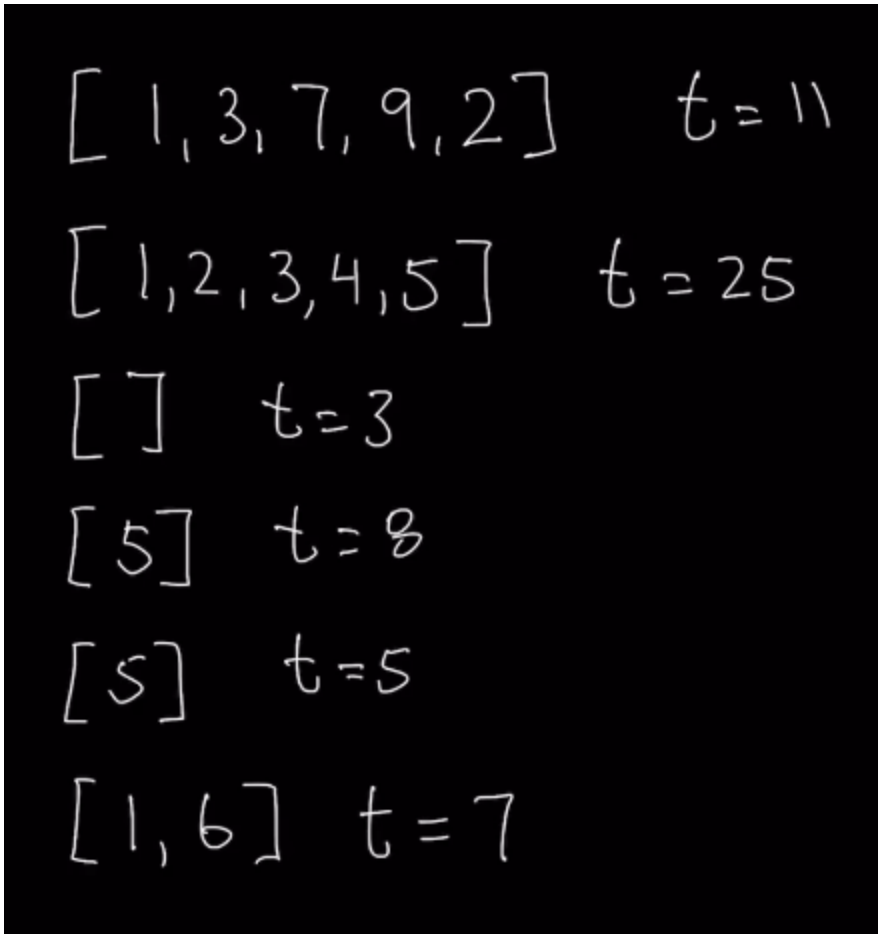
Get the indexes in an array that given the numbers we can sum to get a target

Problem

First check constraints.

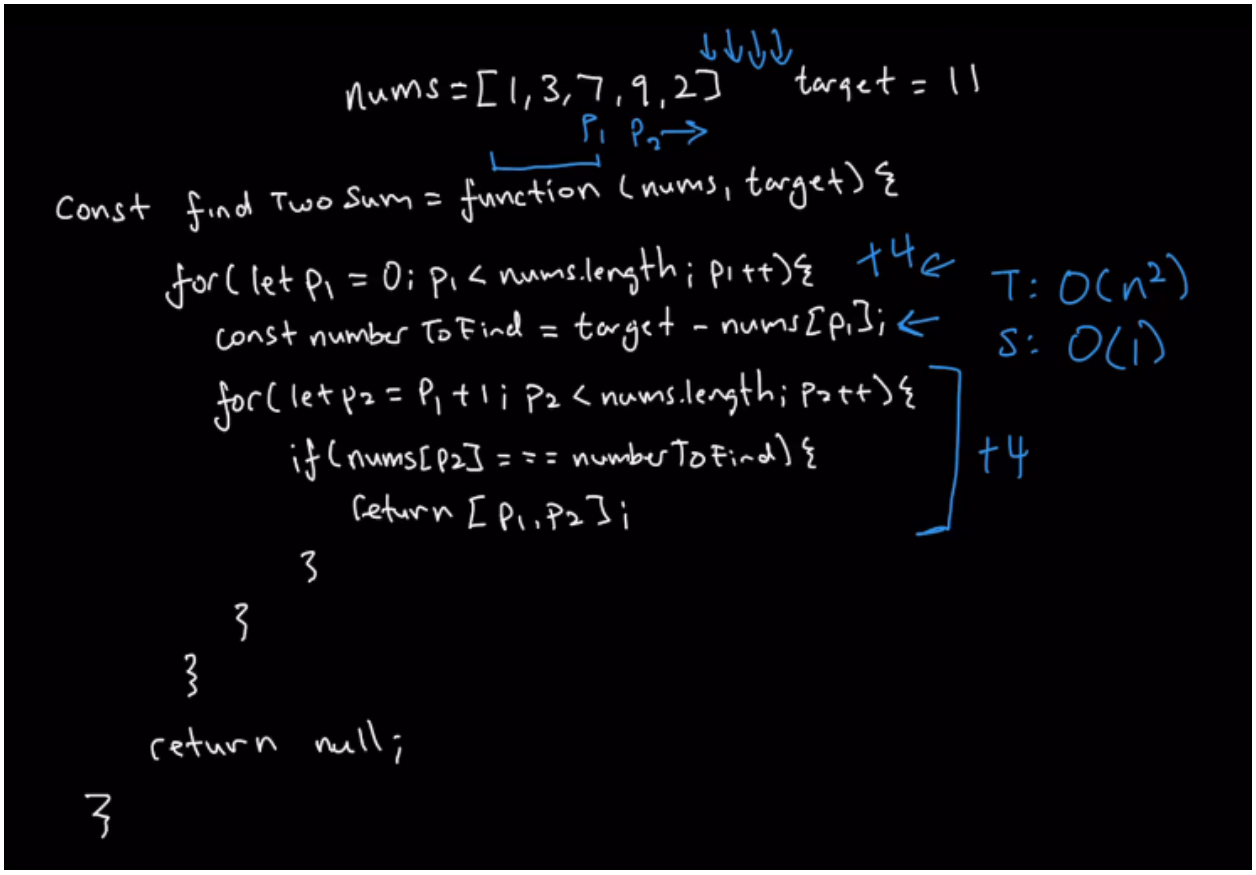


Write some test cases using the restrictions



The figure out a solution without code. We can first see the brute force solution

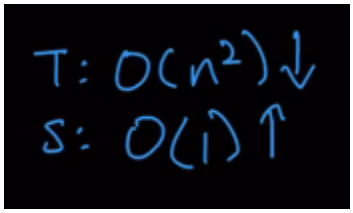
In this case we can analizy iteratively with two pointers. Then we should check for a solution. We should test in all test cases



Space complexity is really good, but time compléxity is really bad

Optimize the solution

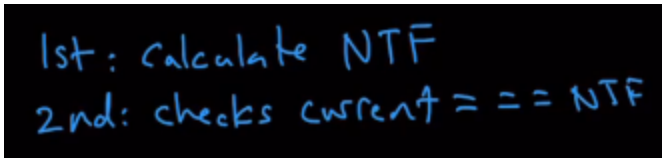
A good option is to make the space complexity worst and the time complexity better.



¿Can we remove a loop of the two loops?

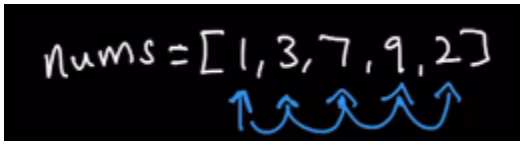
Para pasar un 2 loop to two loops we should figure out what is doing each loop

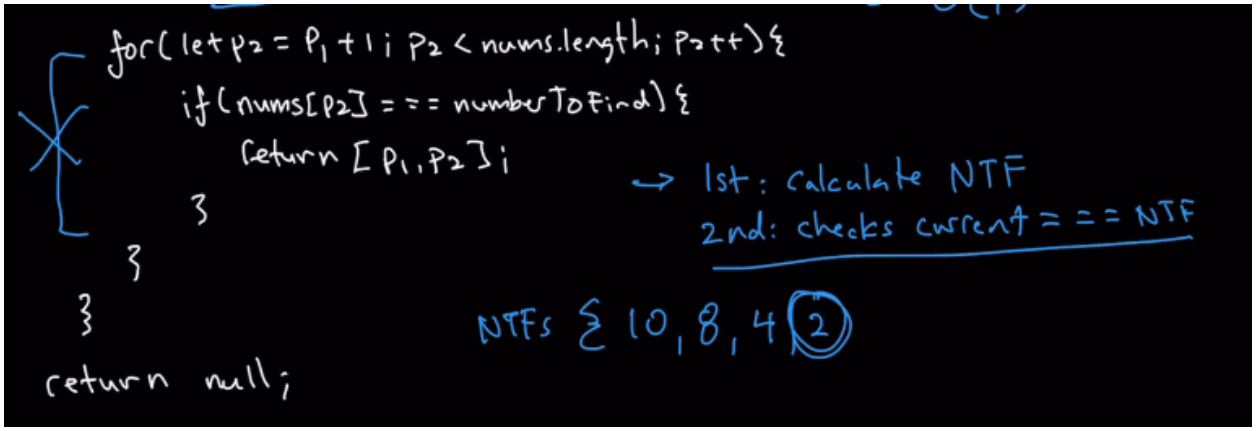
NTF: number to find



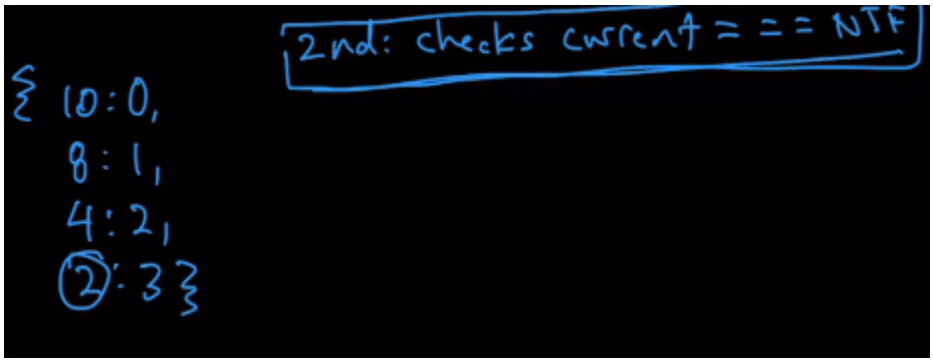
We can use a hashmap

We can first compute the number to find for each number

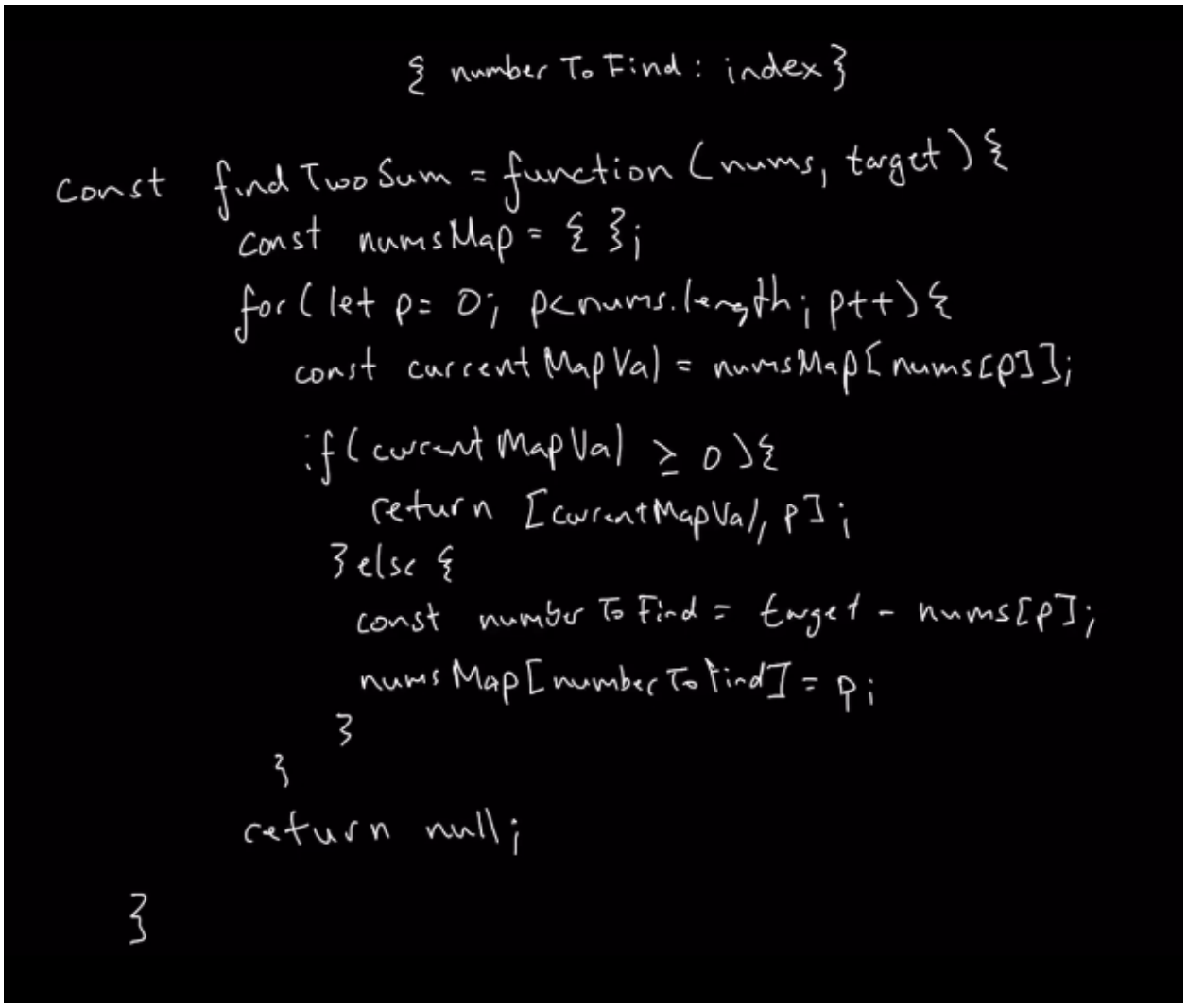




Now we will do the task of second for loop with a hash map

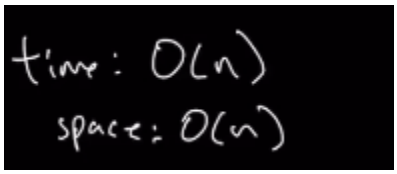


We should check if the value exists, we return the answer, else we generate the solution



Then we test this optimized solution

The space complexity has grown

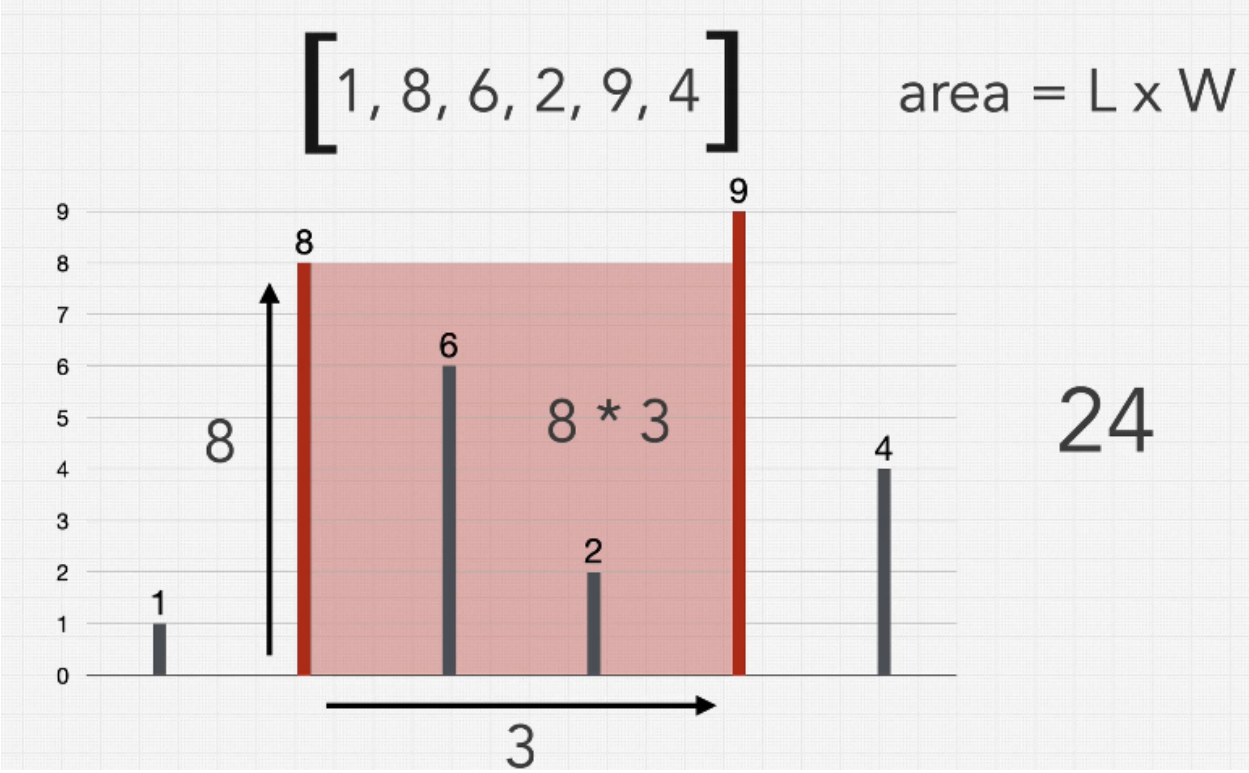


Method was Scale space complexity and reduce time complexity

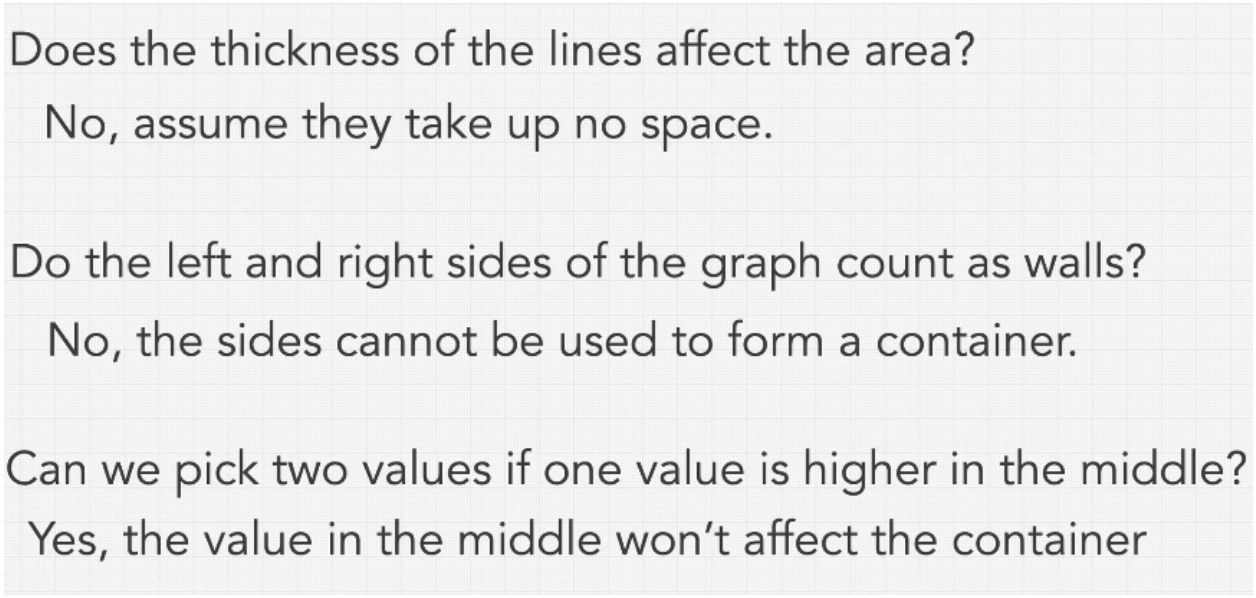
Container with most water

Problem

You are given an array of positive integers where each integer represents the height of a vertical line on a chart. Find two lines which together with the x-axis forms a container that would hold the greatest amount of water. Return the area of water it would hold.

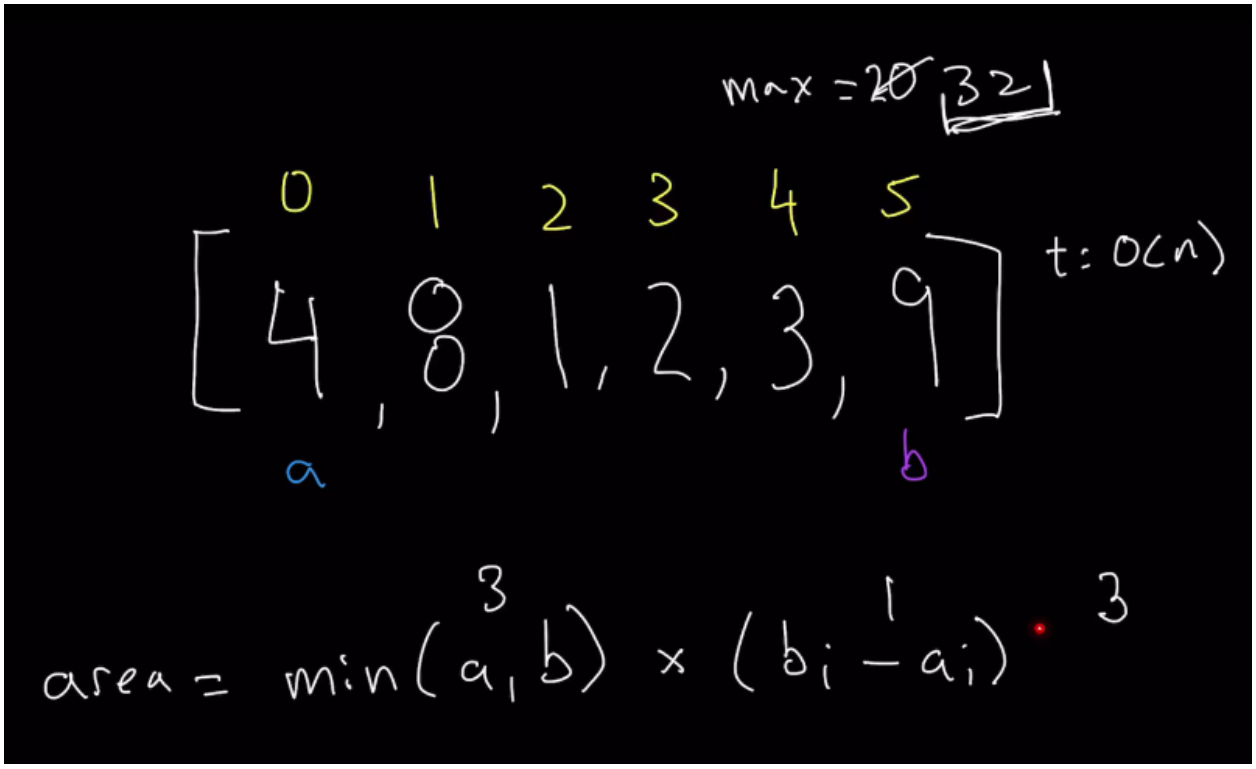


Constraints first



Solve without code

With greatest, this mean we should first calculate all possible solutions



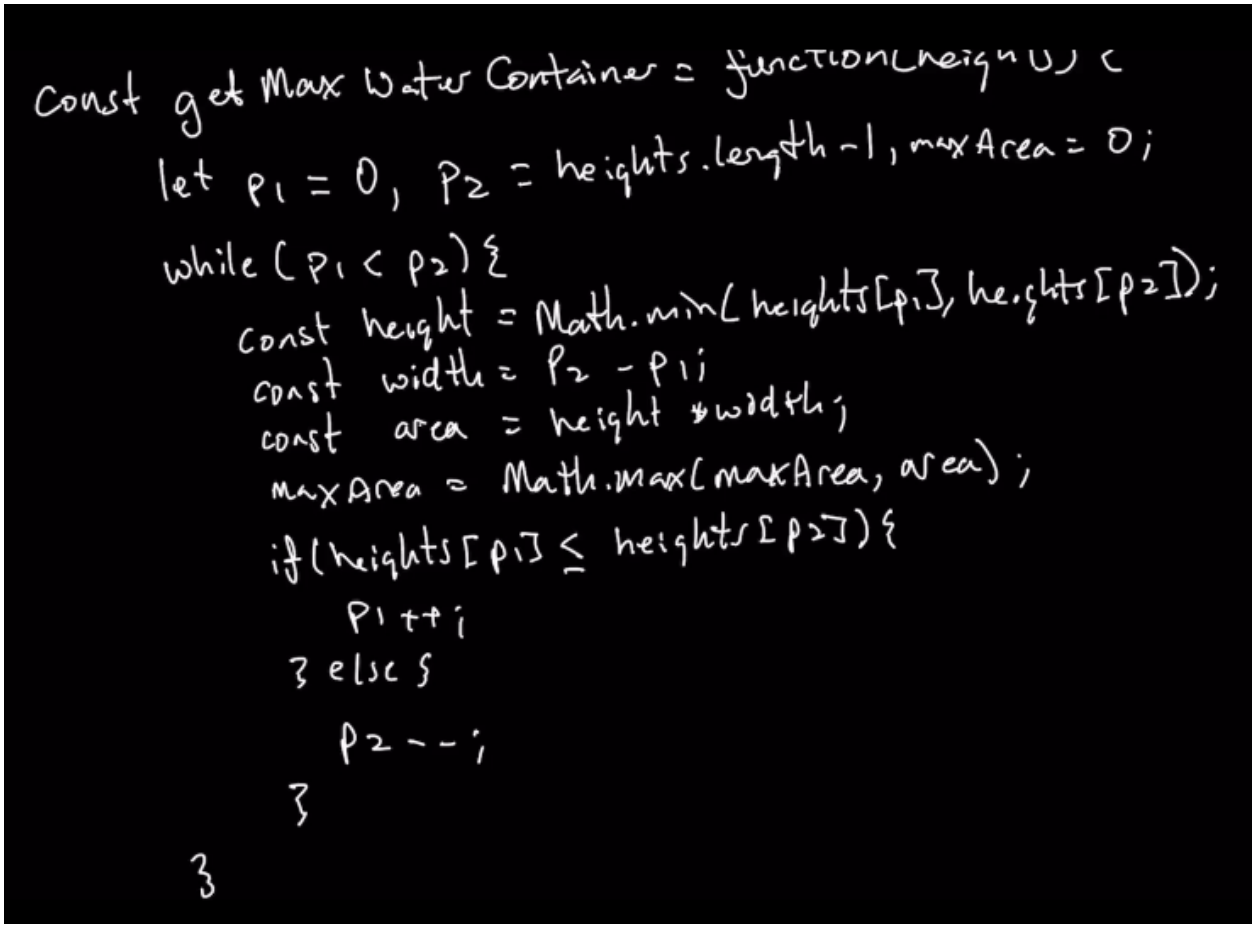
Divide the problem in rational steps

We know that the minimum value has impact

We notice a pattern/part important

Method: The two pointers

Use just a loop, but with reference changing to the pointers

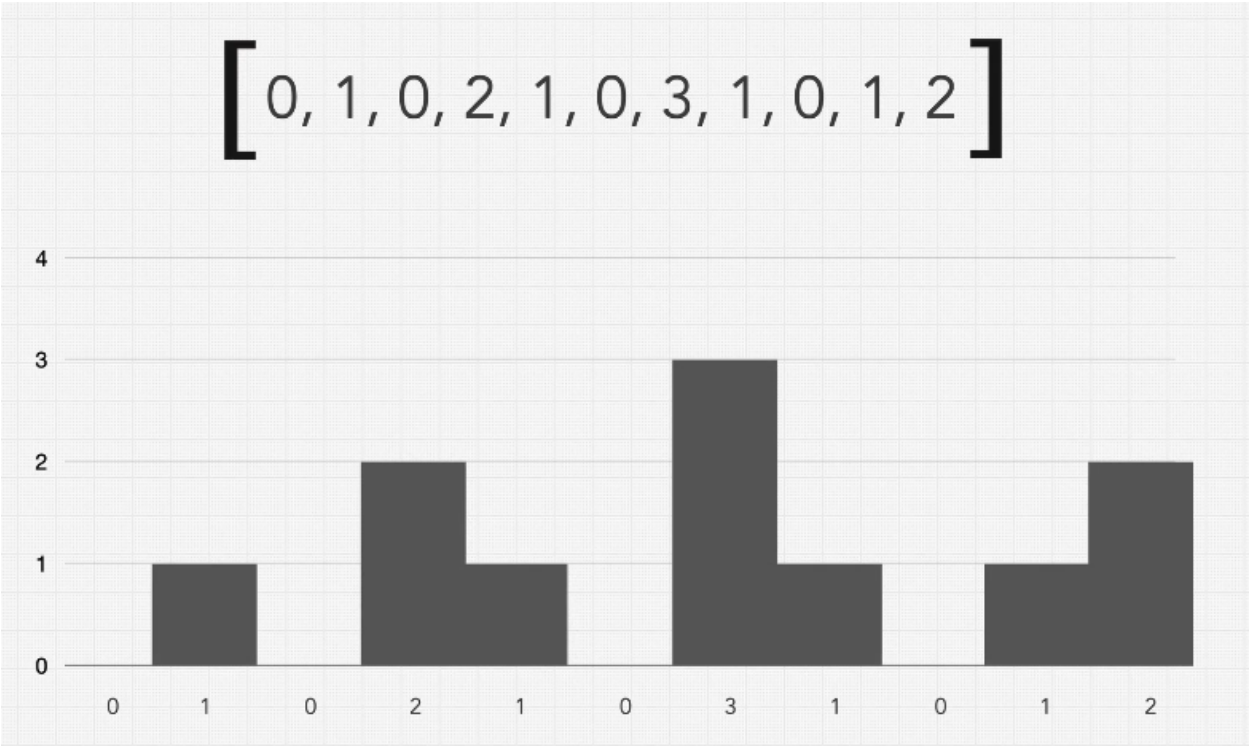


Then return the área

Trapping rainwater

Problem

Given an array of integers representing an elevation map where the width of each bar is 1, return how much rainwater can be trapped.



More useful test cases

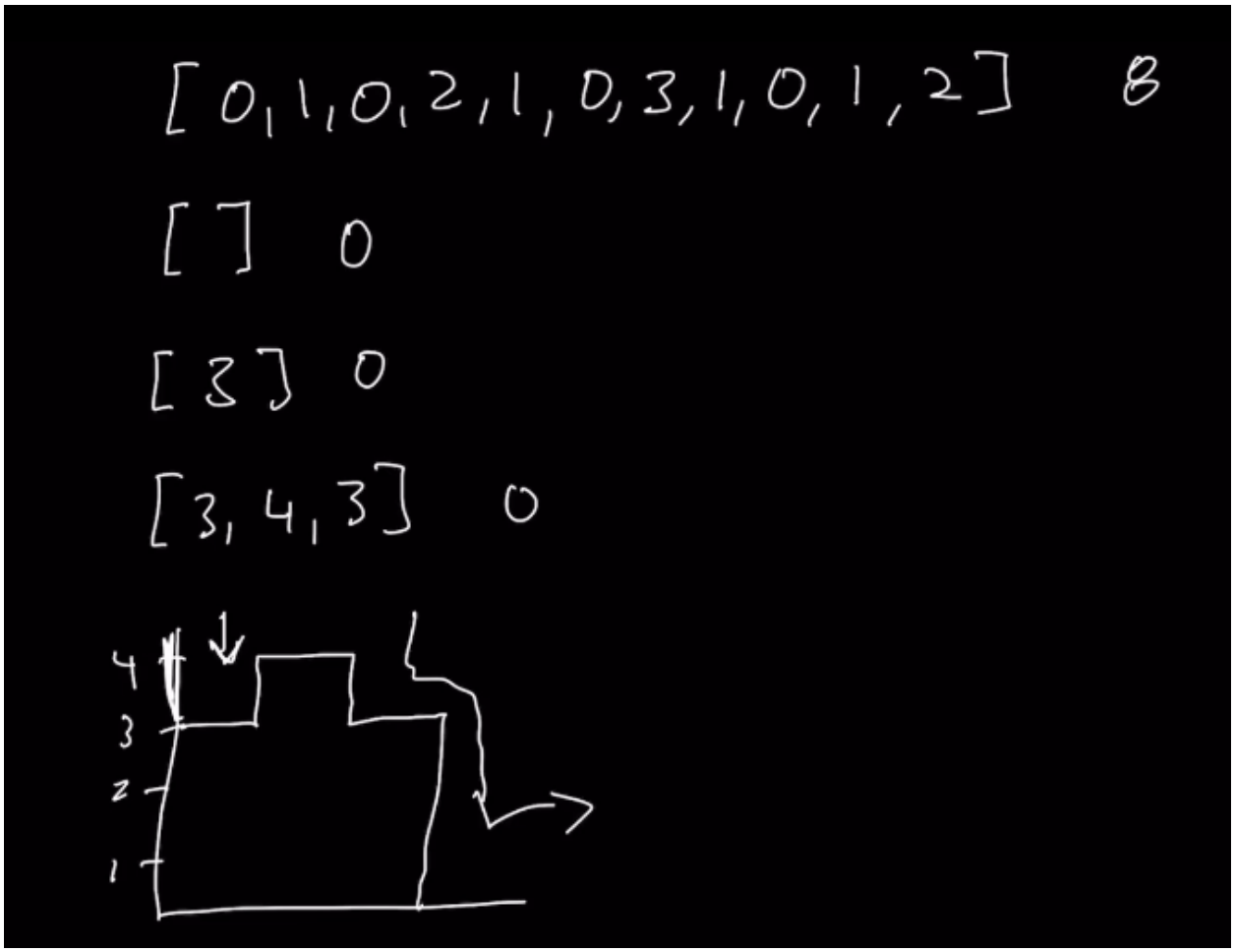
```
const getTrappedRainwater = function(heights) {
    let totalWater = 0;
    for(let p = 0; p < heights.length; p++) {
    let leftP = p, rightP = p, maxLeft = 0, maxRight = 0;
     while (leftP >= 0) {
        maxLeft = Math.max(maxLeft, heights[leftP]);
        leftP--;
     }
     while (rightP < heights.length ) {
        maxRight = Math.max(maxRight, heights[rightP]);
        rightP++;
     }
```

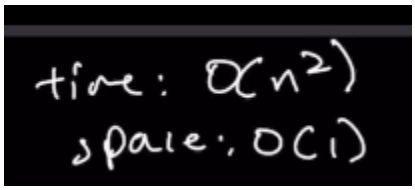
```
    while (rightP < heights.length ) {
        maxRight = Math.max(maxRight, heights[rightP]);
        rightP++;
    }
    const currentWater = Math.min(maxLeft, maxRight) - height[p];
    if(currentWater >= 0) {
        totalWater += currentWater;
    }
}
return totalWater.
```

This pointers methods calculates each piece of water

The performance is very bad

To increase Space complexity we look for variables that can store crucial things



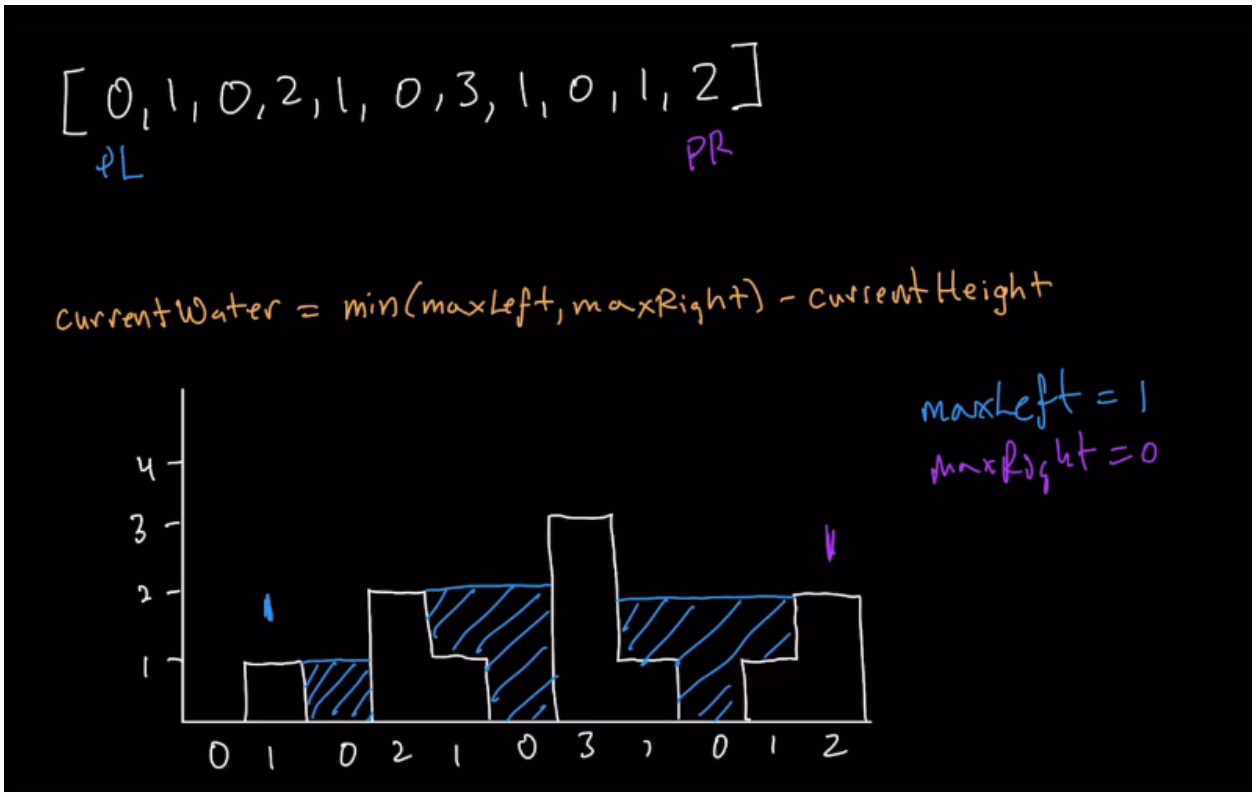


Method: The two pointers

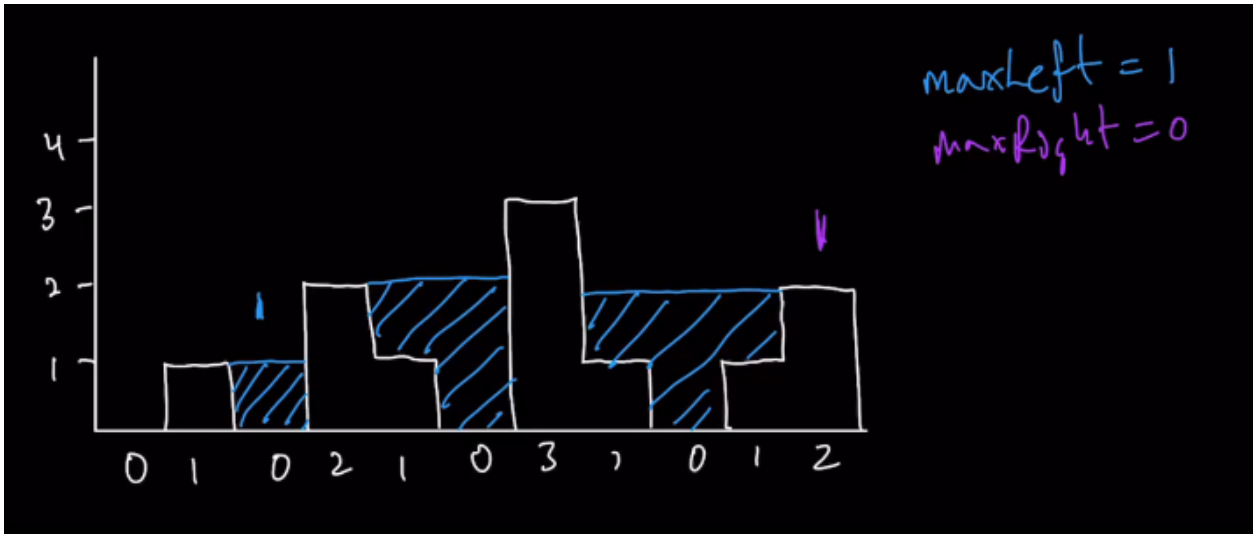
With two pointers in each side of array, we can store data in the path
We should think, when move each one by a condition

The pointers don’t necesarlty move at the same time

So what we can do?, if we have a pointer in wall and another pointer in wall we can’t measure the water because we don’t know what’s inside.

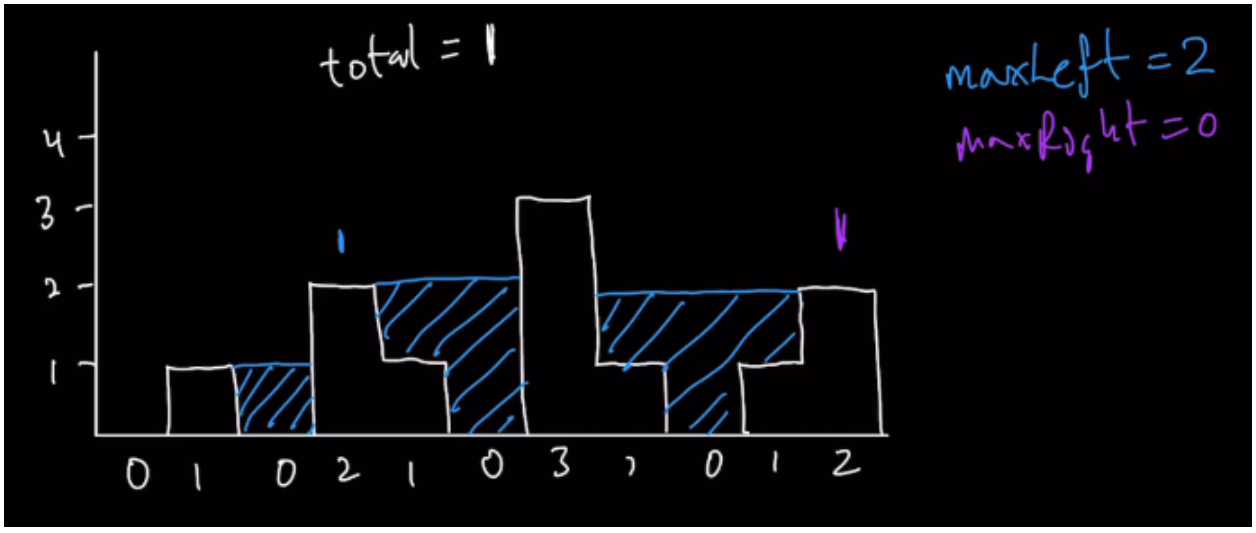


We move the min pointer and use maxleft to calculate the water in each point., maxleft change throu iterations



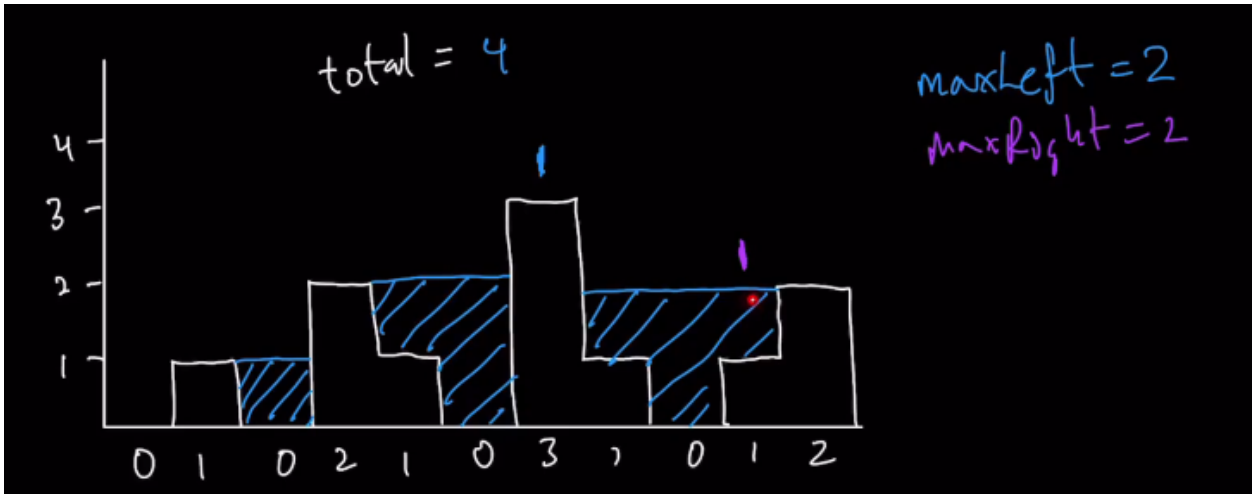
We subtract 1-0, and get area for this point, the max left only affects to moving point

Ya que ya sabemos que el dos es almenos mayor que 1

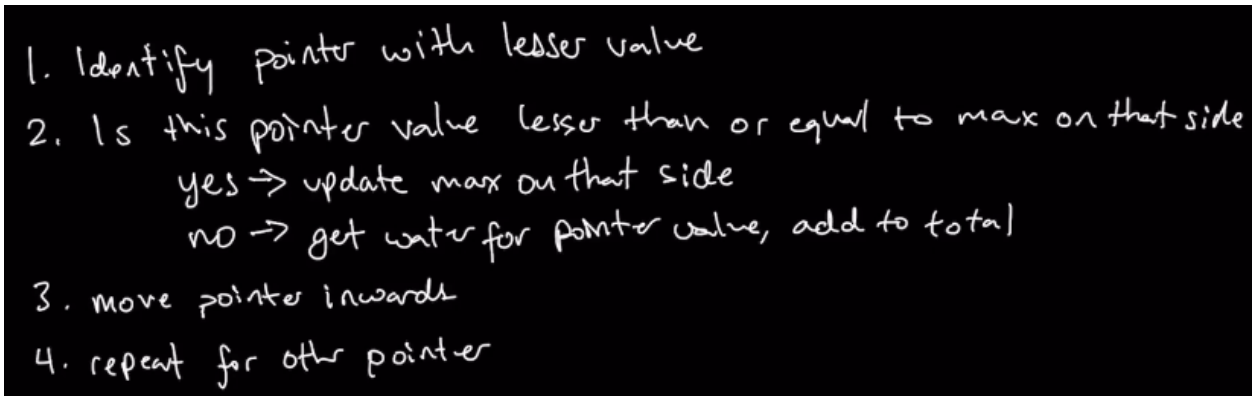


Update maxleft or calculate water, not both

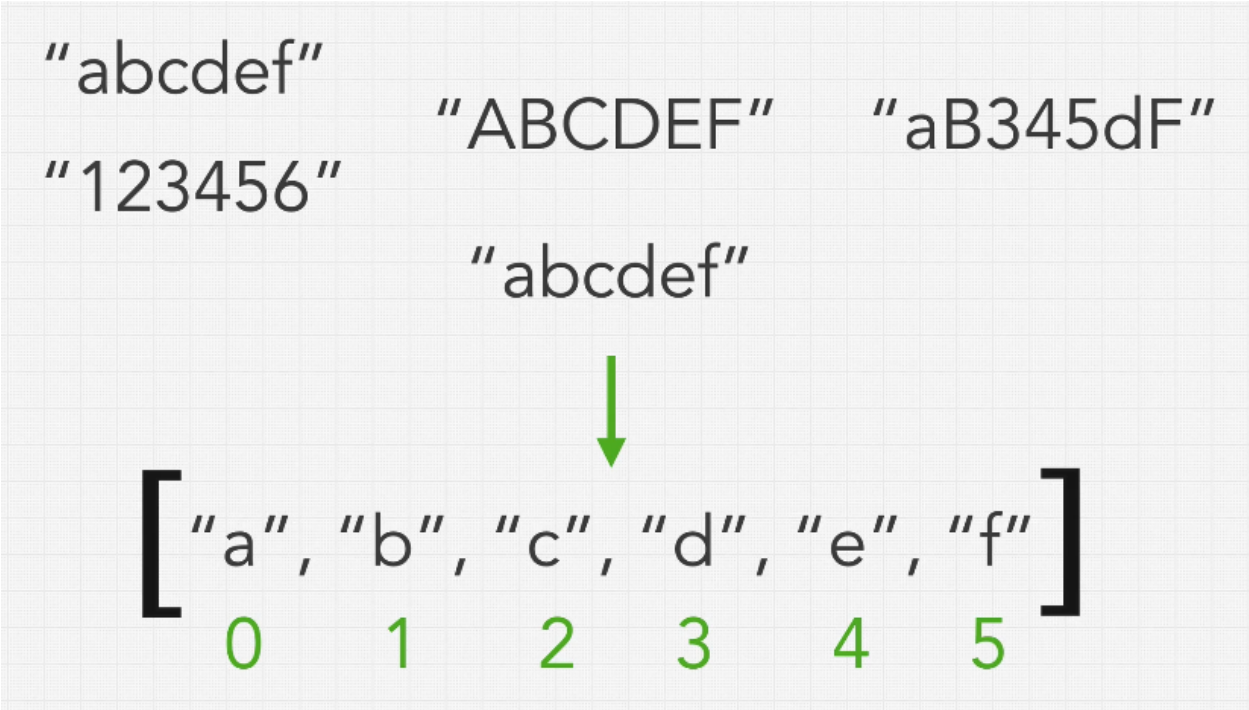
The pointer in the right give us enough information



Write it as algorithm



Strings

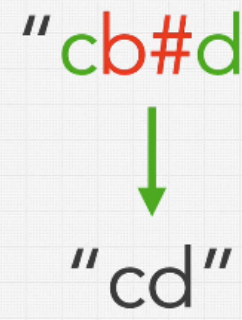


The order usually matter

Typed out

Problem

Given two strings S and T, return if they equal when both are typed out. Any '#' that appears in the string counts as a backspace.



What happens when two #'s appear beside each other?

Delete the two values before the first #.

What happens to # when there is no character to remove?

It deletes nothing then, just like backspace would.

Are two empty strings equal to each other?

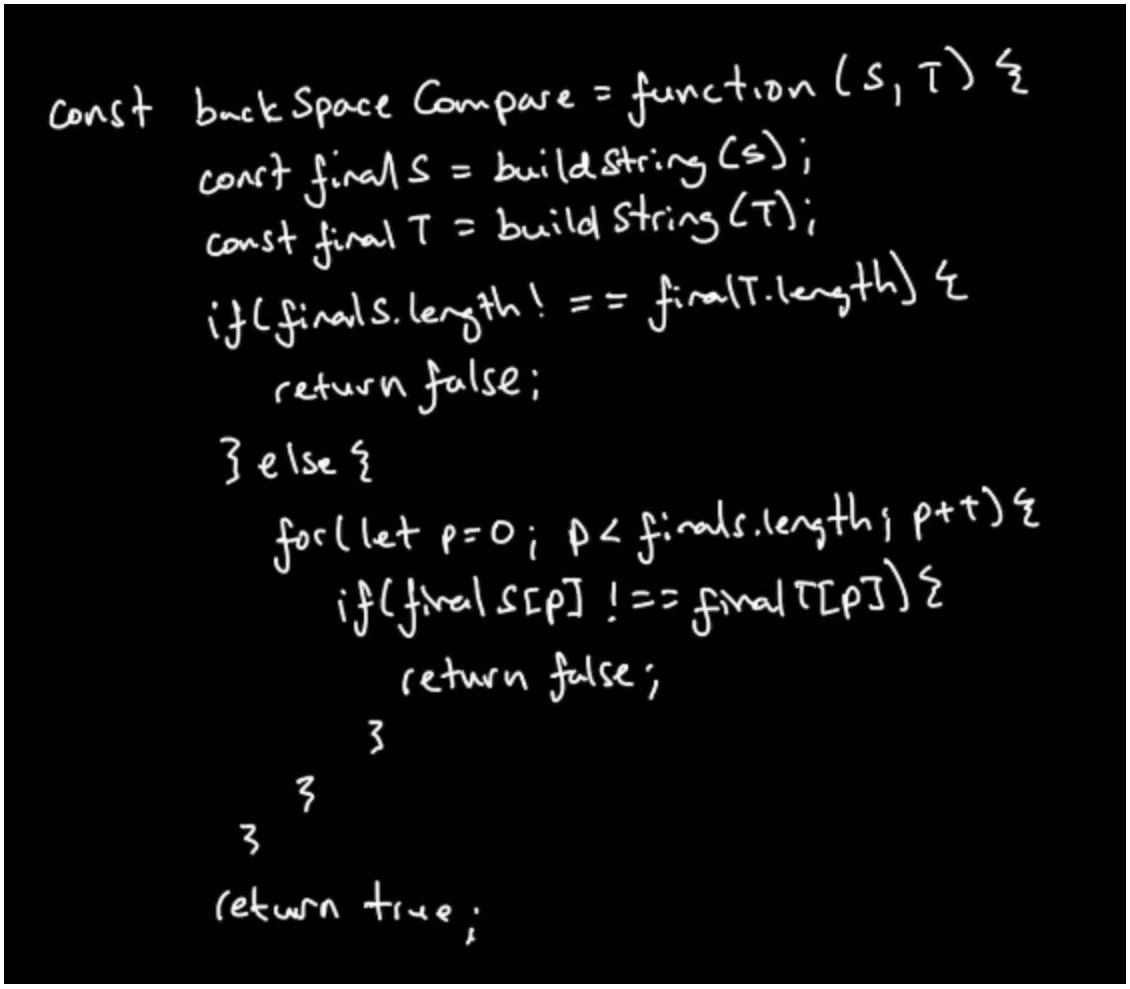
Yes, consider two empty strings as equal.

Does case sensitivity matter?

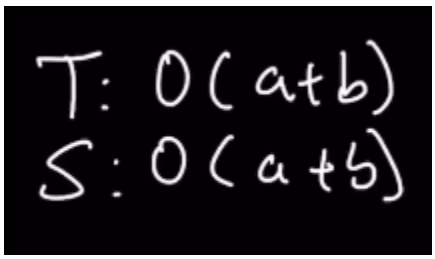
Yes it does, "a" does not equal "A".

S: "ab#z"	T: "az#z"	True
S: "abc#d"	T: "acc#c"	False
S: "x#y#z#"	T: "a#"	True
S: "a###b"	T: "b"	True
S: "Ab#z"	T: "ab#z"	False

```
const buildString = function(string){
    const builtArray = [];
    for(let p = 0; p < string.length; p++){
        if(string[p] !== '#'){
            builtArray.push(string[p]);
        }else{
            builtArray.pop();
        }
    }
    return builtArray;
}
```



```
const backSpaceCompare = function(S, T) {
    const finalS = buildString(S);
    const finalT = buildString(T);
    if(finalS.length !== finalT.length) {
        return false;
    } else {
        for(let p=0; p<finals.length; p++) {
            if(finalS[p] !== finalT[p]) {
                return false;
            }
        }
    }
    return true;
```



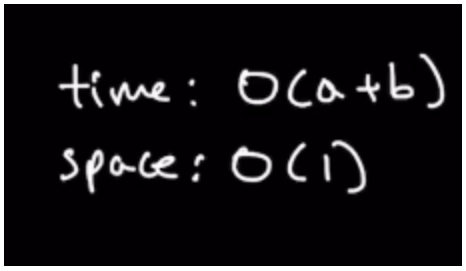
```
T: O(a+b)
S: O(a+b)
```

Method: The two pointers

```
const backSpaceCompare = function(S, T) {
    let p1 = S.length-1, p2 = T.length-1;
    while (p1 >= 0 || p2 >= 0) {
        if( S[p1] === "#" || T[p2] === "#") {
            if(S[p1] === "#" ){
                let backcount = 2;
                while(backCount > 0){
                    p1--;
                    backCount--;
                    if(S[p1] === "#"){
                        backCount = backcount +2;
                    }
                }
```

```
if( T[p2] === "#") {
    let backCount = 2;
    while (backCount > 0) {
      p2--;
      backCount--;
      if(T[p2] === "#") {
        backCount = backCount +2;
      }
    }
}
```

```
      }else {
        if( S[p1] !== T[p2]) {
            return false;
        } else {
          p1--;
          p2--;
        }
    }
  return true;
}
```



Longest substring without repeating characters

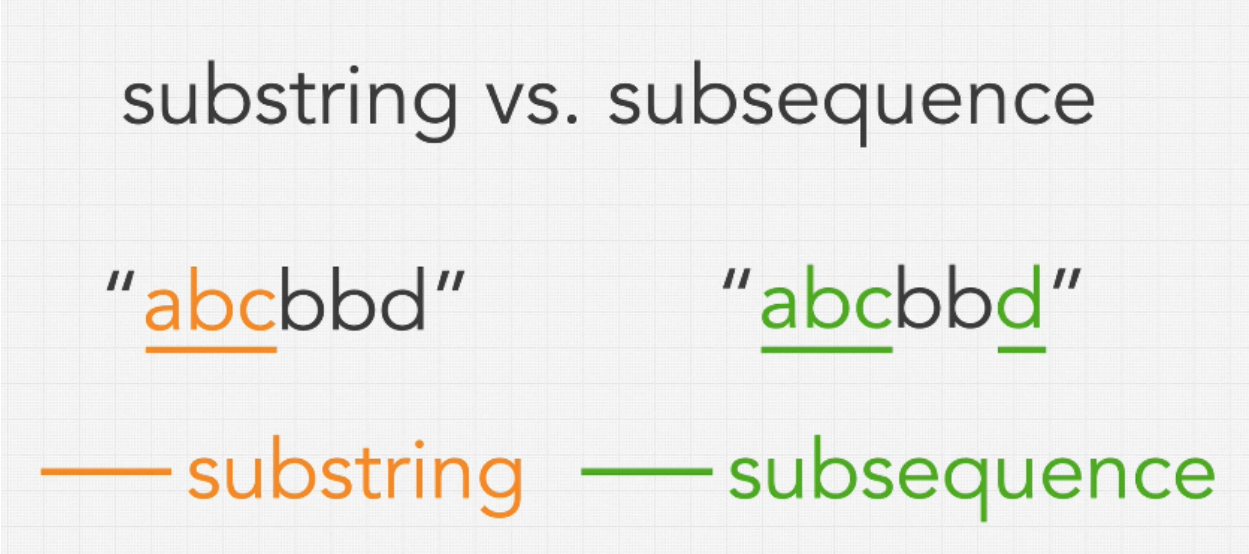
Problem

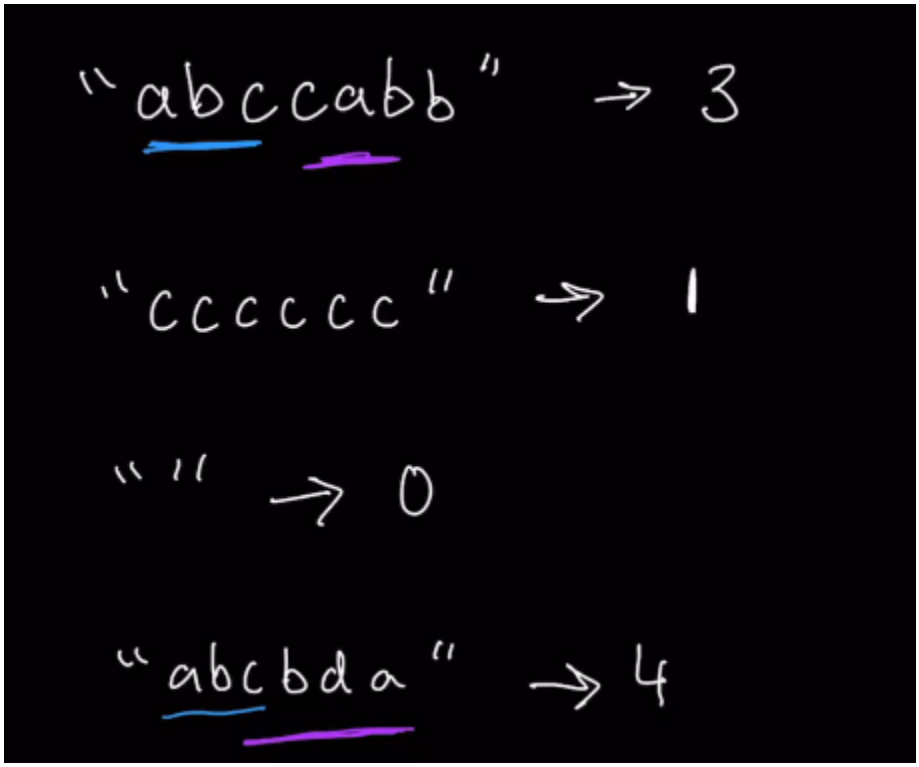
Given a string, find the length of the longest substring without repeating characters.

Given a string, find the length of the longest substring without repeating characters.

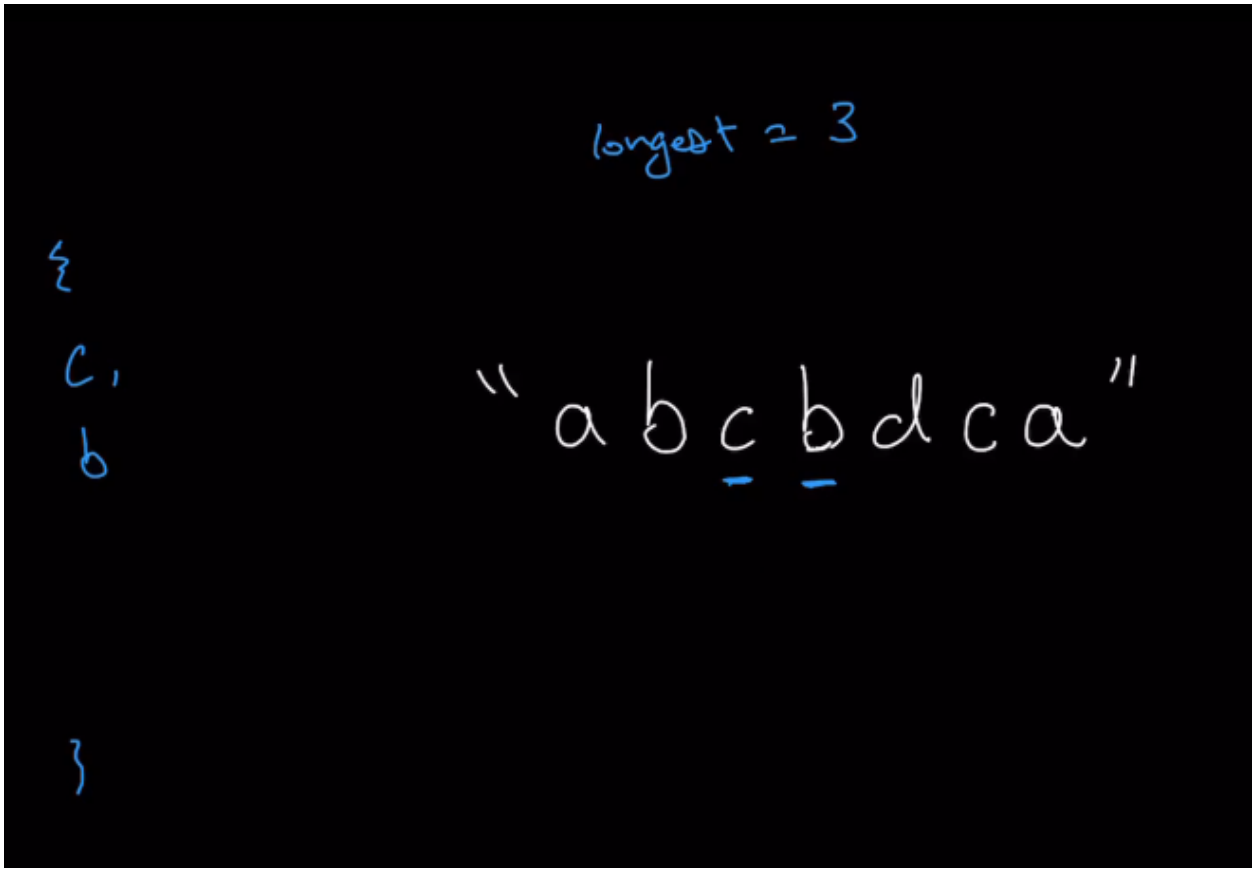
"abccabb"

3





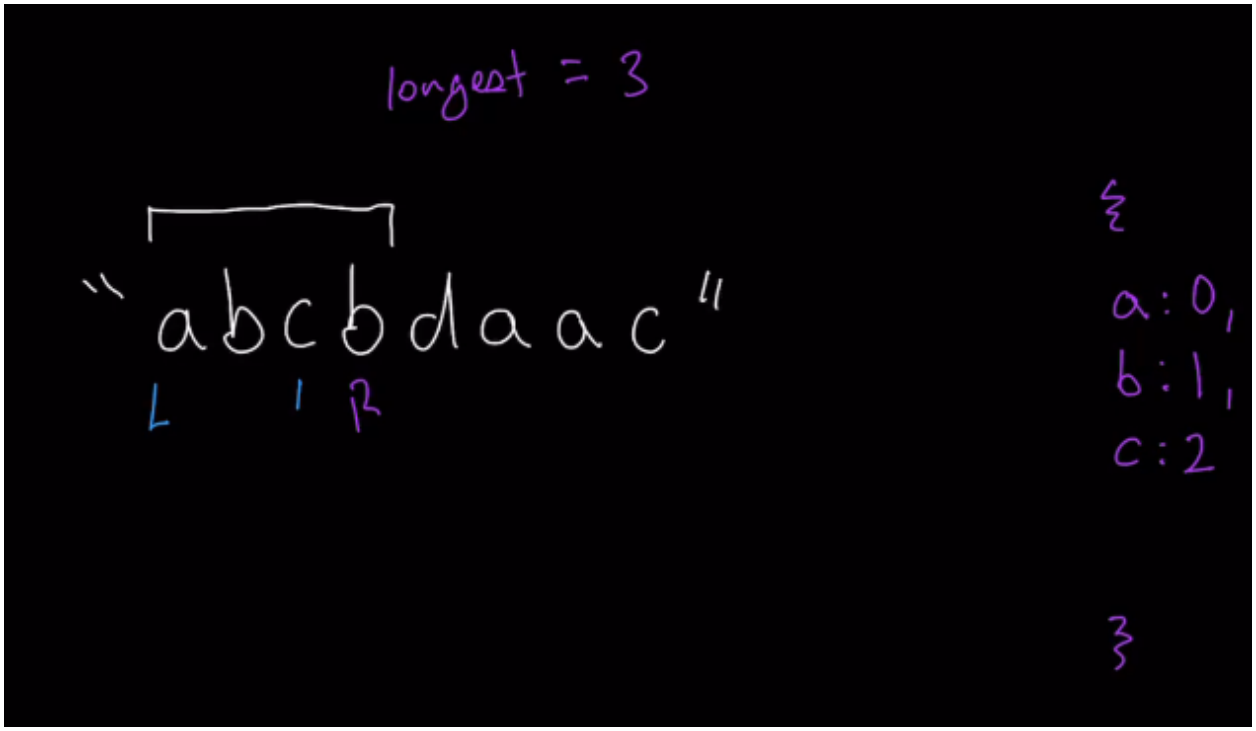
Normal solution



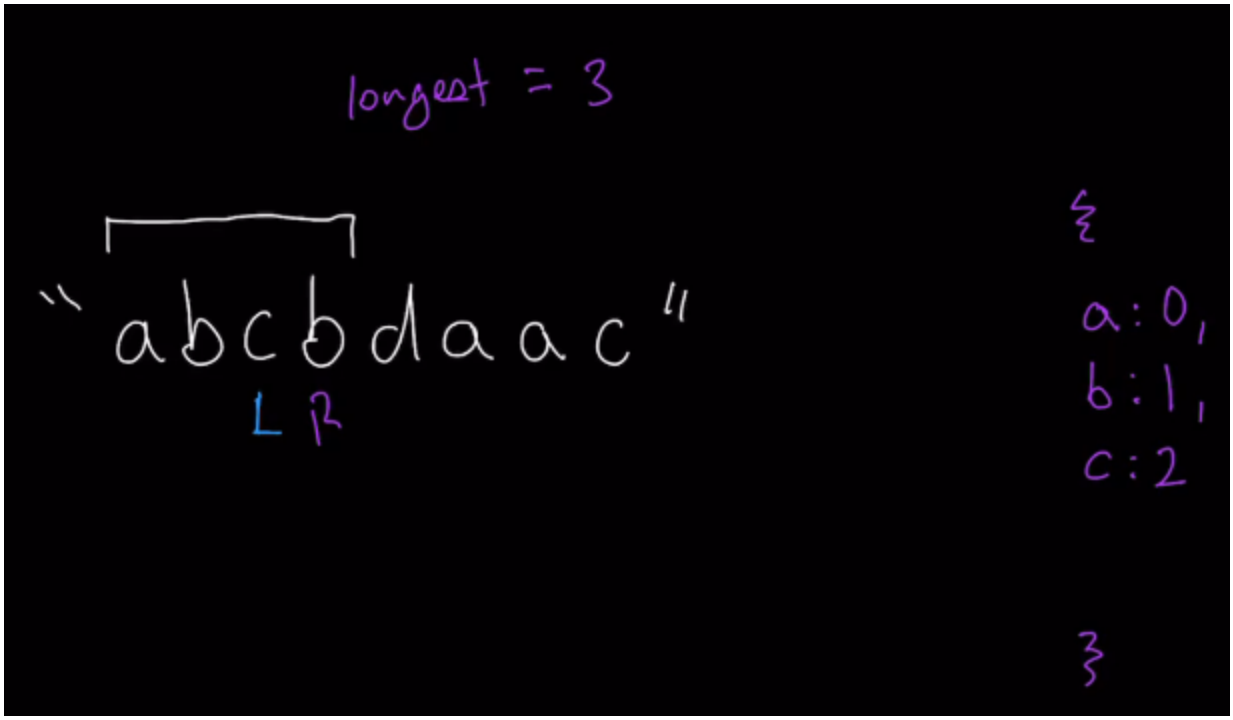

```
const lengthOfLongestSubstring = function(s) {
    if(s.length <= 1) return s.length;

    let longest = 0;

    for(let left = 0; left < s.length; left++) {
      let seenChars = {}, currentLongest = 0;

      for(let right = left; right < s.length; right++) {
        const currentChar = s[right];

        if(!seenChars[currentChar]) {
          currentLongest++;
          seenChars[currentChar] = true;
          longest = Math.max(longest, currentLongest);
        } else {
          break;
        }
      }
    }
```

Complexity is $O(n^2)$ ans $S(n^2)$

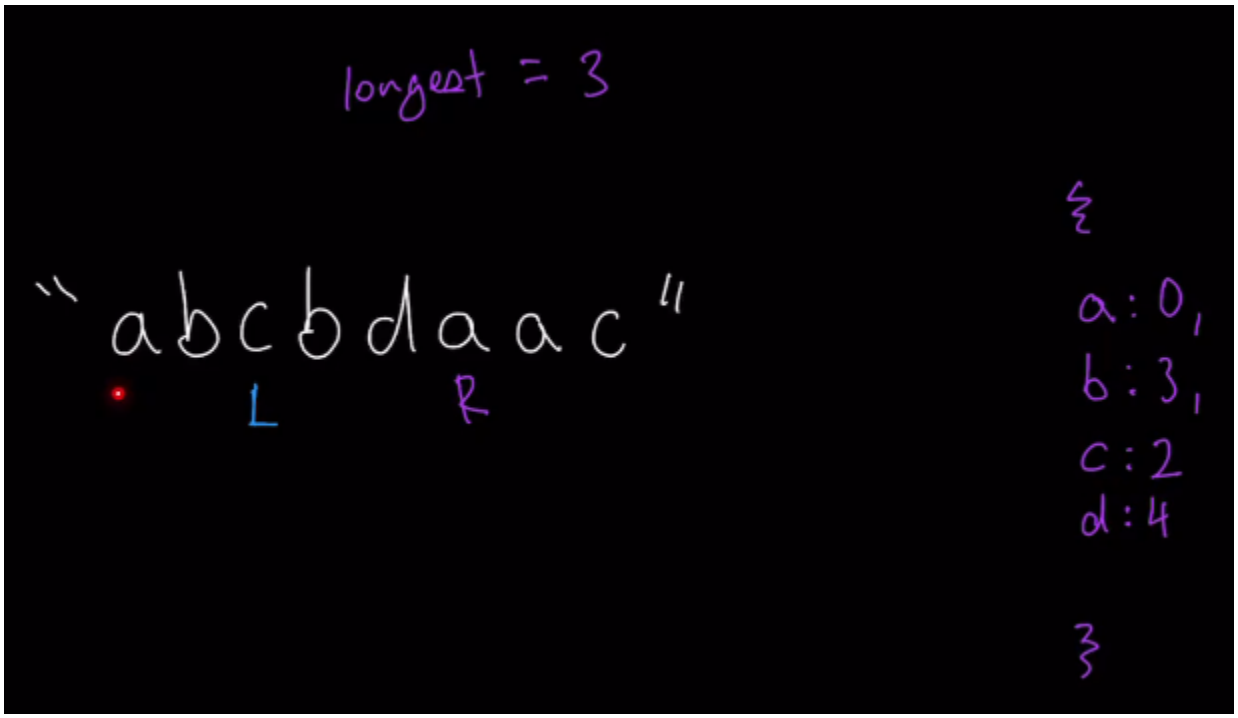
Method: sliding window

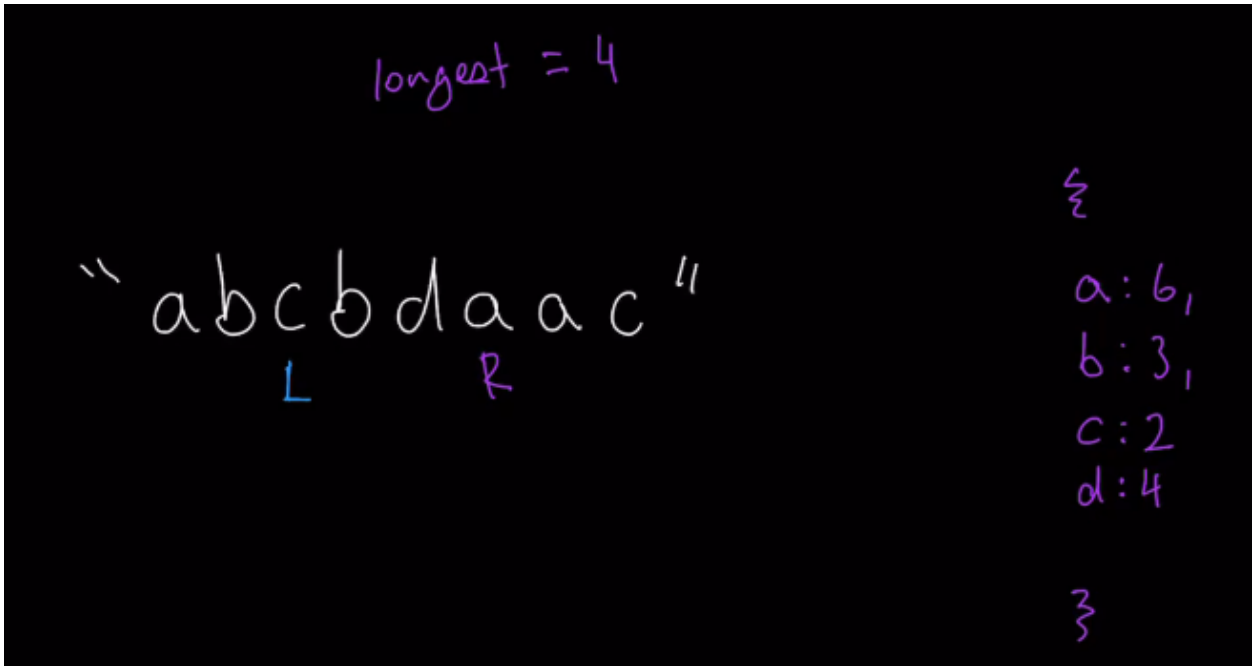


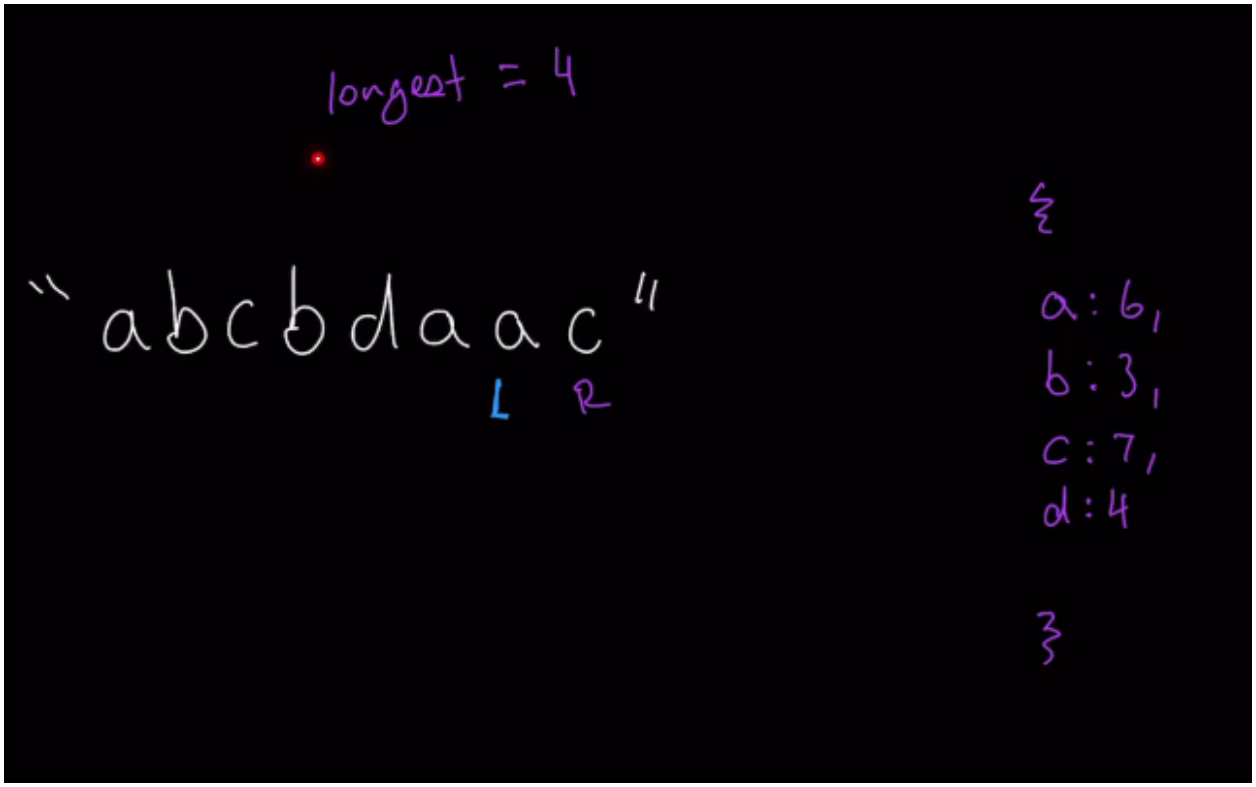
We can derivate a conclusion, the elements after first b are unique, and before are elements with b so will not work



The index is 0 of a is less than L so we haven't seen already




longest = 4
"abcbdaac"
L
R
{
a: 6,
b: 3,
c: 2
d: 4
}


longest = 4
"abcbdaac"
L
R
{
a: 6,
b: 3,
c: 7,
d: 4
}

```
*/
const lengthOfLongestSubstring = function(s){
    if(s.length <= 1) return s.length;

    const seen = {};
    let left = 0, longest = 0;

    for(let right = 0; right <s.length; right++){
        const currentChar = s[right];
        const previouslySeenChar = seen[currentChar];

        if(previouslySeenChar >= left){
          left = previouslySeenChar + 1;
        }

        seen[currentChar] = right;

        longest = Math.max(longest, right - left + 1);
    }

    return longest;
};
```

Runtime: 80 ms, faster than 91.32% of JavaSc
Longest Substring Without Repeating Characters.

Memory Usage: 37.6 MB, less than 96.46% c
submissions for Longest Substring Without Repeati

Next challenges:

Longest Substring with At Most K Distinct Charac

Subarrays with K Different Integers

Show off your acceptance:

```
*/
const lengthOfLongestSubstring = function(s) {
    if(s.length <= 1) return s.length;

    const seen = new Map();
    let left = 0, longest = 0;

    for(let right = 0; right < s.length; right++) {
        const currentChar = s[right];
        const previouslySeenChar = seen.get(currentChar);

        if(previouslySeenChar >= left) {
          left = previouslySeenChar + 1;
        }

        seen.set(currentChar, right);
```

Each language have a specializaed data structure, in javascript using object has a really poor performance agains Map class

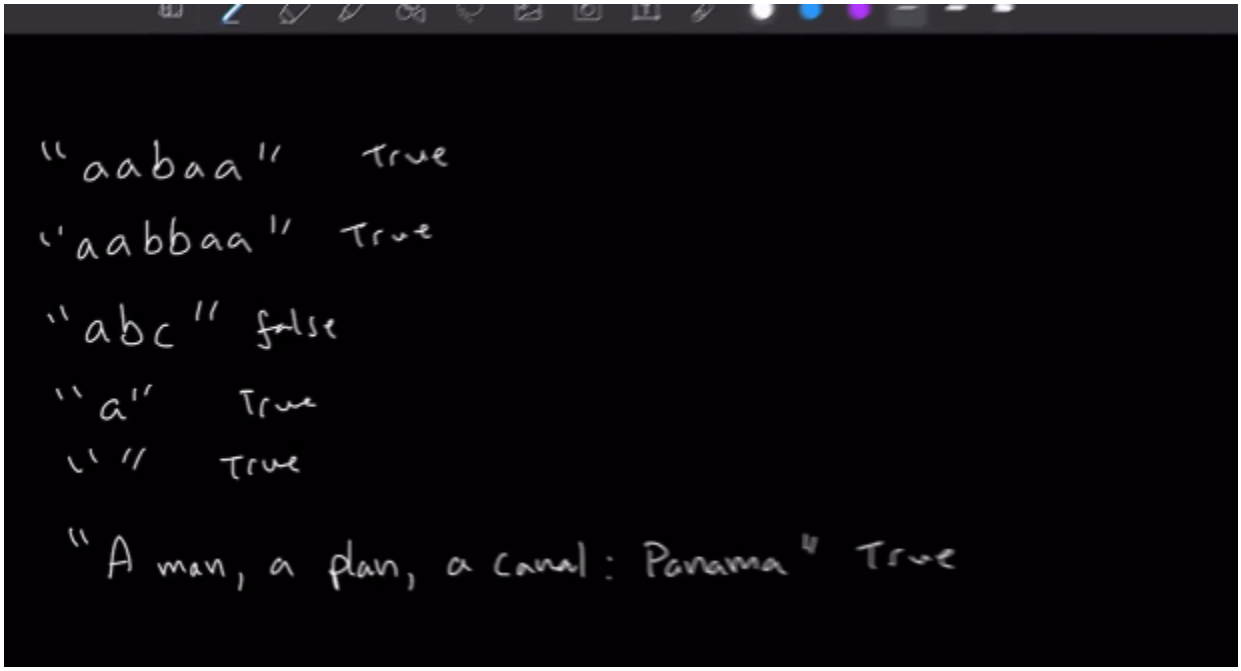
Is palindrome

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring case sensitivity.

"A man, a plan, a canal: Panama"

↓

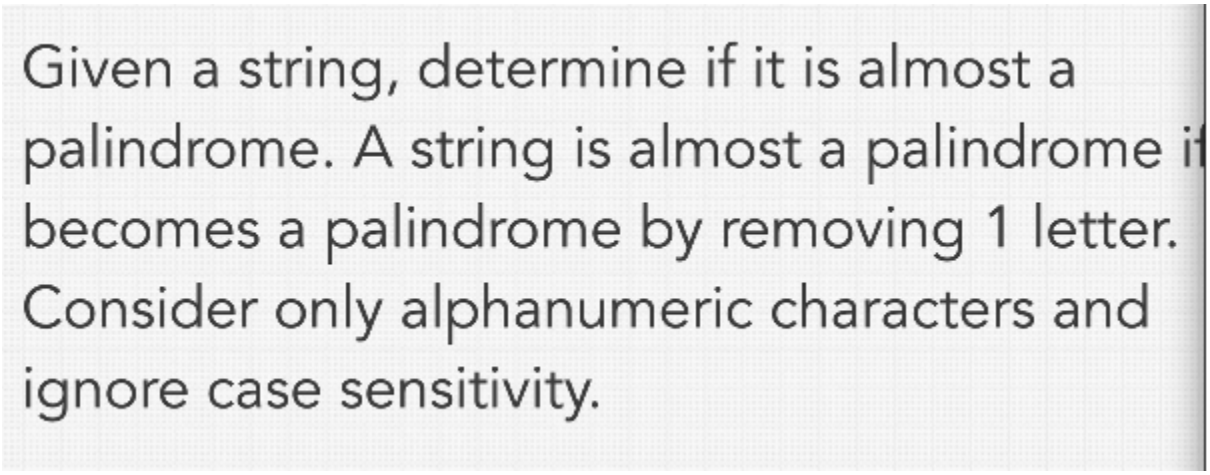
"amanaplanacanalpanama"

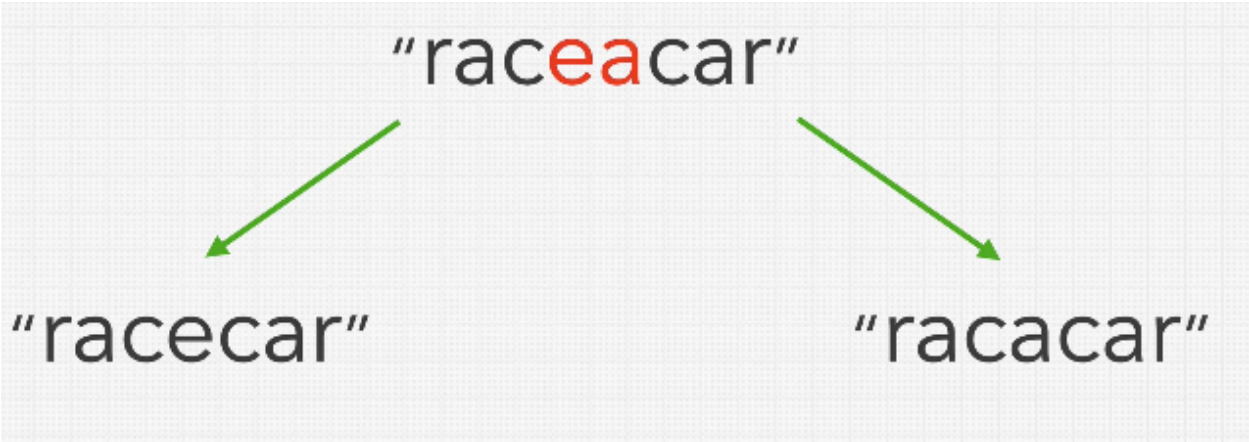


Method: The two pointers

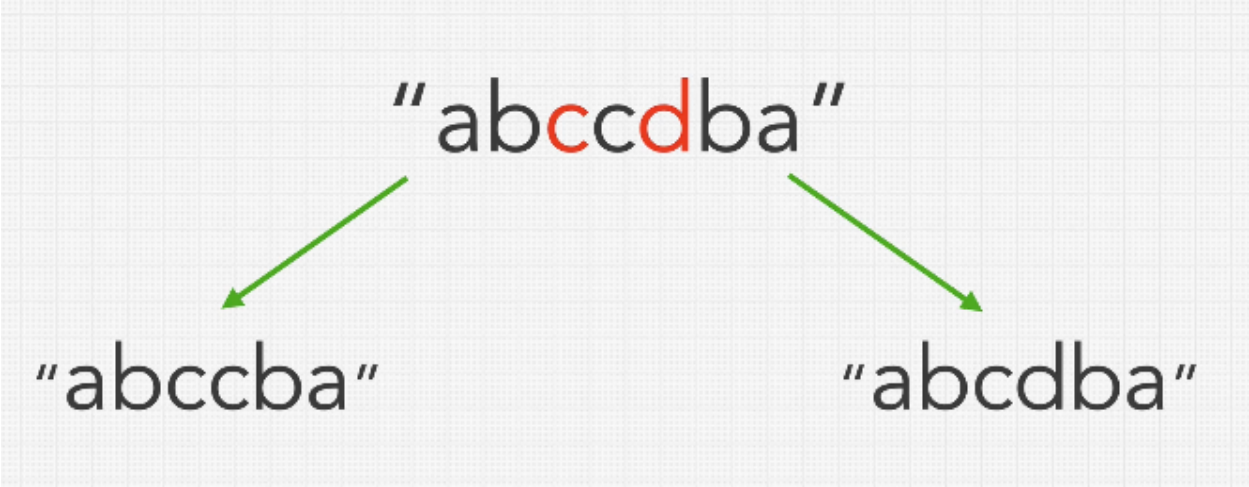
```
const isValidPalindrome = function(s) {
    s = s.replace(/[^A-Za-z0-9]/g, '').toLowerCase();

    // initialize left/right pointers at start and end of
    string respectively
    let left = 0; right = s.length - 1;

    // loop through string characters while comparing them,
    then move the pointers closer to the center
    while(left < right) {
        if(s[left] !== s[right]) {
            return false
        }

        left++;
        right--;
    }

    return true;
};
```

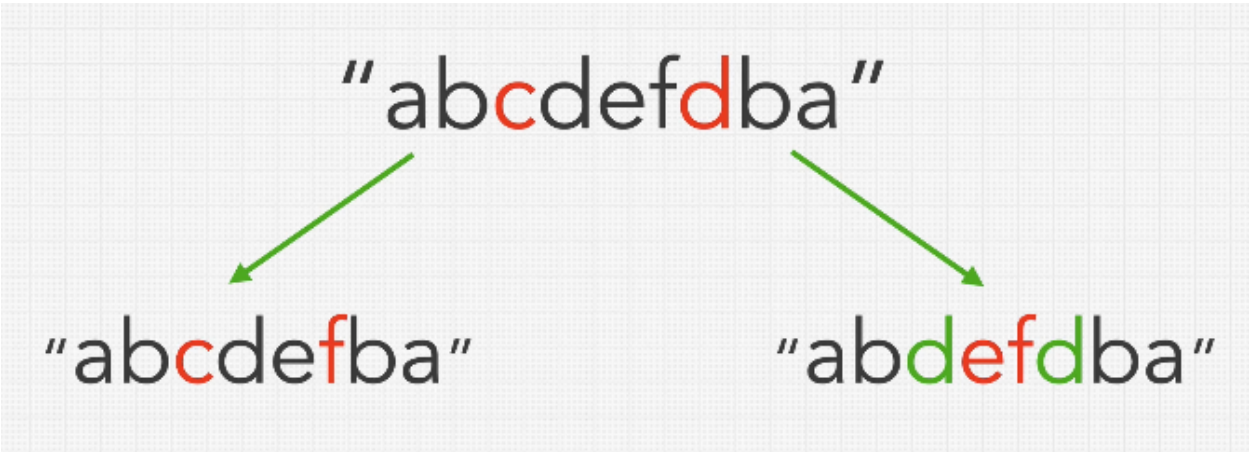
Almost a palindrome

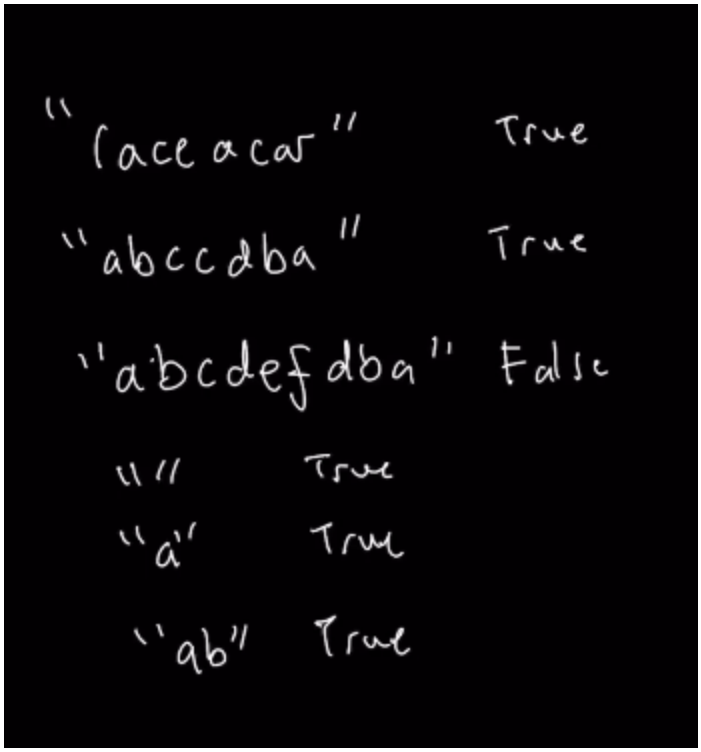




We can remove either the e or a







Method: Two pointers

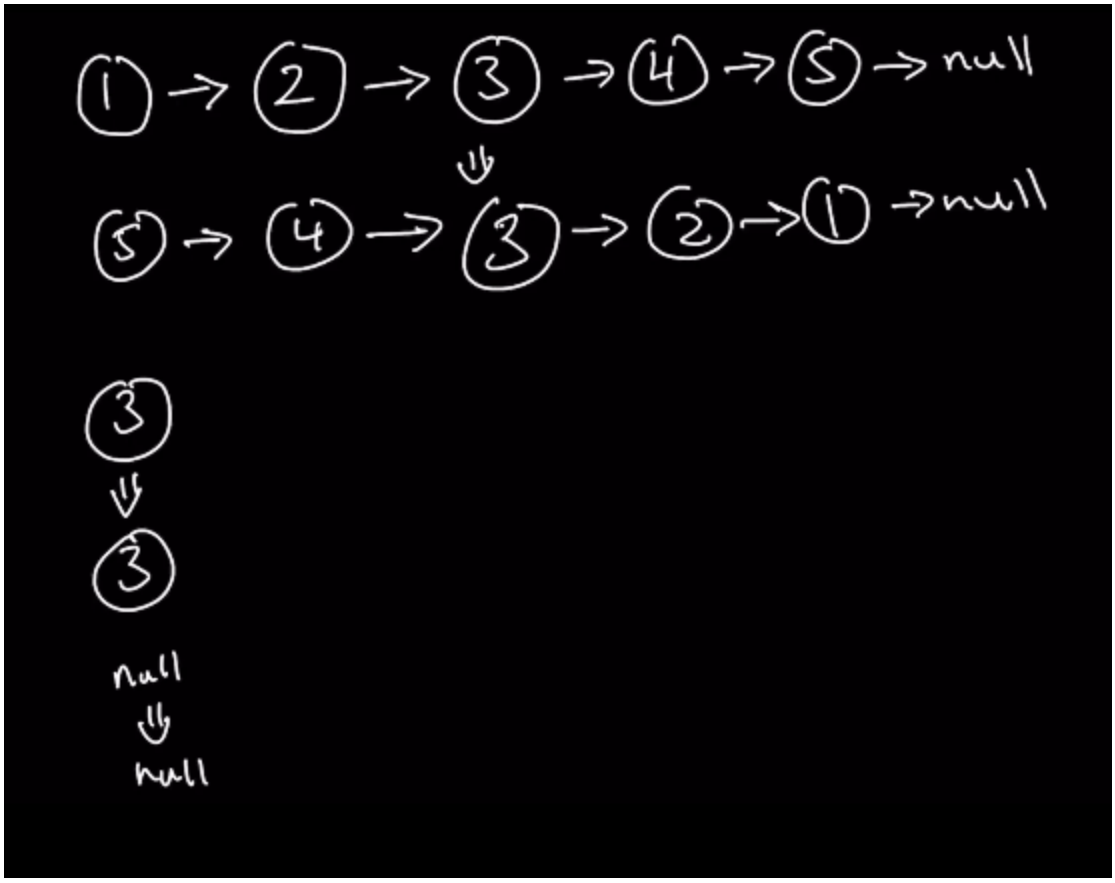
With brute force

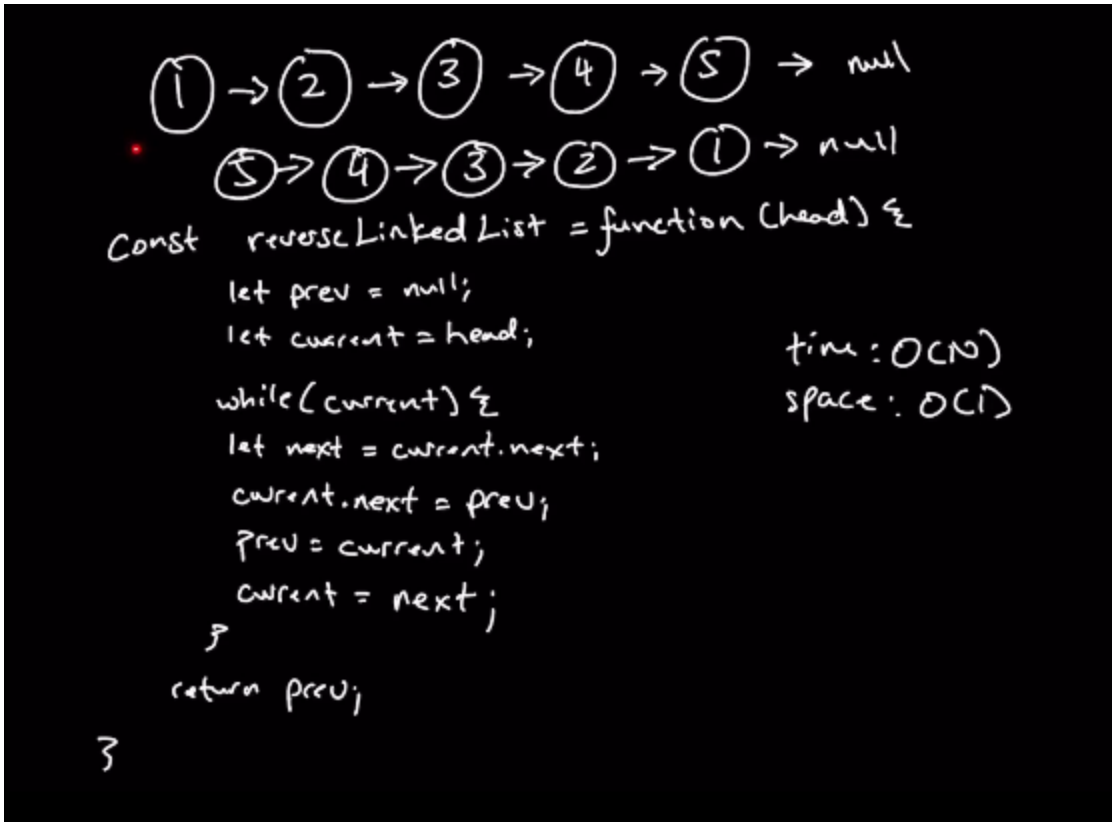
We solve as a subproblems, using two pointers

It can be optimized by not sending two possible palindromes but comparing what is to the right of p1 and what is to the left of p2.

Linked list

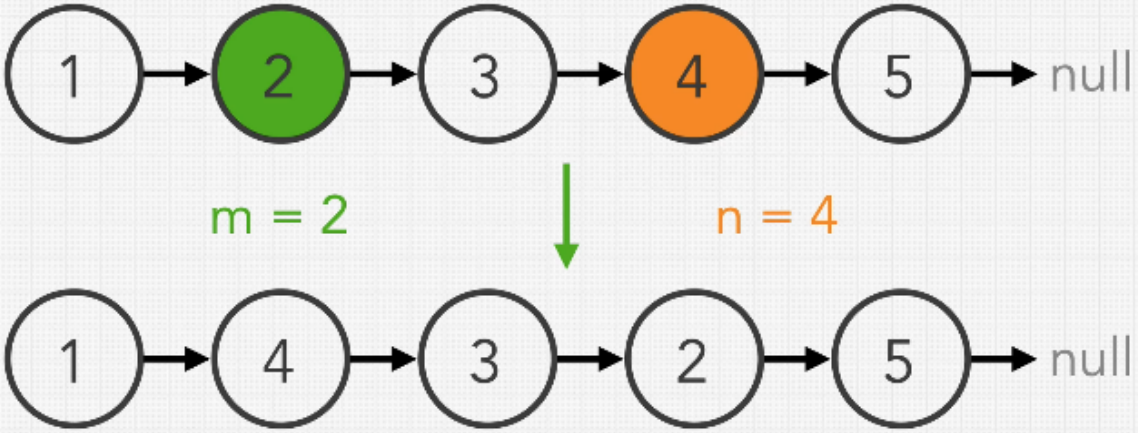
Given a linked list return it in reverse

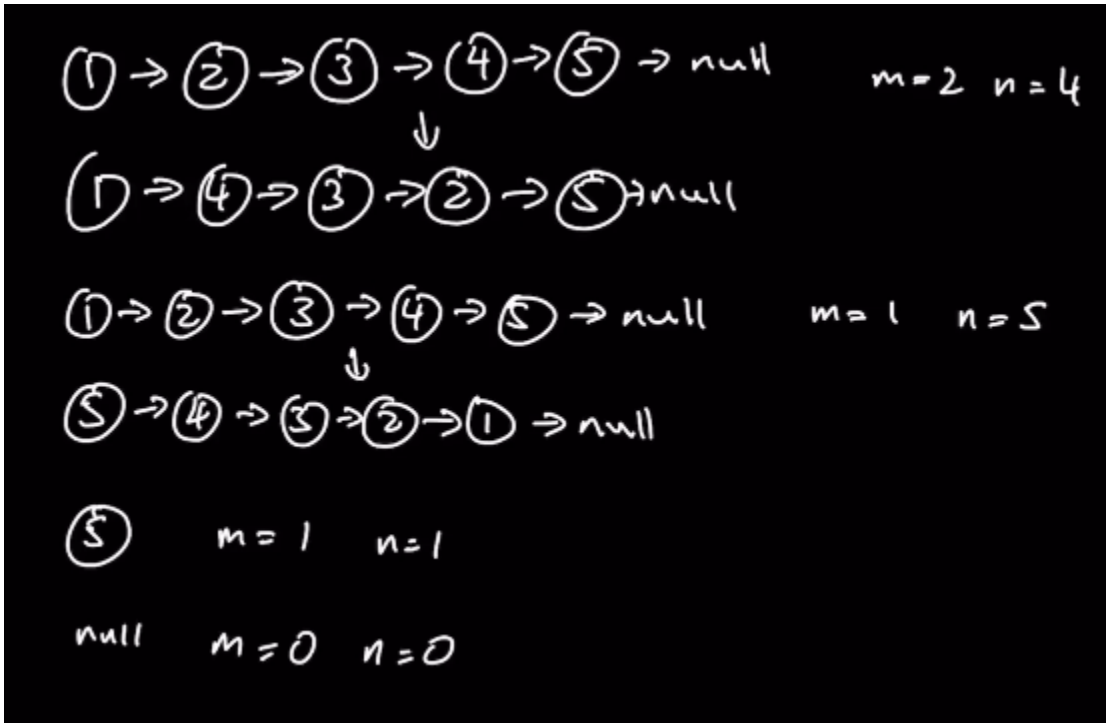

1 → 2 → 3 → 4 → 5 → null
5 → 4 → 3 → 2 → 1 → null
3
3
null
null

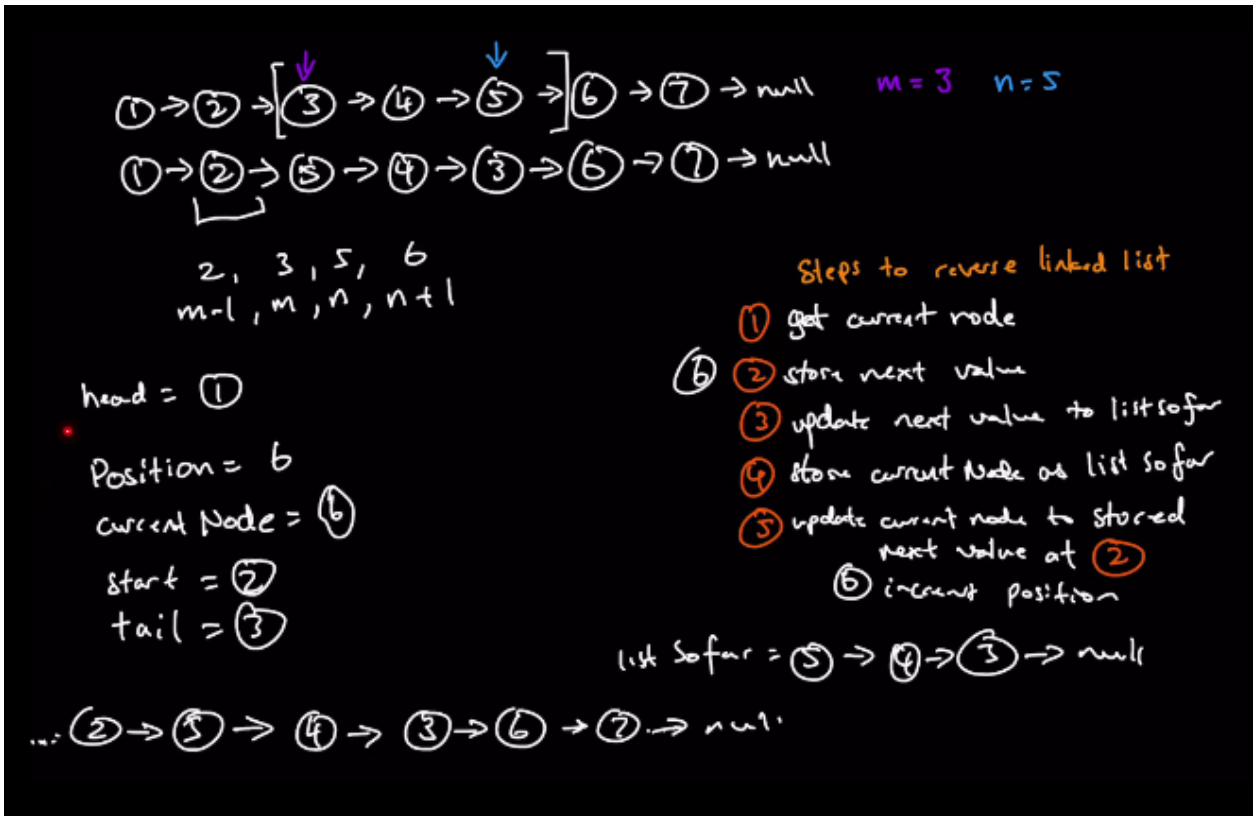


M N Revelsals

Given a linked list and numbers *m* and *n*, return it back with only positions *m* to *n* in reverse.





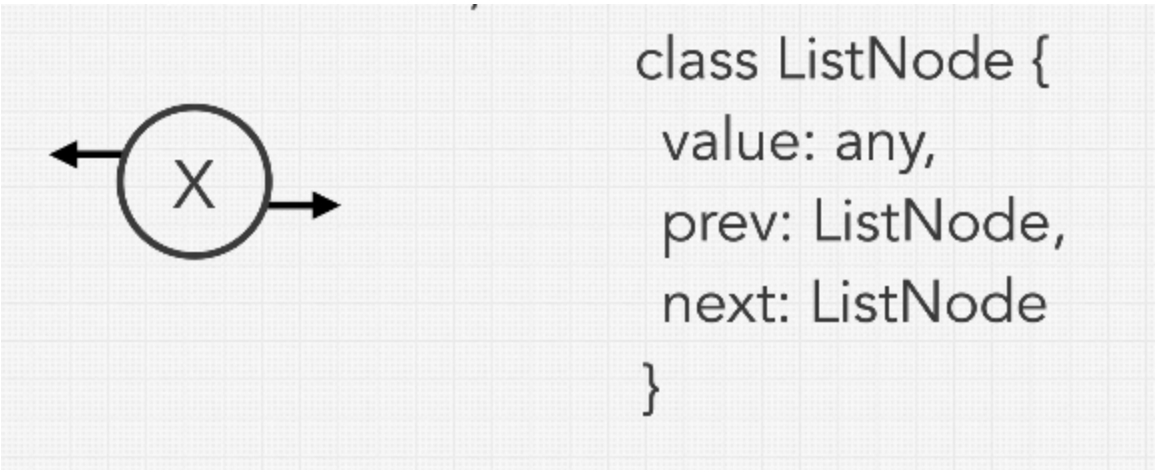


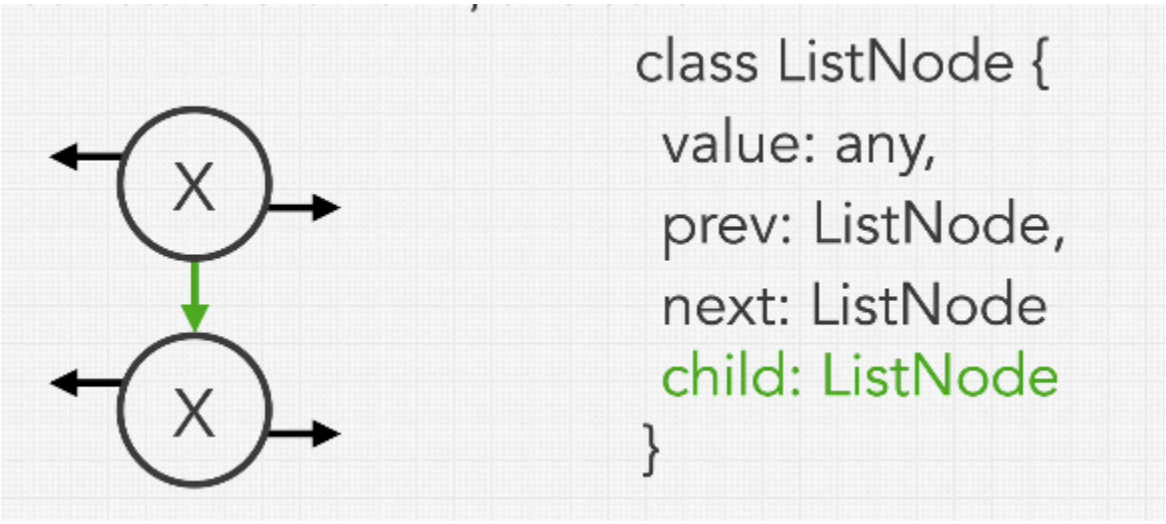
Doubly linked lists

Merge Multi-Level

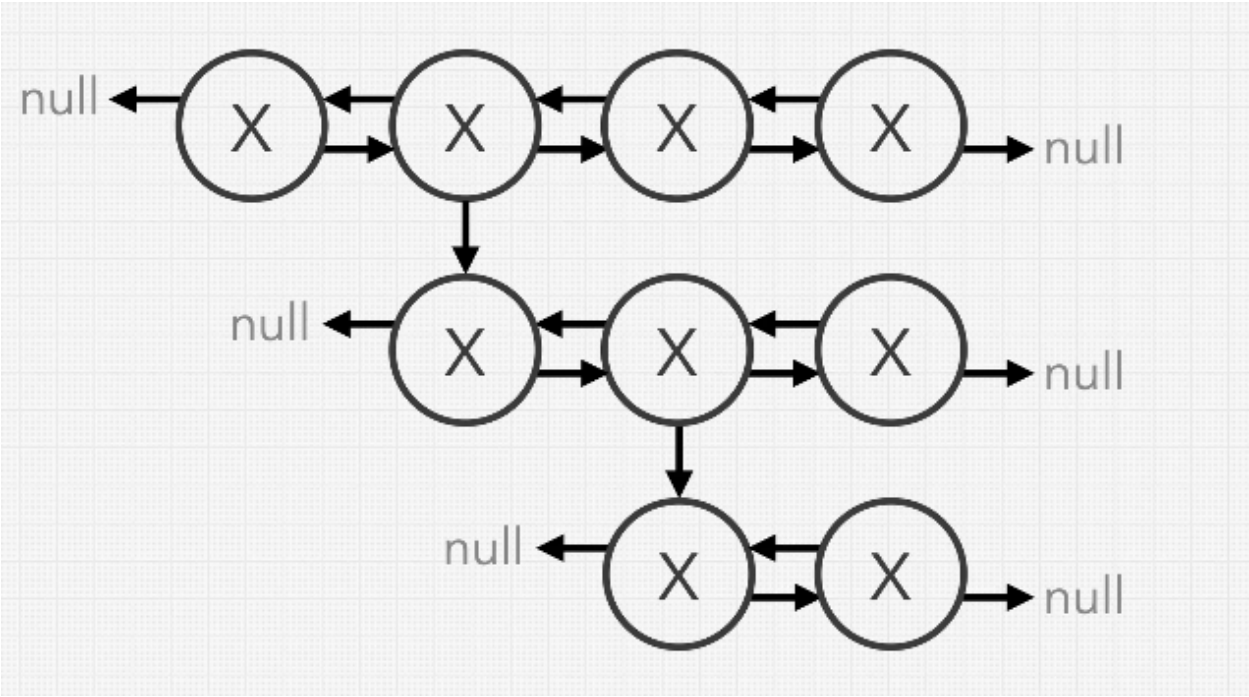
Given a doubly linked list, list nodes also have a child property that can point to a separate doubly linked list. These child lists can also have one or more child doubly linked lists of their own, and so on.

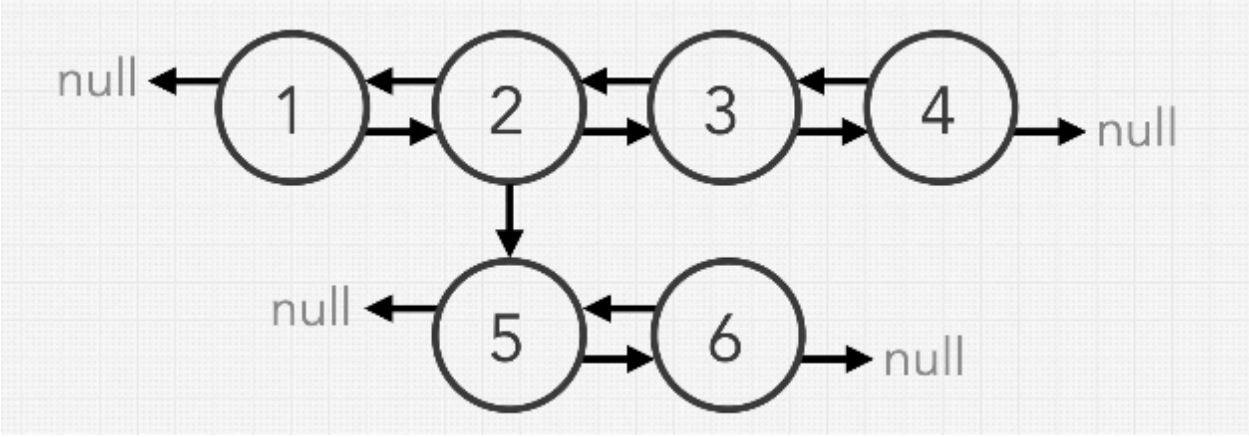
Return the list as a single level flattened doubly linked list.

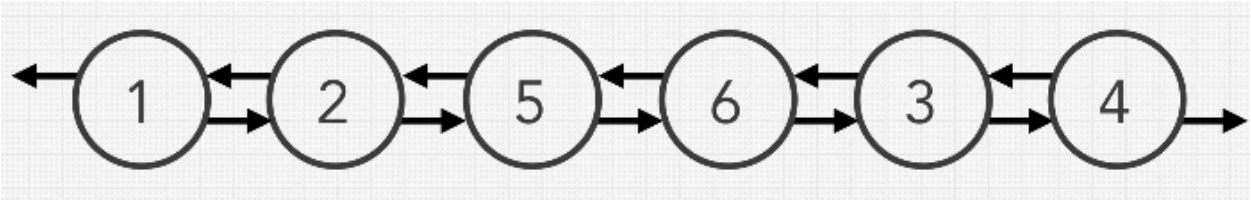


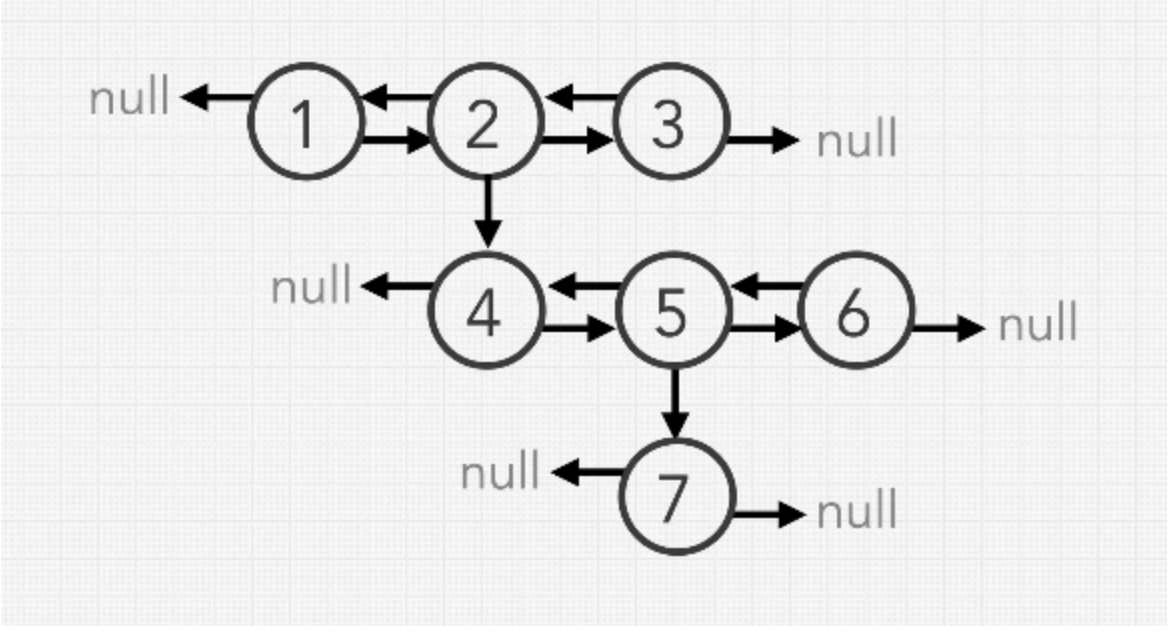


Child can point to null also


null
X
X
X
X
null
null
X
X
X
null
null
X
X
null


null
1
2
3
4
null
null
5
6
null

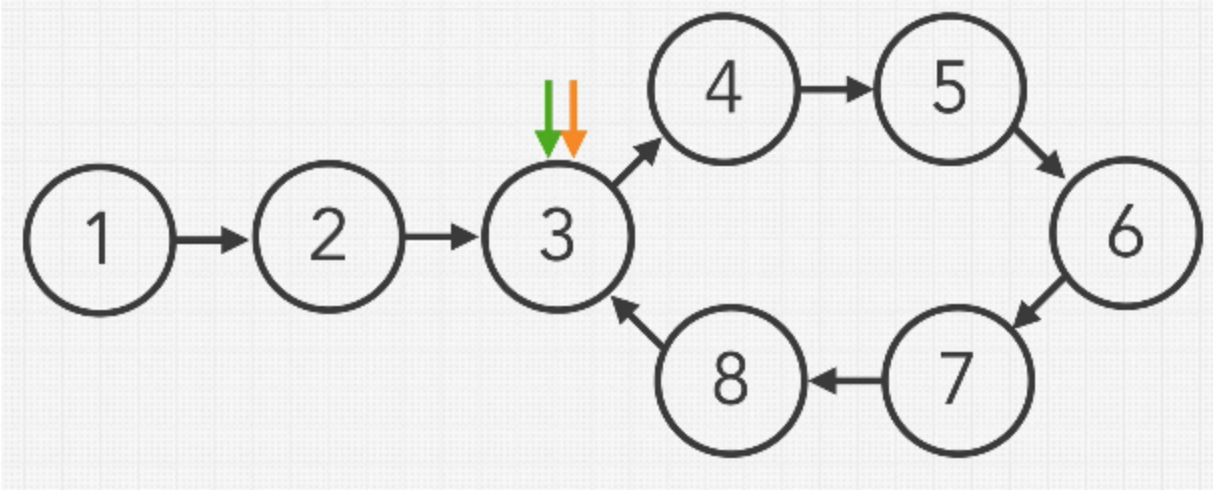

1
2
5
6
3
4




```
let currentNode = head;
while (currentNode !== null) {
  if(currentNode.child === null){
    currentNode = currentNode.next;
  } else {
    let tail = currentNode.child;
    while(tail.next !== null){
      tail = tail.next;
    }
    tail.next = currentNode.next;
    if(tail.next !== null) {
      tail.next.prev = tail;
    }
    currentNode.next = currentNode.child;
    currentNode.next.prev = currentNode;
```

time: O(N)
space: O(1)

Cycle detection



Method: Floyd’s tortoise and hare

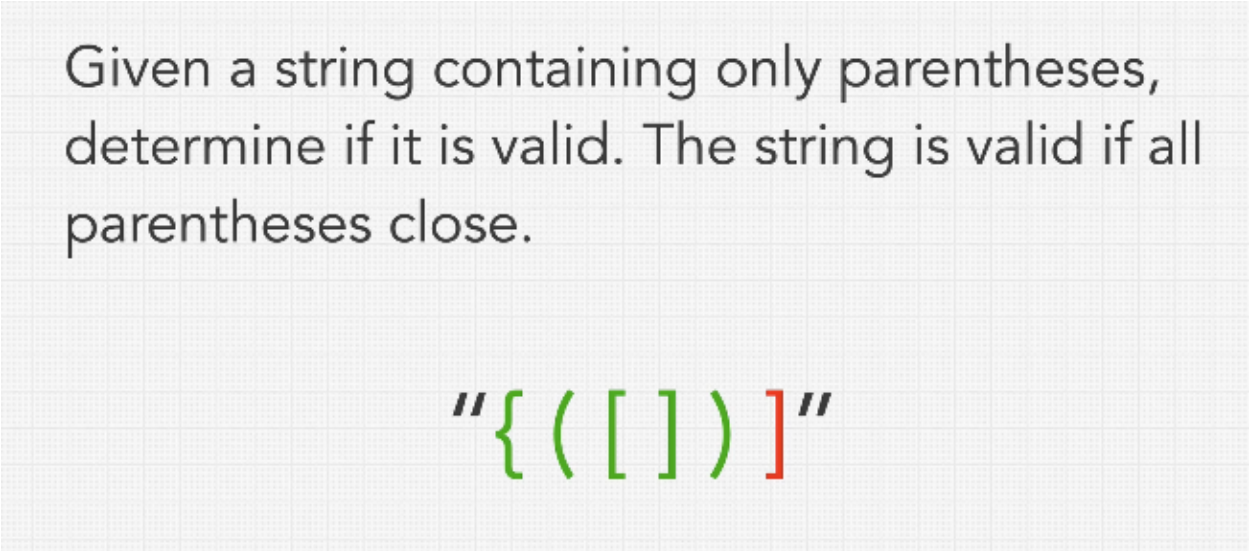
```
let hare = head, tortoise = head;

while (true) {
   hare = hare.next;
   tortoise = tortoise.next;
   if(hare === null || hare.next === null) {
      return false;
   } else {
     hare = hare.next;
   }

   if( tortoise === hare) break;
}

let p1 = head, p2 = tortoise;

while( p1 !== p2){
```

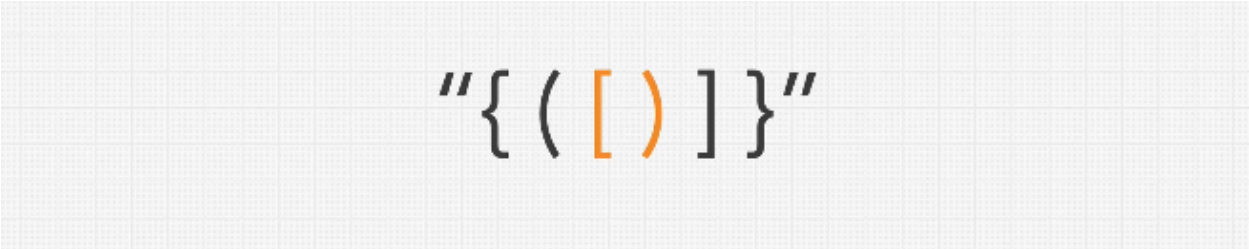
time : O(N)
space : O(1)

Stacks

Valid parentheses

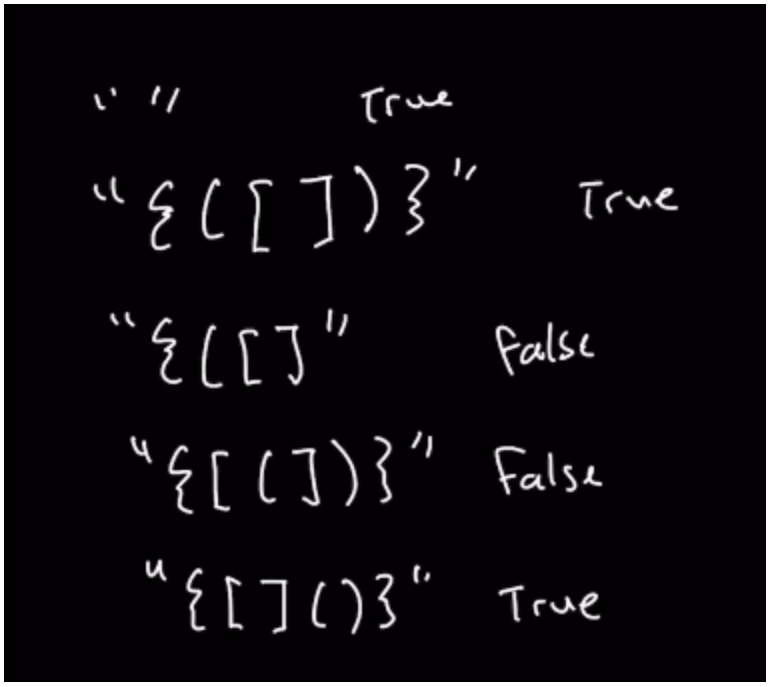
Problem



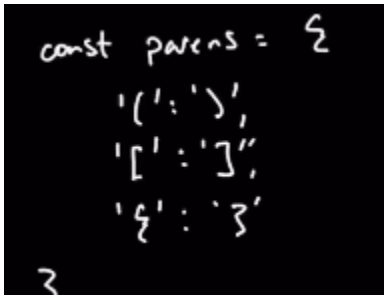


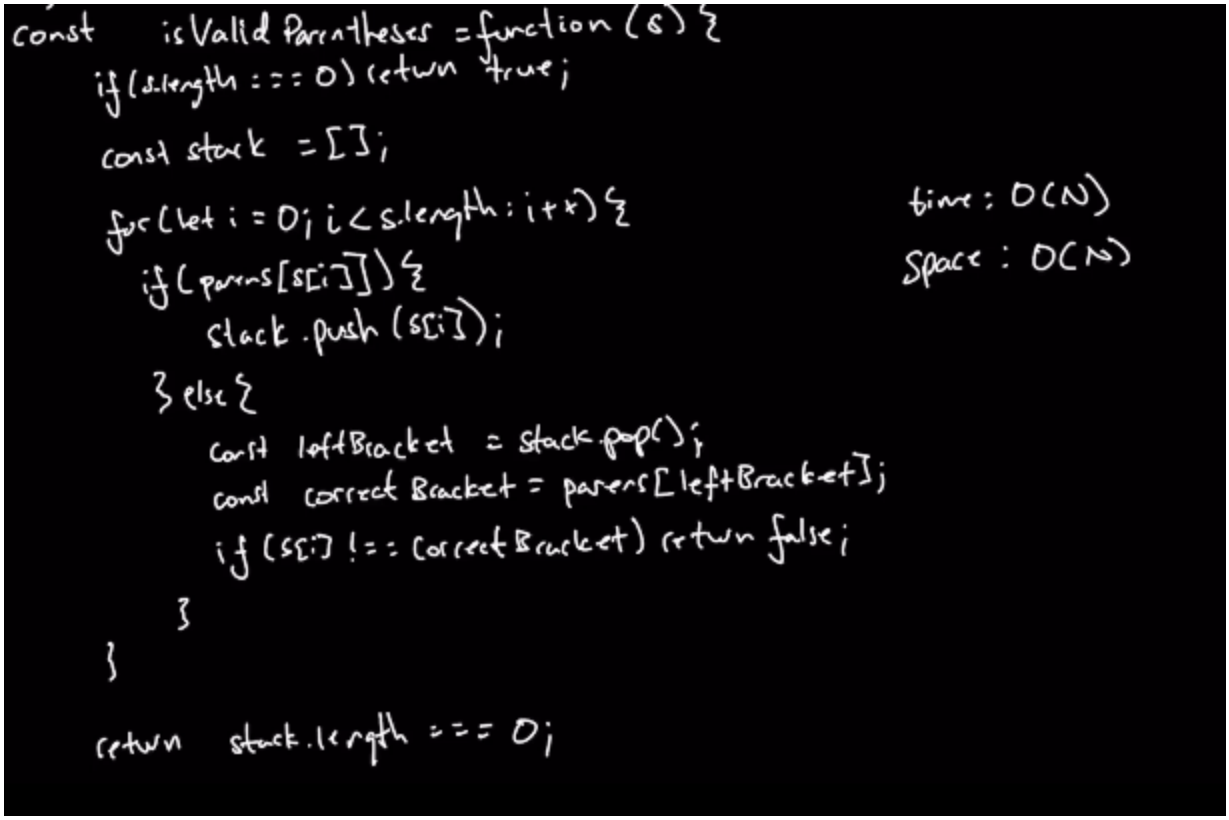
Here the position is incorrect

The order is important so we can use Stack/queues



We can use a bracket to map matches





Method: stack

Minimum brackets remove

Problem

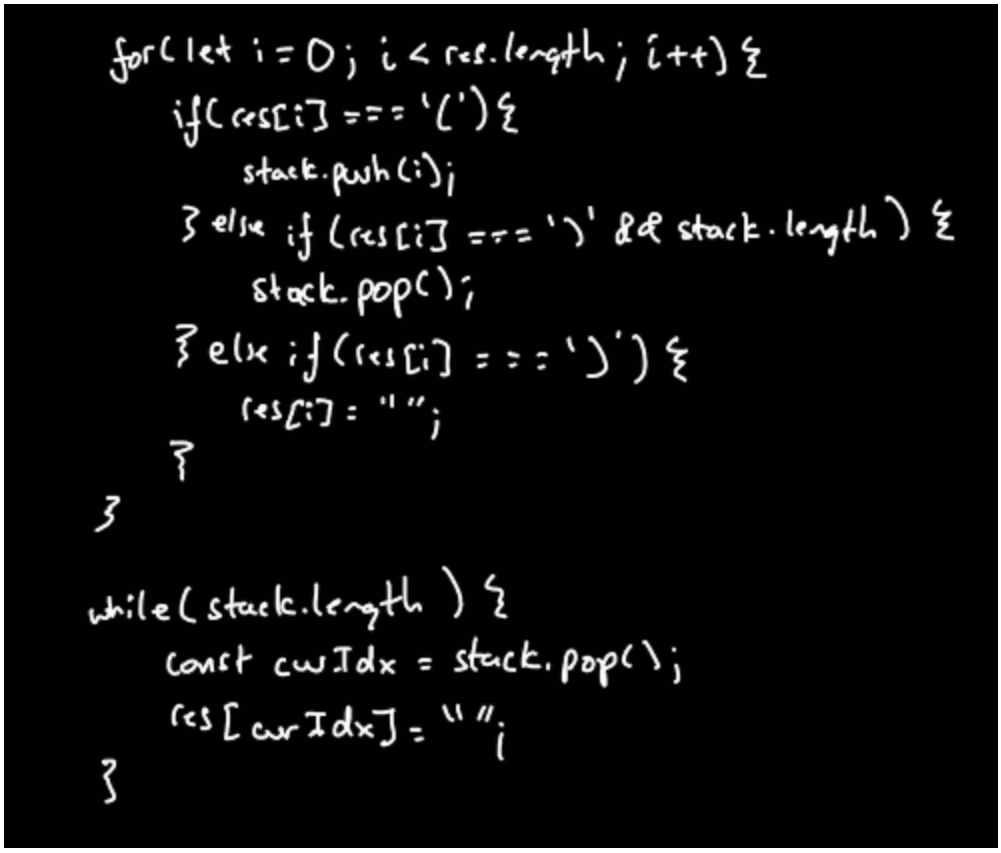
Given a string only containing round brackets '(' and ')' and lowercase characters, remove the least amount of brackets so the string is valid.

"a)bc(d)"

This is now valid

We can remove whatever bracket

Method: stack



```
for(let i = 0; i < res.length; i++) {
    if(res[i] === '(') {
        stack.push(i);
    } else if (res[i] === ')' && stack.length) {
        stack.pop();
    } else if (res[i] === ')') {
        res[i] = "";
    }
}

while(stack.length) {
    const curIdx = stack.pop();
    res[curIdx] = "";
}
```

But another way to do this is storing the sign and position in the stack , doing a stack of arrays, but if only exists “(“ and “)” the last algorithm is correct.

Queue

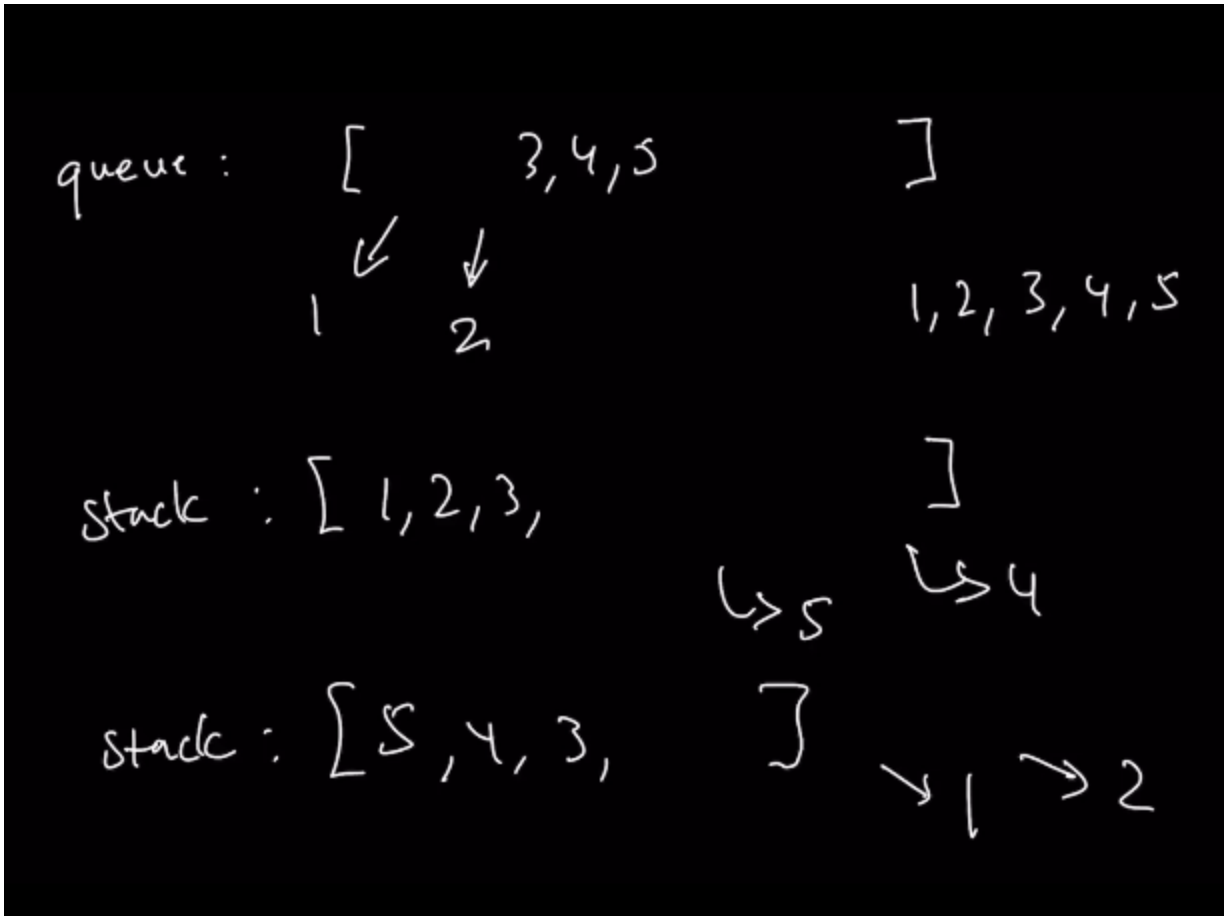
Implement queue with stacks

Implement the class Queue using stacks. The queue methods you need to implement are *enqueue*, *dequeue*, *peek*, and *empty*.

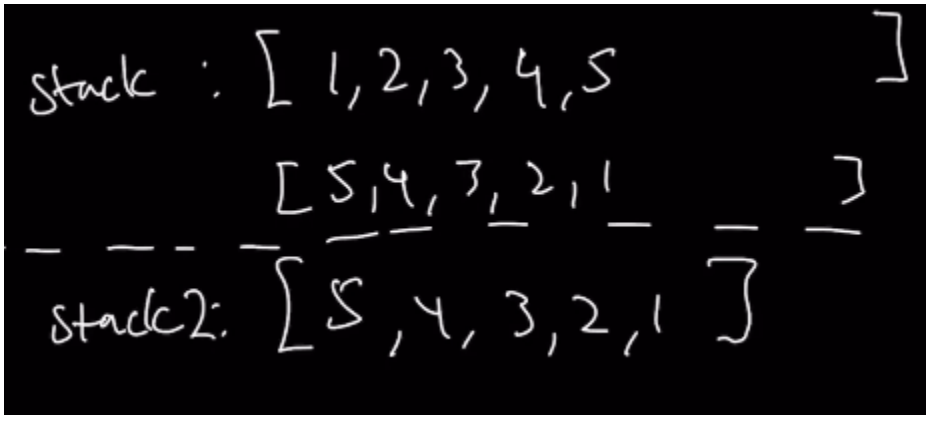
enqueue: append a value to the end of the queue.

dequeue: Remove and the value at the start of the queue.

peek: return the value at the start of the queue.



Only can have a queue if we reverse the array



Recursion

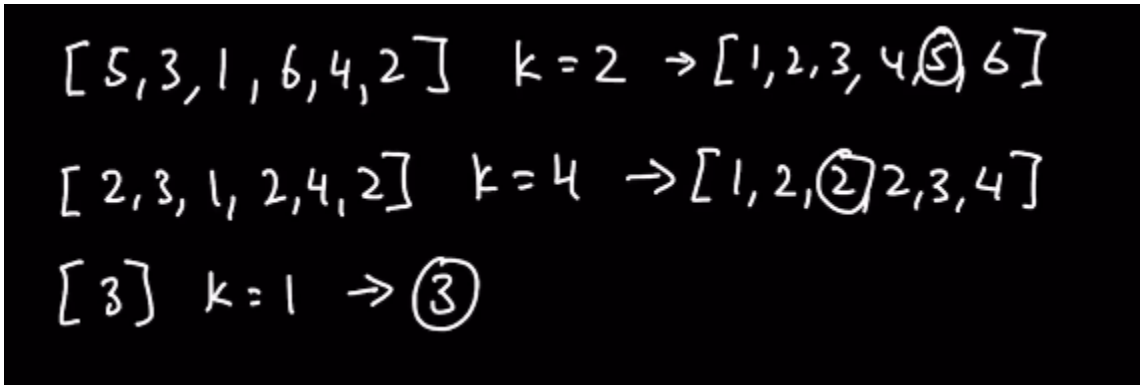
Kth largest element

Problem

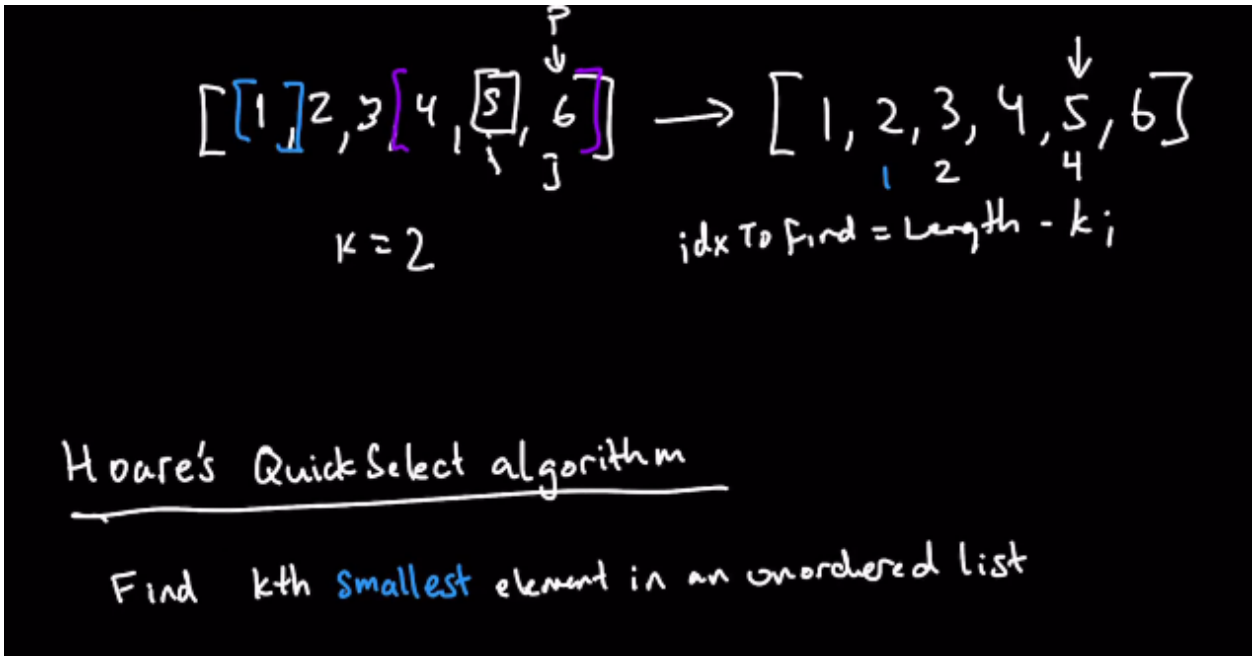
Given an unsorted array, return the kth largest element. It is the kth largest element in sorted order, not the kth distinct element.

k = 2

[5, 3, 1, 6, 4, 2] → [1, 2, 3, 4, 5, 6]



Method: Quick select



We choose what branch (left or right) of the recursion will be sorted, so it's a modification of quicksort

Start and end of a target in a sorted array

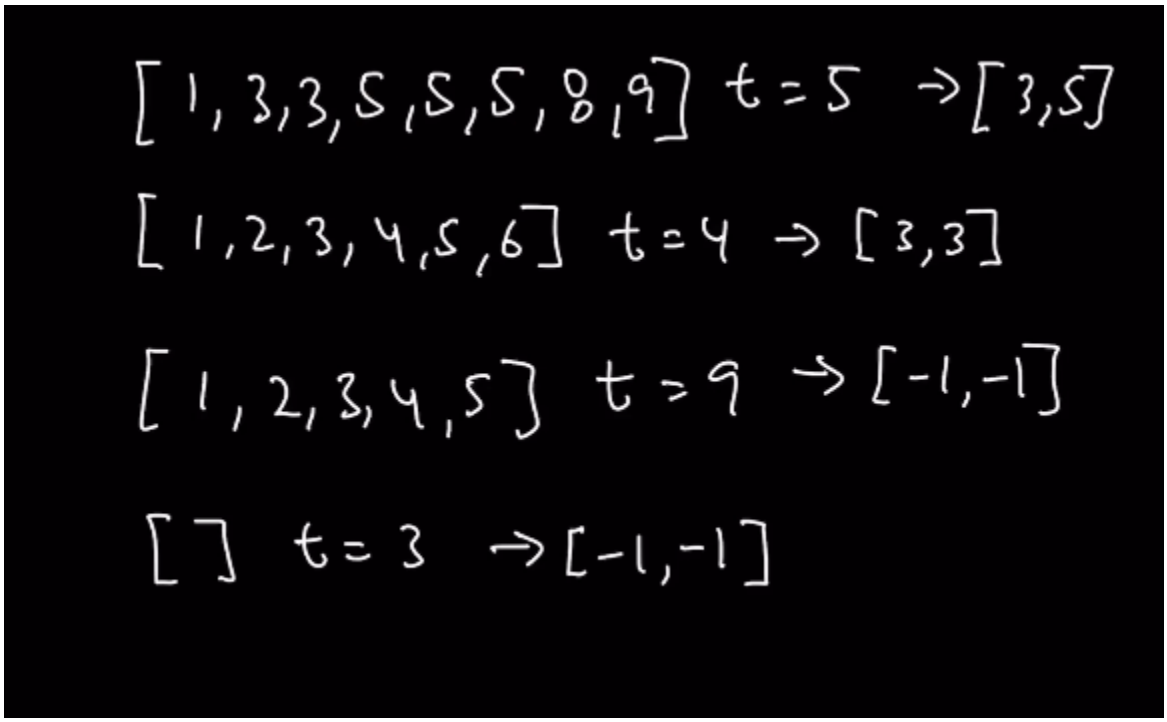
Problem

Given an array of integers sorted in ascending order, return the starting and ending index of a given target value in an array, i.e. [x, y].

5

[1, 3, 3, 5, 5, 5, 8, 9]
0 1 2 3 4 5 6 7

start: 3 ending: 5



Method: Binary search with n run times

Sometimes it's necessary to run binary search more than one time

```
const searchRange = function(nums, target) {
    if(nums.length === 0) return [-1, -1];
    const firstPos = binarySearch(nums, 0, nums.length-1, target);
    if(firstPos === -1) return [-1,-1];

    let startPos = firstPos,
        endPos = firstPos;
        temp1, temp2;

    while(startPos !== -1) {
      temp1 = startPos;
      startPos = binarySearch(nums, 0, startPos-1, target);
```

time : O(log n)

space : O(1)

It's not n*log(n)

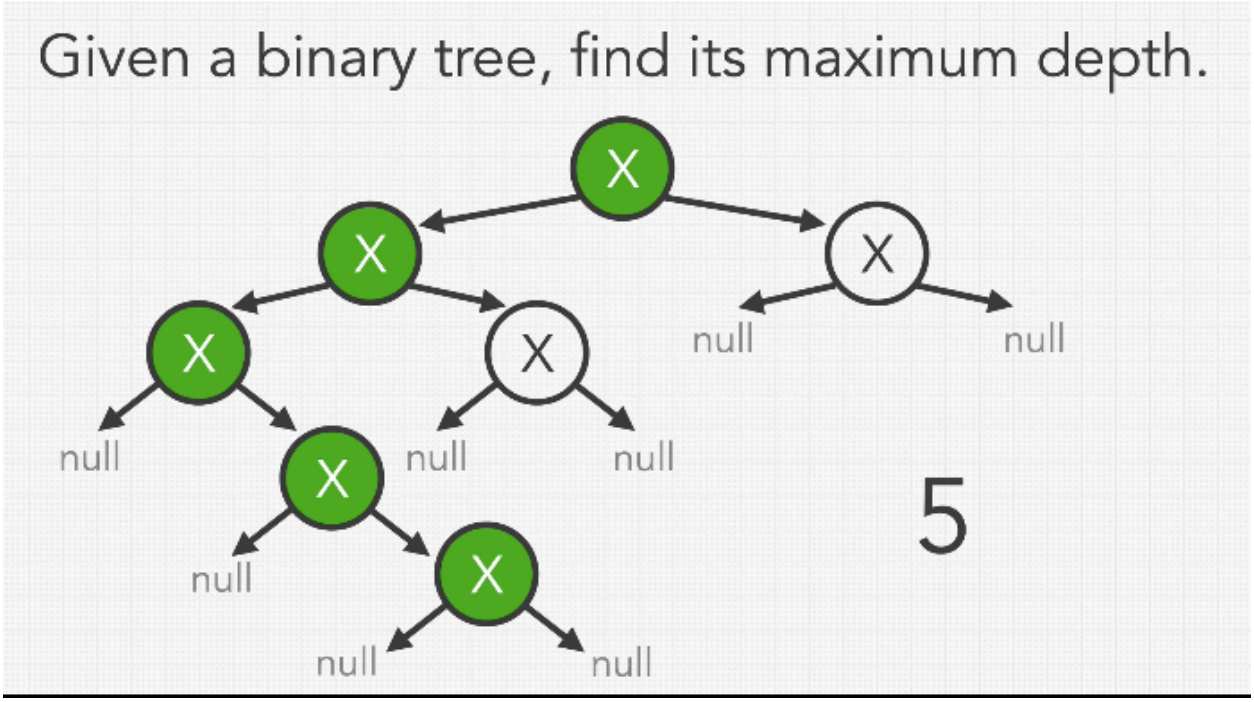
$$\log n + \log(n/2) + \log(n/4)$$

$$\log n + \log(n) + \log n$$

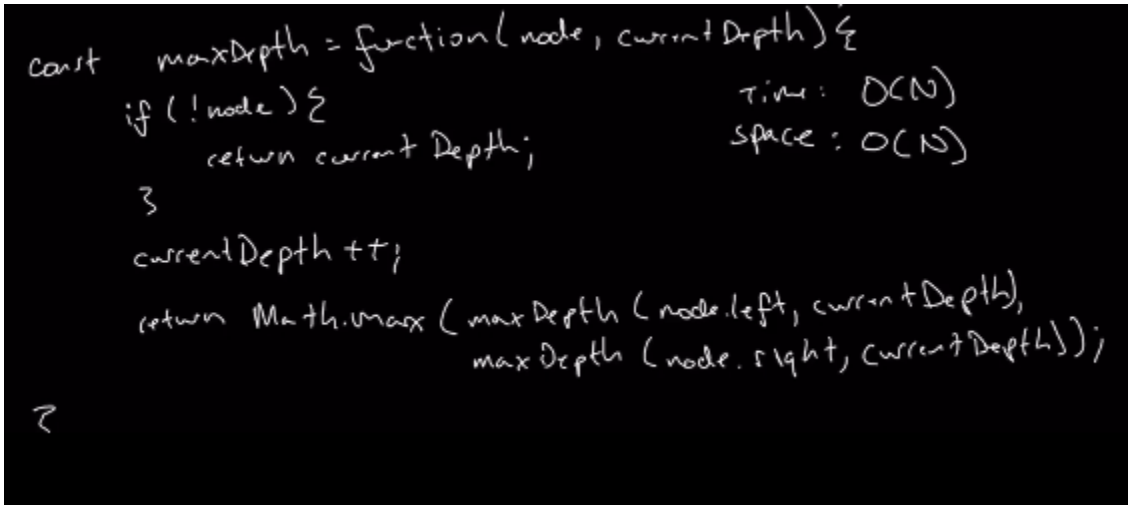
Binary tree

Max depth

Problem



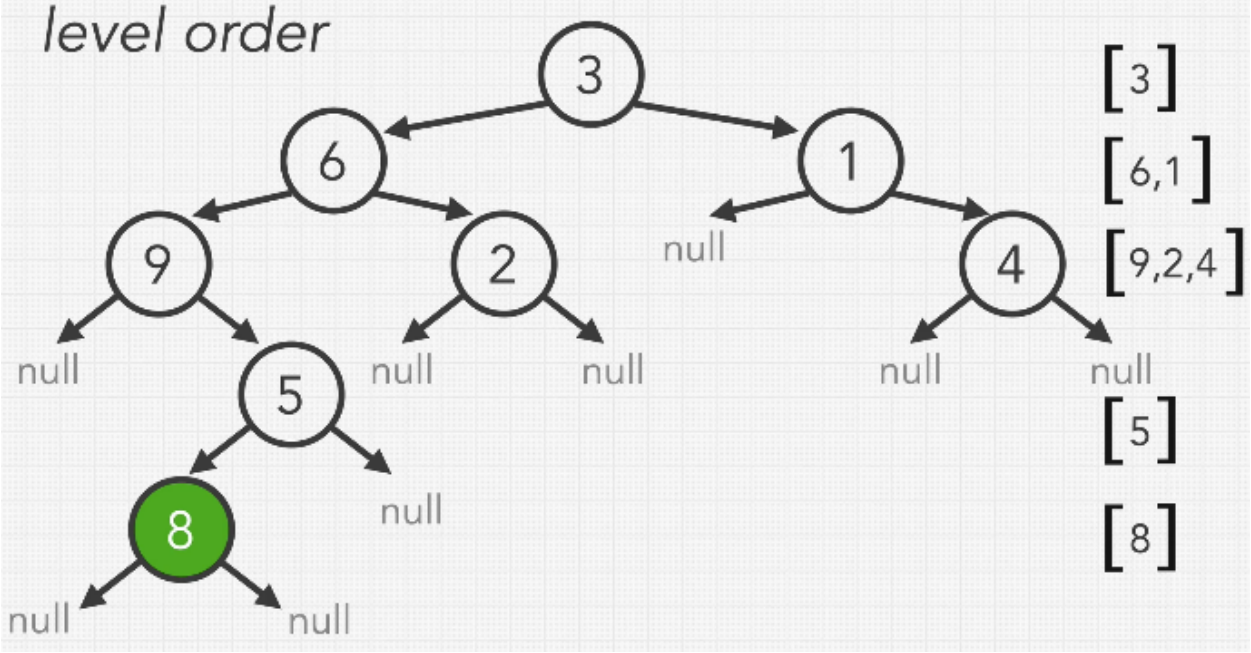
Method: DFS

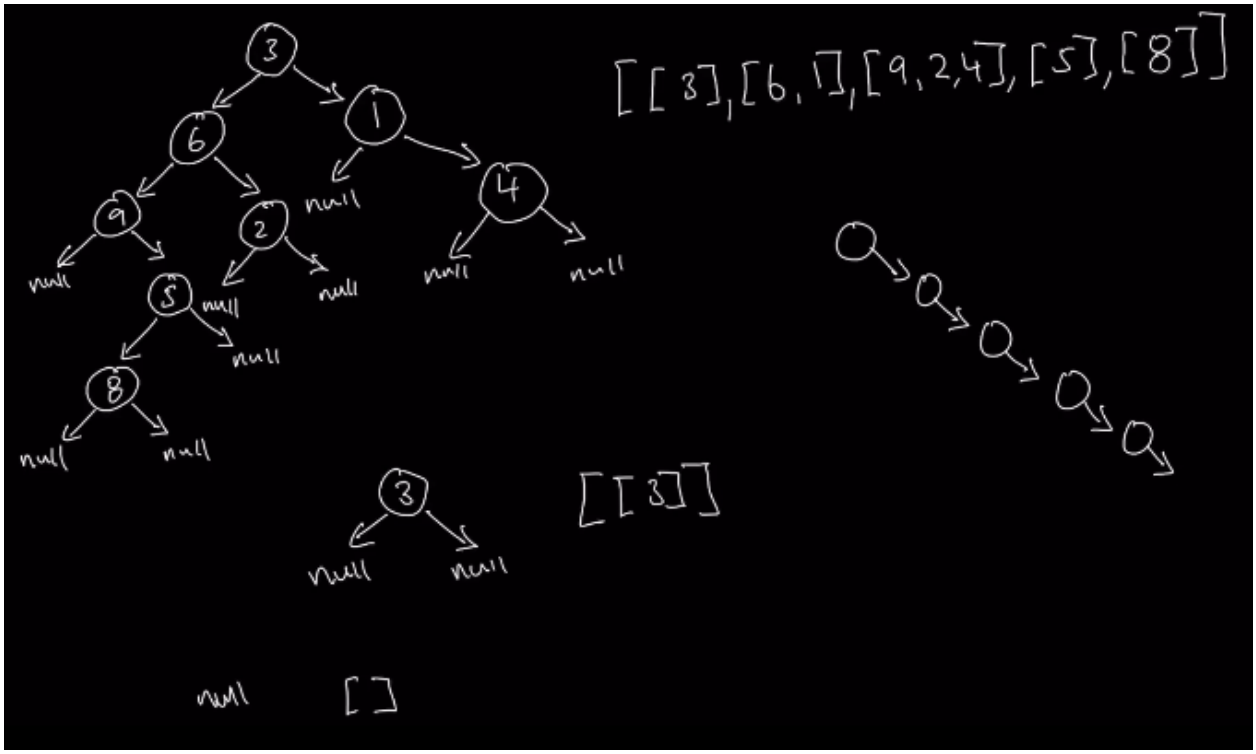


Level order as a binary tree

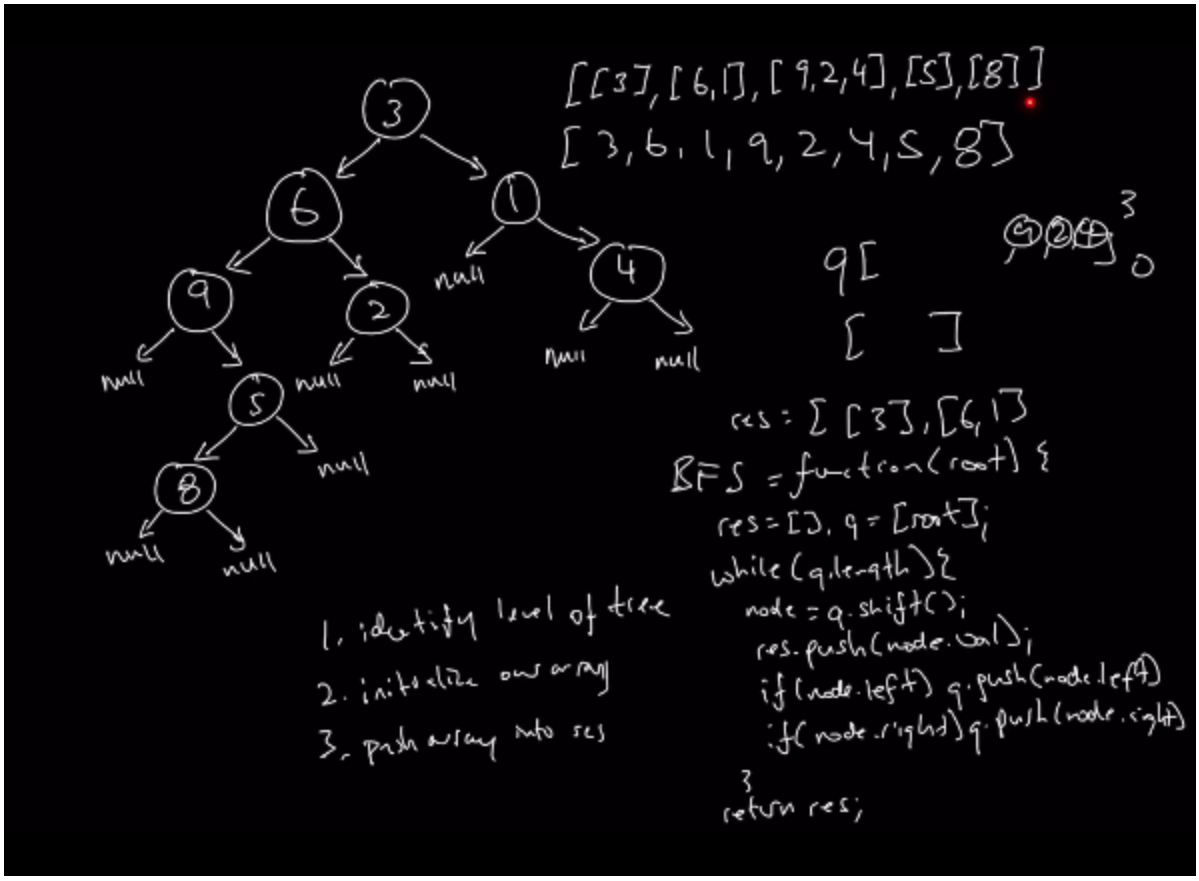
Problem

Given a binary tree, return the *level order* traversal of the nodes' values as an array.

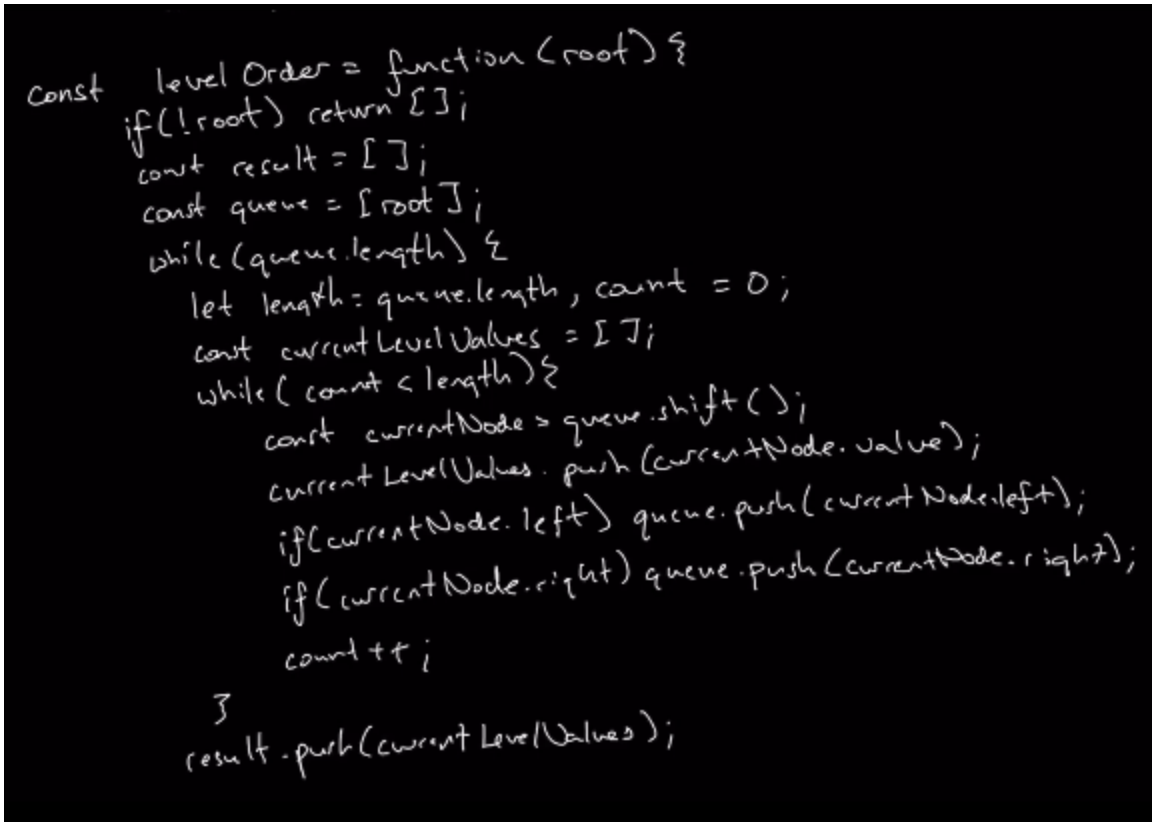




Method: BFS



We iterate through an array that grows in the time

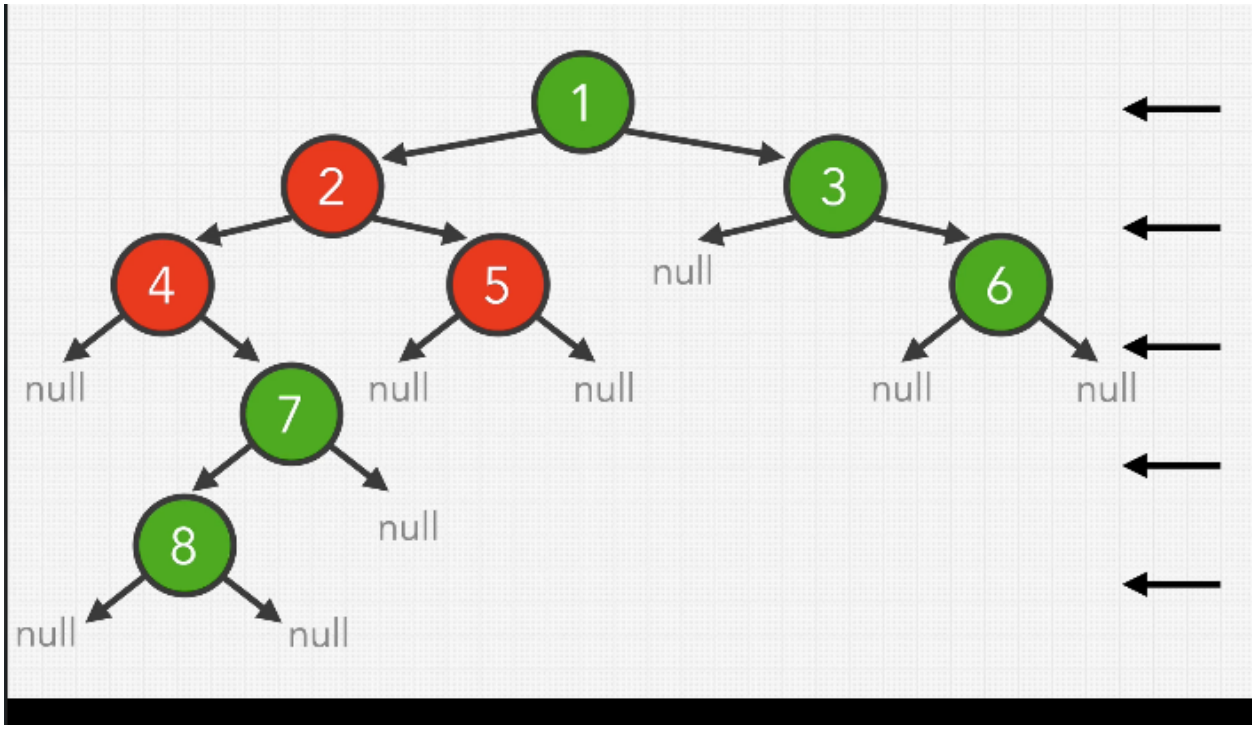


At the end return the result

Right side of tree

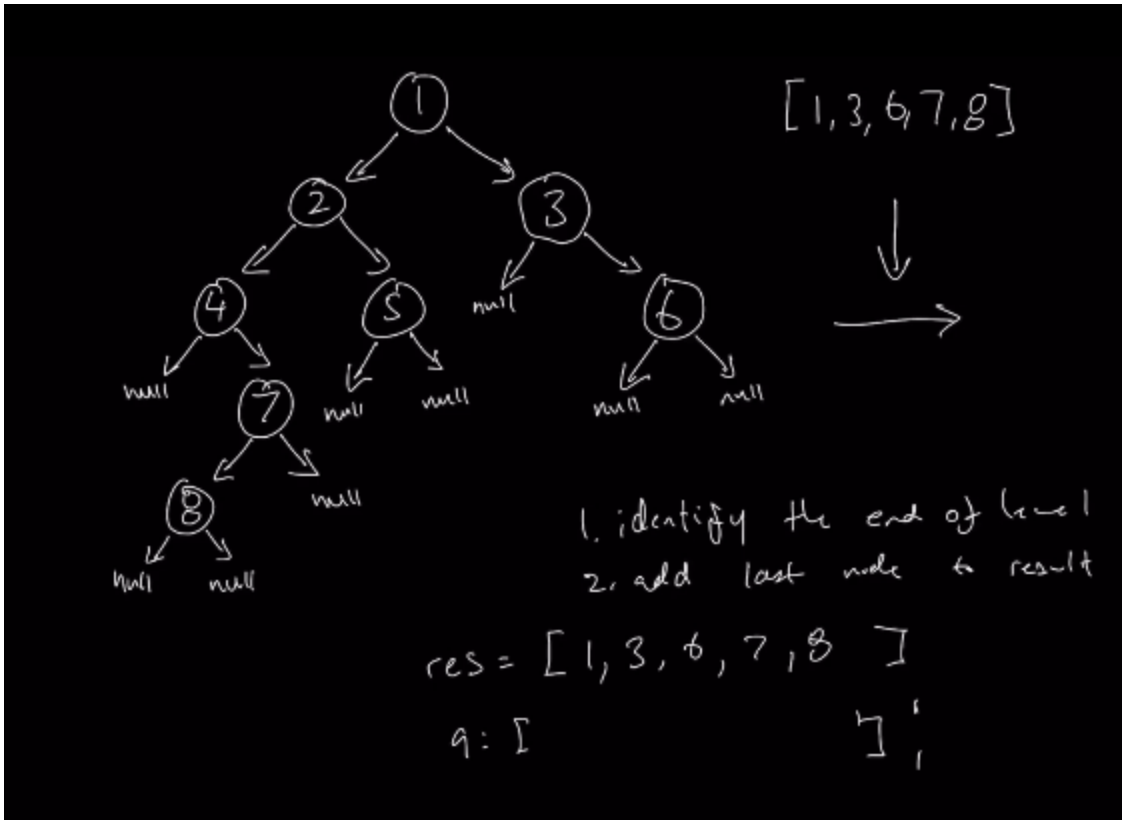
Problem

Given a binary tree, imagine you're standing to the right of the tree. Return an array of the values of the nodes you can see ordered from top to bottom.



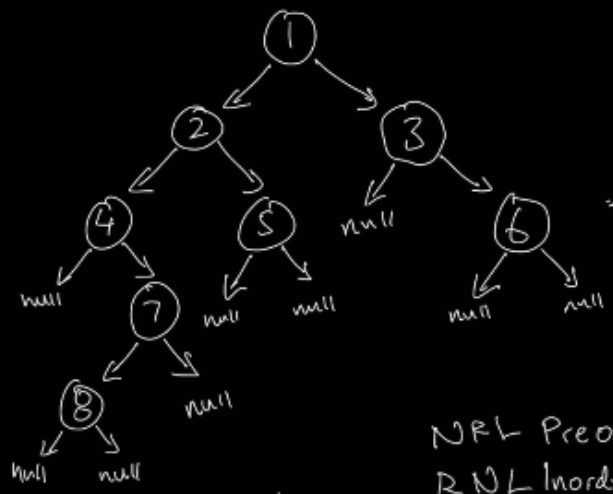
Method 1: BFS

We get just the last value of BFS, because we visit each level


[1, 3, 6, 7, 8]
1. identify the end of level
2. add last node to result
res = [1, 3, 6, 7, 8]
q: [
]

Method 2: DFS

DFS



[1, 3, 6, 7, 8]

1. prioritize right side
2. keep track level of nodes

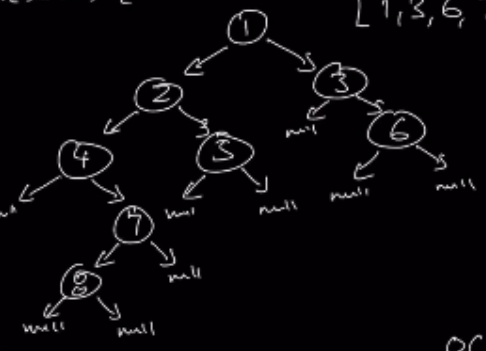
R L

NRL Preorder: [1, 3, 6, 2, 5, 4, 7, 8]
RNL Inorder: [6, 3, 1, 5, 2, 7, 8, 4]
RL N Postorder: [6, 3, 5, 8, 7, 4, 2, 1]

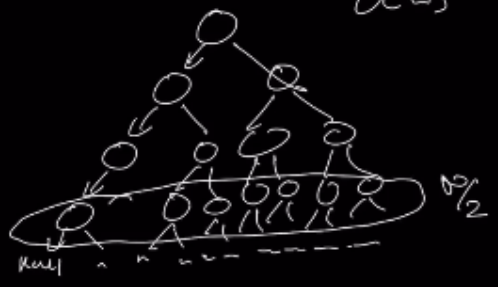
```
const dfs = function( node , currentLevel, result) {
    if(!node) return ;
    if(currentLevel >= result.length) {
      result.push(node.value);
    }
    if(node.right) {
      dfs(node.right, currentLevel +1, result);
    }
    if(node.left) {
      dfs(node.left, currentLevel +1, result);
    }
}
const rightSideView = function (root) {
    const result = [];
    dfs(root, 0, result);
    return result;
}
```

T: O(N)
S: O(H)

[1, 3, 6, 7, 8]

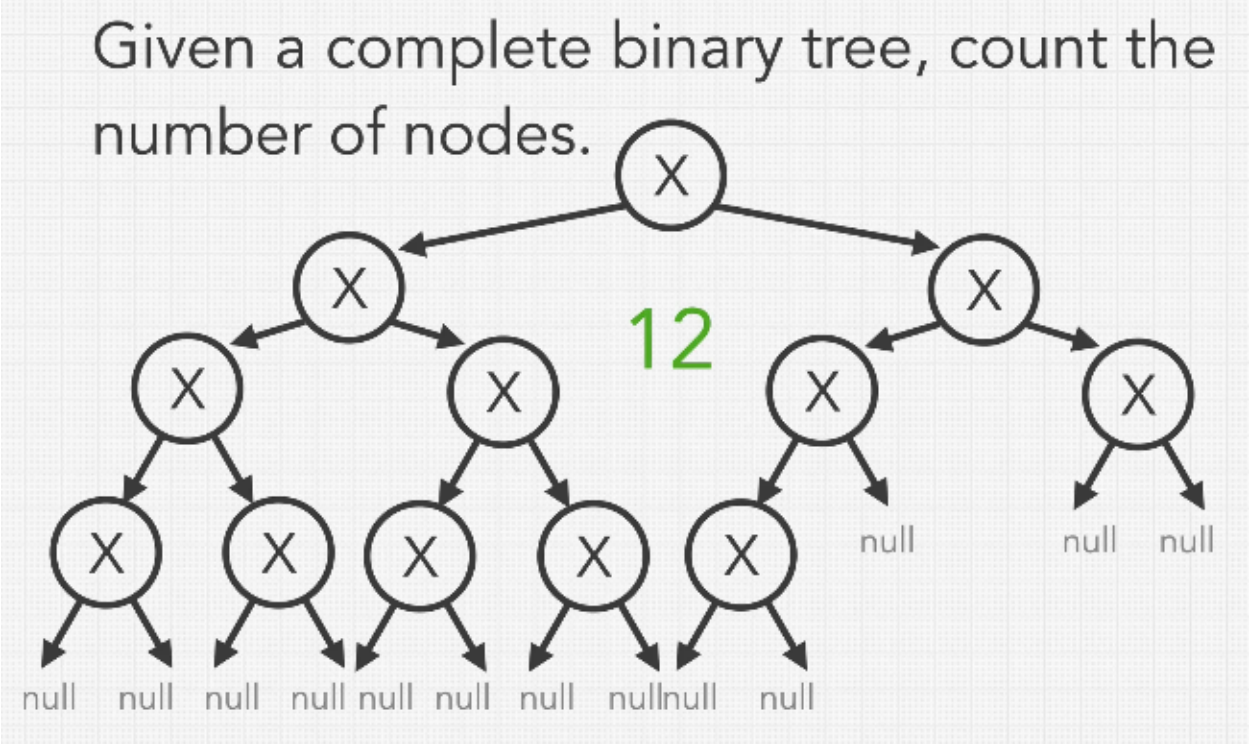


O(N)
O(W)



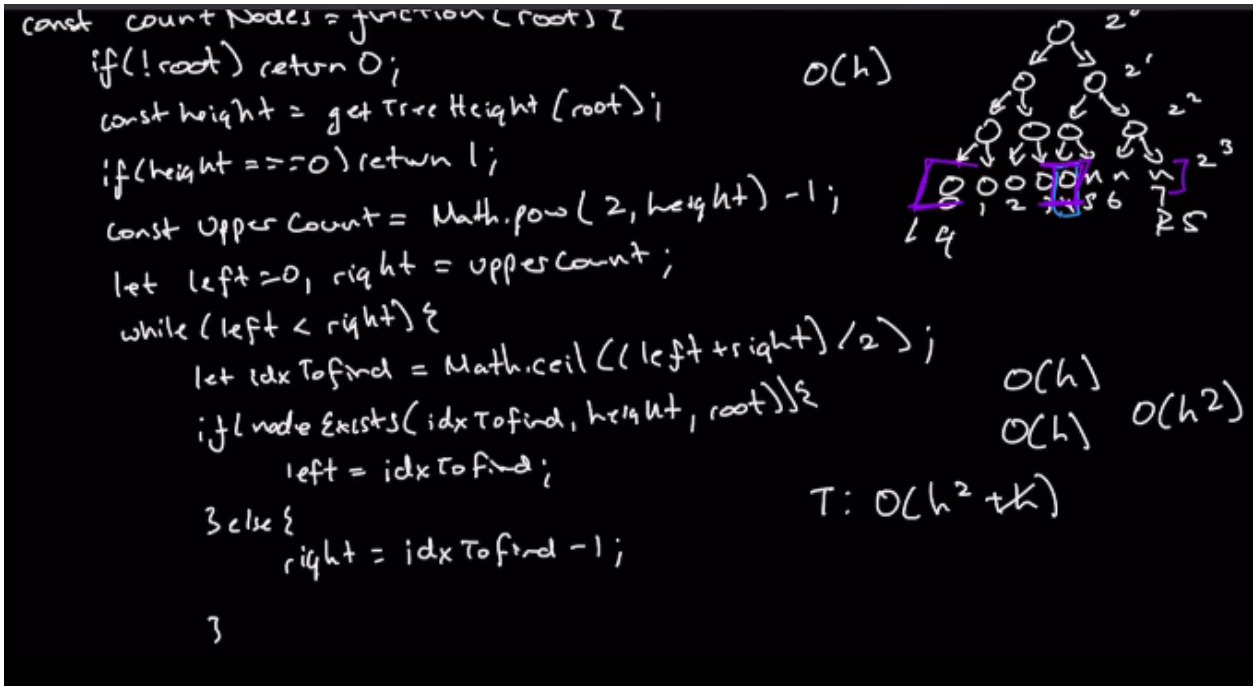
Number of nodes in complete binary tree

Problem



Only binary search can search in log(n) time

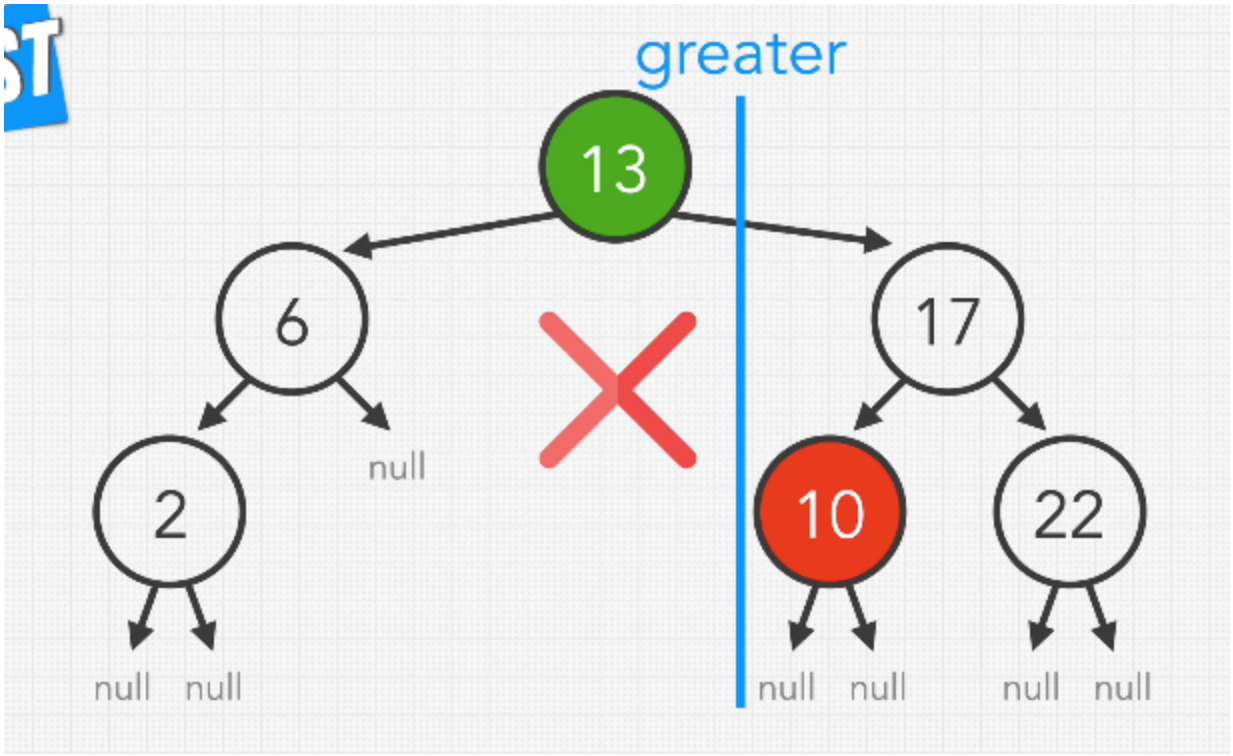
Method: Binary search



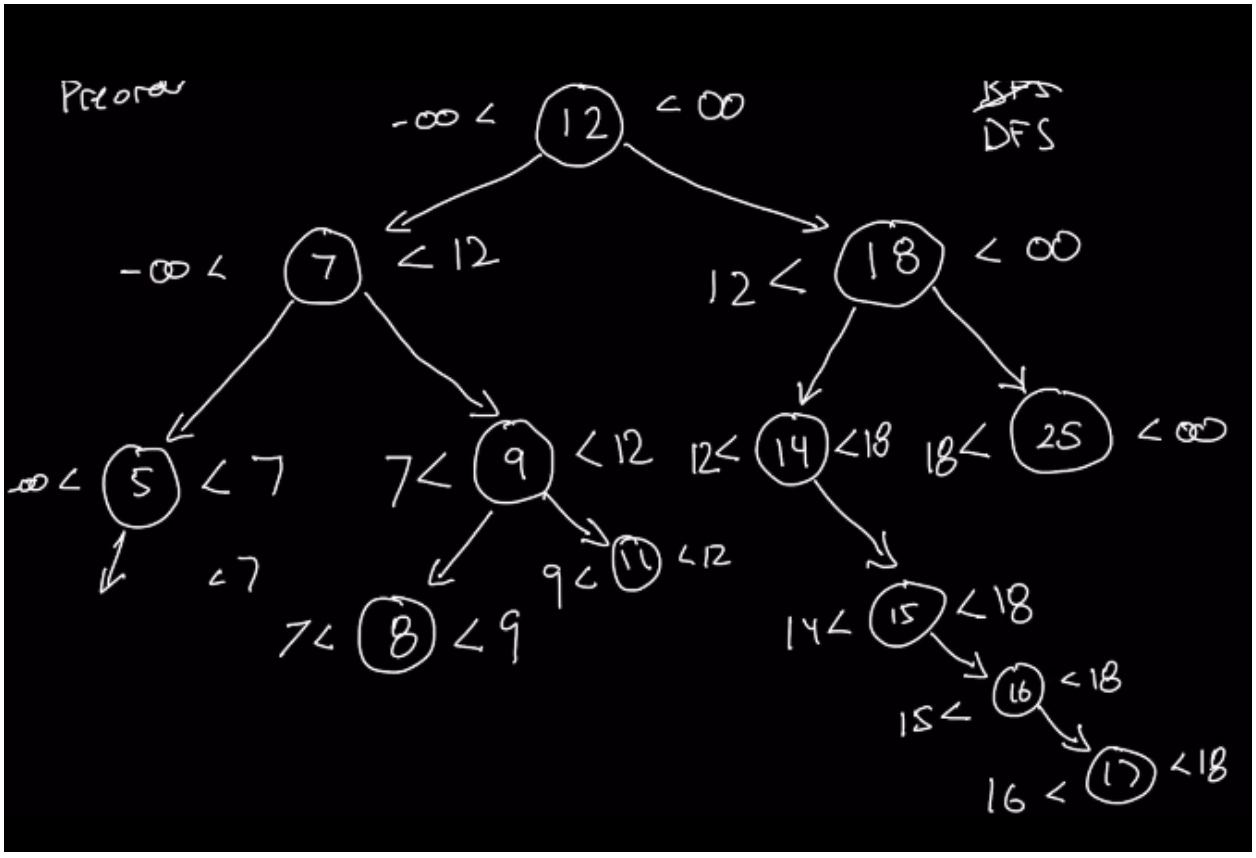
Validate binary search tree

Problem

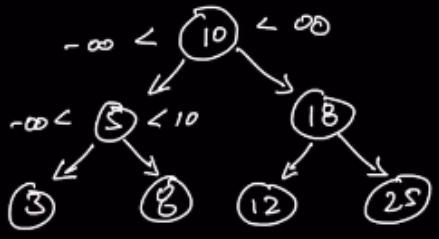
Given a binary tree, determine if it is a valid binary search tree.



Method: DFS Preorder




```
const isValidBST = function(root) {
    if(!root) return true;
    return dfs(root, -Infinity, Infinity);
};

const dfs = function(node, min, max) {
    if(node.val <= min || node.val >= max) {
        return false;
    }
    if(node.left) {
        if(!dfs(node.left, min, node.val)){
            return false;
        }
```



T: O(N)
S: O(N)

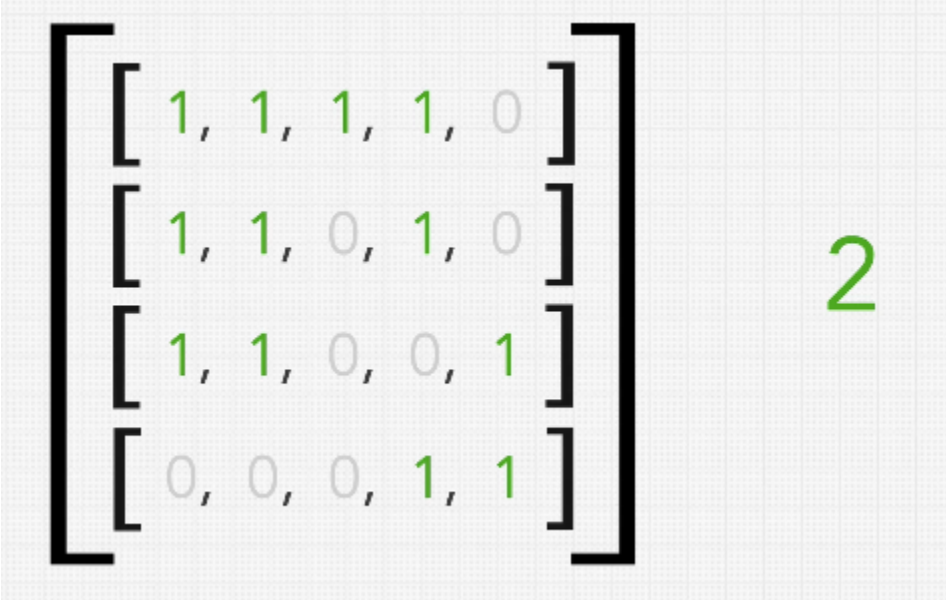
2D Arrays

Number of islands

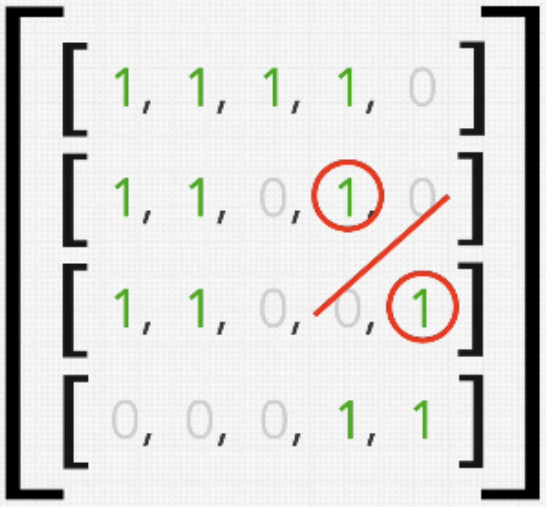
Problem

Given a 2D array containing only 1's (land) and 0's (water), count the number of islands.

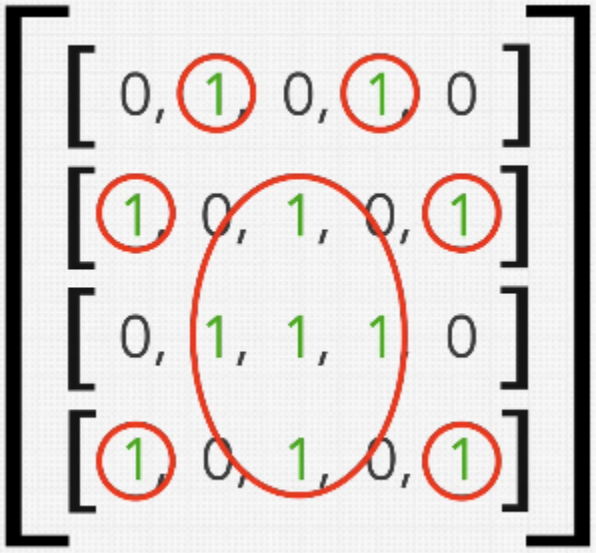
An island is land connected horizontally or vertically.



An island is land connected horizontally or vertically.



2



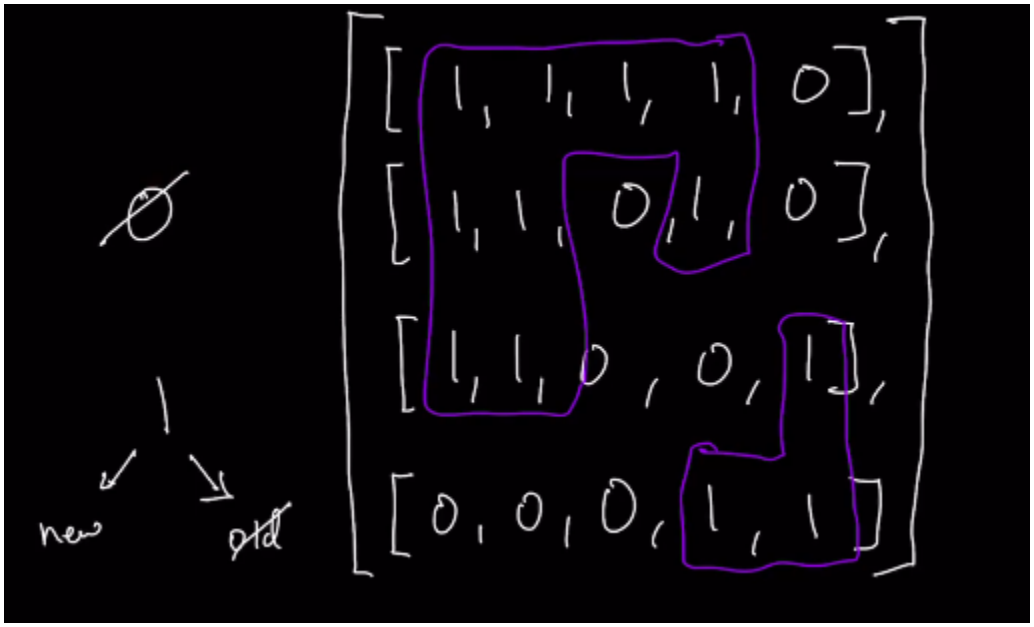
7

Yes, assume anything outside of the 2D array is water.

Think critically about the numbers

The 0 is not meaningful such as one

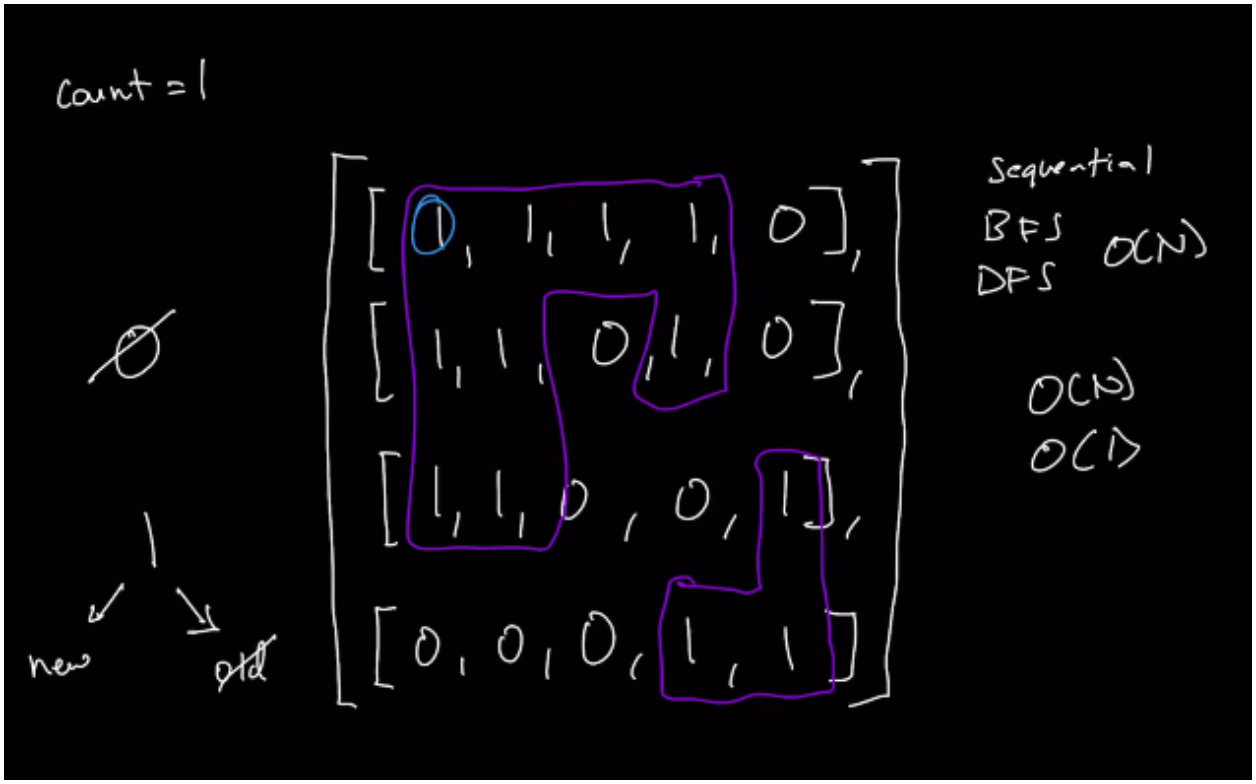
If 1 is about an old iland we don’t want to count the island



Method: 2D array with BFS | DFS

BFS and DFS is for solving a subproblem no the whole problem usually

With sequential order we get advantages

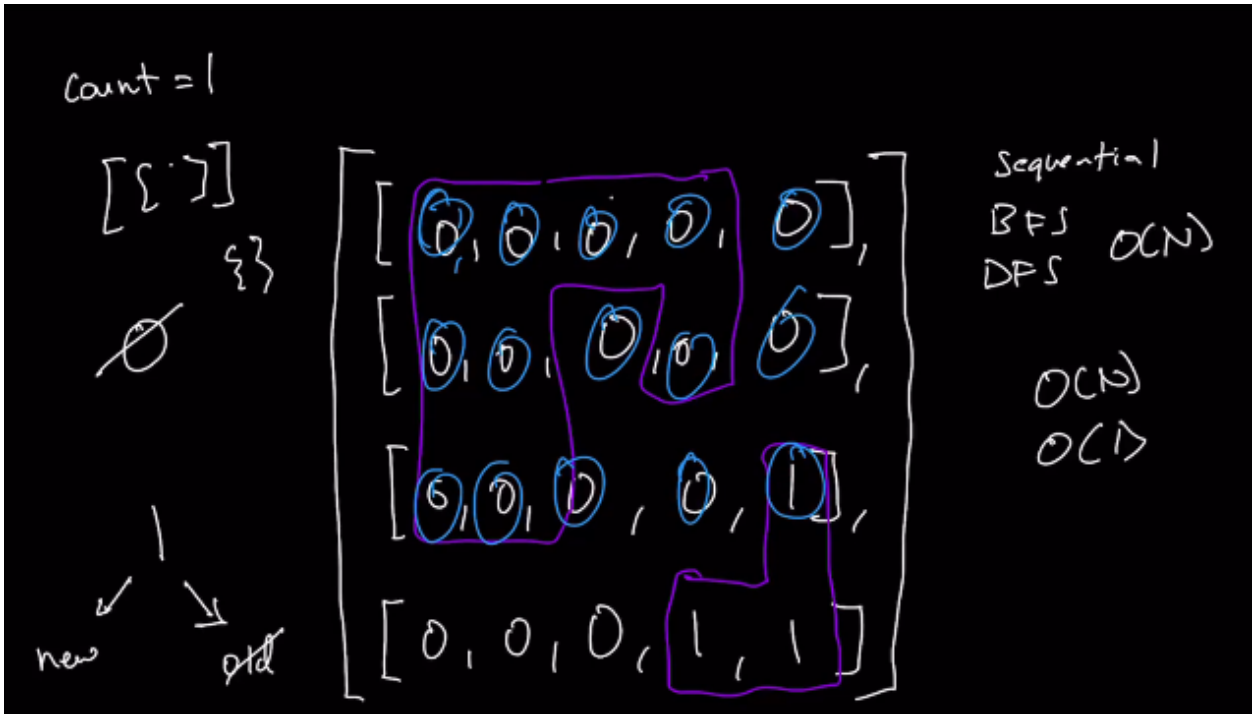


We need to classify between old and new island

We continue navigating to adjacent values and marking that positions as old.

Or simply transforming the ones into 0s

The solution I made was BFS, using two matrices in reference.




```
const directions = [[-1,0],
                    [0,1],
                    [1,0],
                    [0,-1]]

const numberOfIslands = function(matrix) {
    if(matrix.length === 0) return 0;
    let islandCount = 0;
    const queue = [];
```

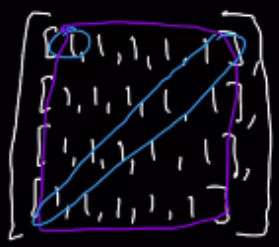
```
[[1,1,1,1,0],
 [1,1,0,1,0],
 [1,1,0,0,1],
 [0,0,0,1,1]]
```

```
const numberOfIslands = function(matrix) {
    if(matrix.length === 0) return 0;
    let islandCount = 0;
    const queue = [];
    for(let row = 0; row < matrix.length; row++) {
        for(let col = 0; col < matrix[0].length; col++) {
            if(matrix[row][col] === 1) {
                islandCount++;
                queue.push([row, col]);
                while(queue.length) {
```

```
const currentPos = queue.shift();
const currentRow = currentPos[0];
const currentCol = currentPos[1];

matrix[currentRow][currentCol] = 0;
for(let i = 0; i < directions.length; i++) {
    const currentDir = directions[i];
    const nextRow = currentRow + currentDir[0];
    const nextCol = currentCol + currentDir[1];
    if(nextRow < 0 || nextRow >= matrix.length ||
       nextCol < 0 || nextCol >= matrix[0].length) {
        continue;
    }
```

```
                if(matrix[nextRow][nextCol] === 1){
                  queue.push([nextRow, nextCol]);
                }

            }
        }
    }
  }
return islandCount;
}
```

```
const directions = [[-1,0],
                    [0,1],
                    [1,0],
                    [0,-1]]

const numberOfIslands = function(matrix){
    if(matrix.length === 0) return 0;
    let islandCount = 0;
    const queue = [];
    for(let row = 0; row < matrix.length; row++){
      for(let col = 0; col < matrix[0].length; col++){
        if(matrix[row][col] === 1){
          islandCount++;
          queue.push([row, col]);
          while(queue.length){
            const currentPos = queue.shift();
            const currentRow = currentPos[0];
```



T: O(M x N)

S: O(min(M,N))

DFS

T: O(M x N)

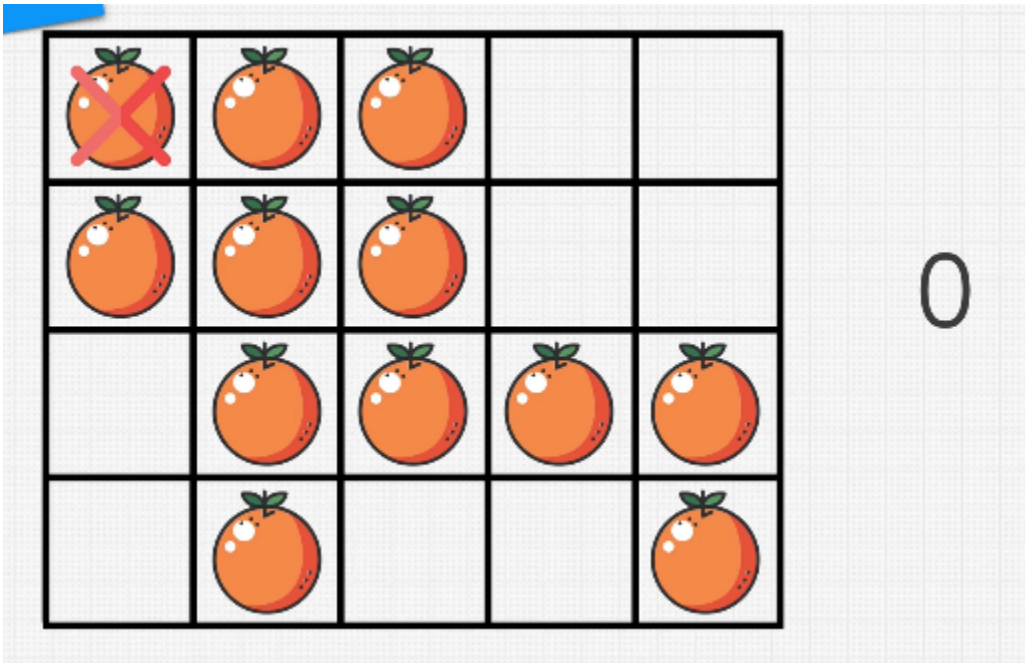
S: O(M x N)

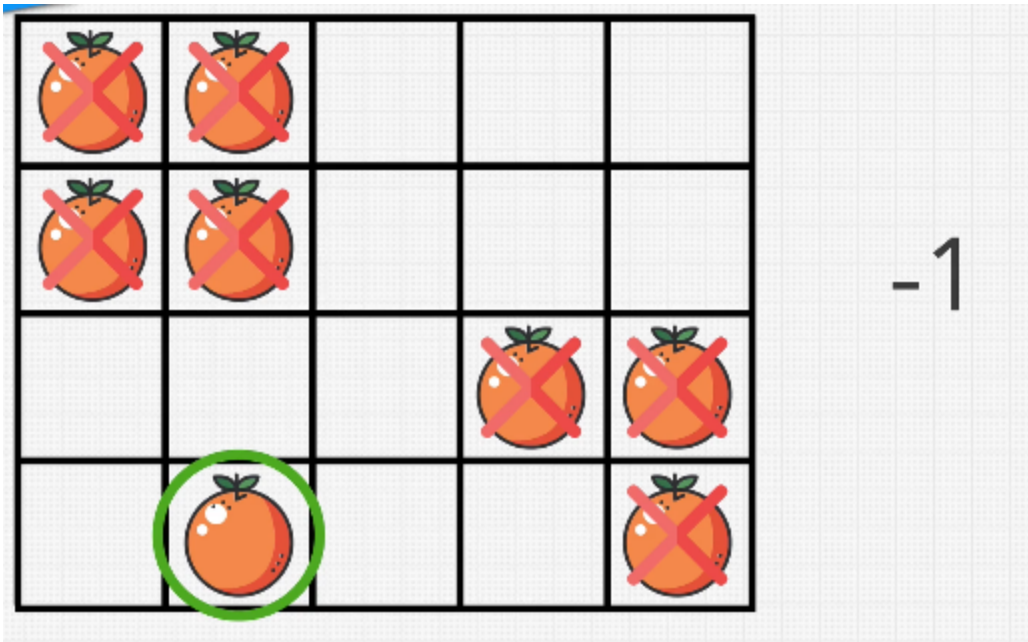
Rotting oranges

Problem

Given a 2D array containing 0's (empty cell), 1's (fresh orange) and 2's (rotten orange). Every minute, all fresh orange immediately adjacent (4 directions) to rotten oranges will rot.

How many minutes must pass until all oranges are rotten?

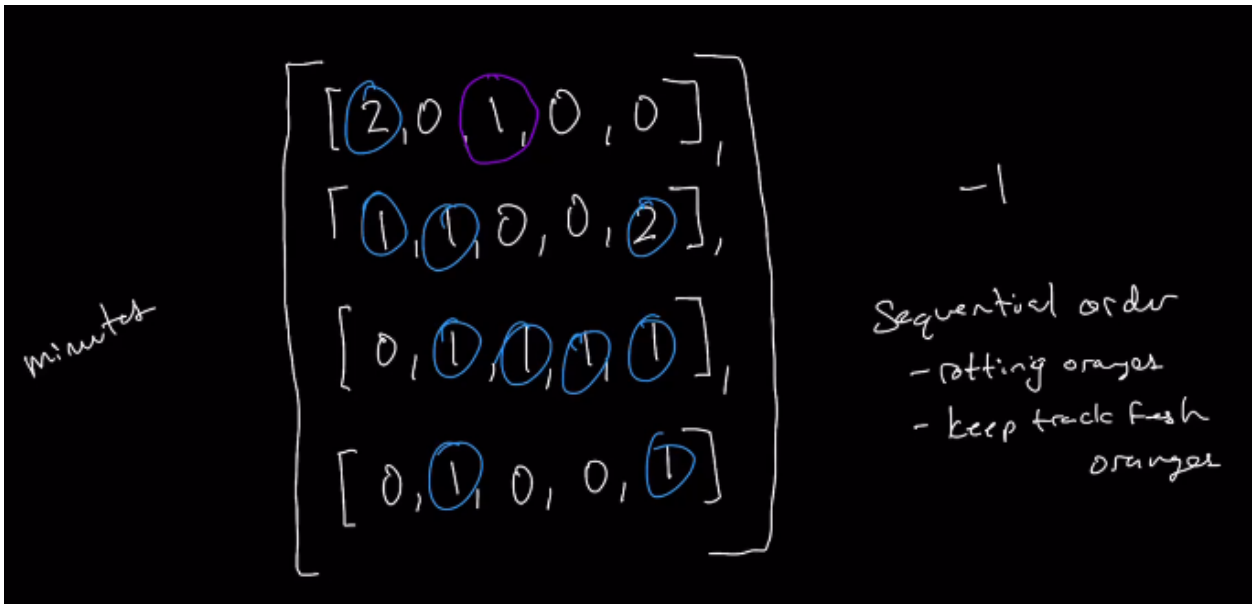




First scan the rotten oranges

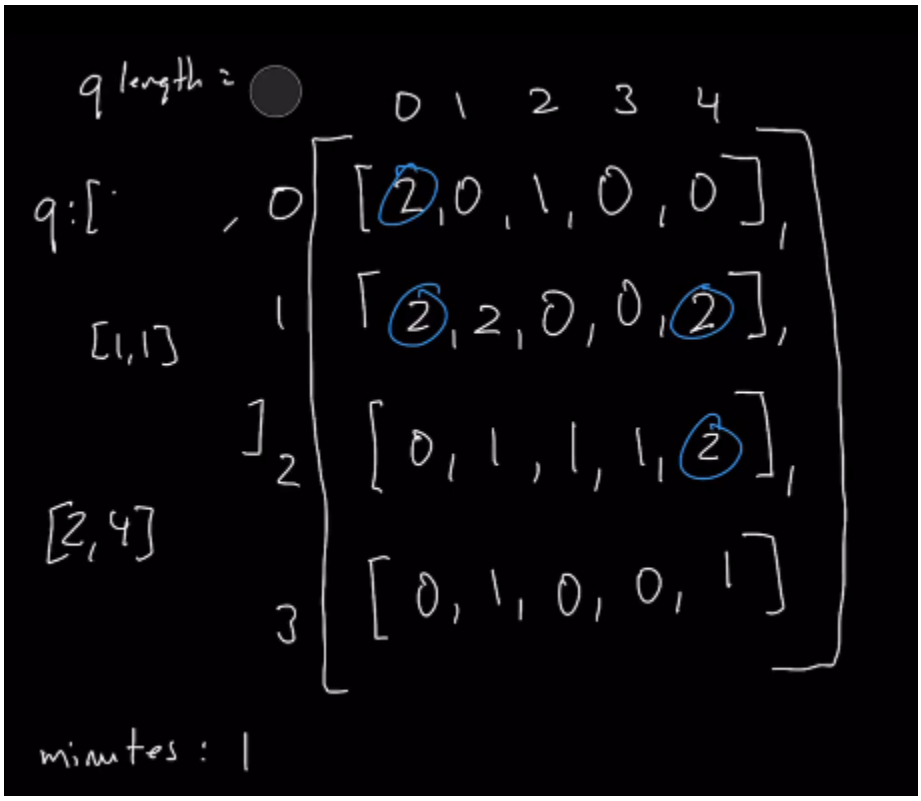
Method: 2D Array BFS

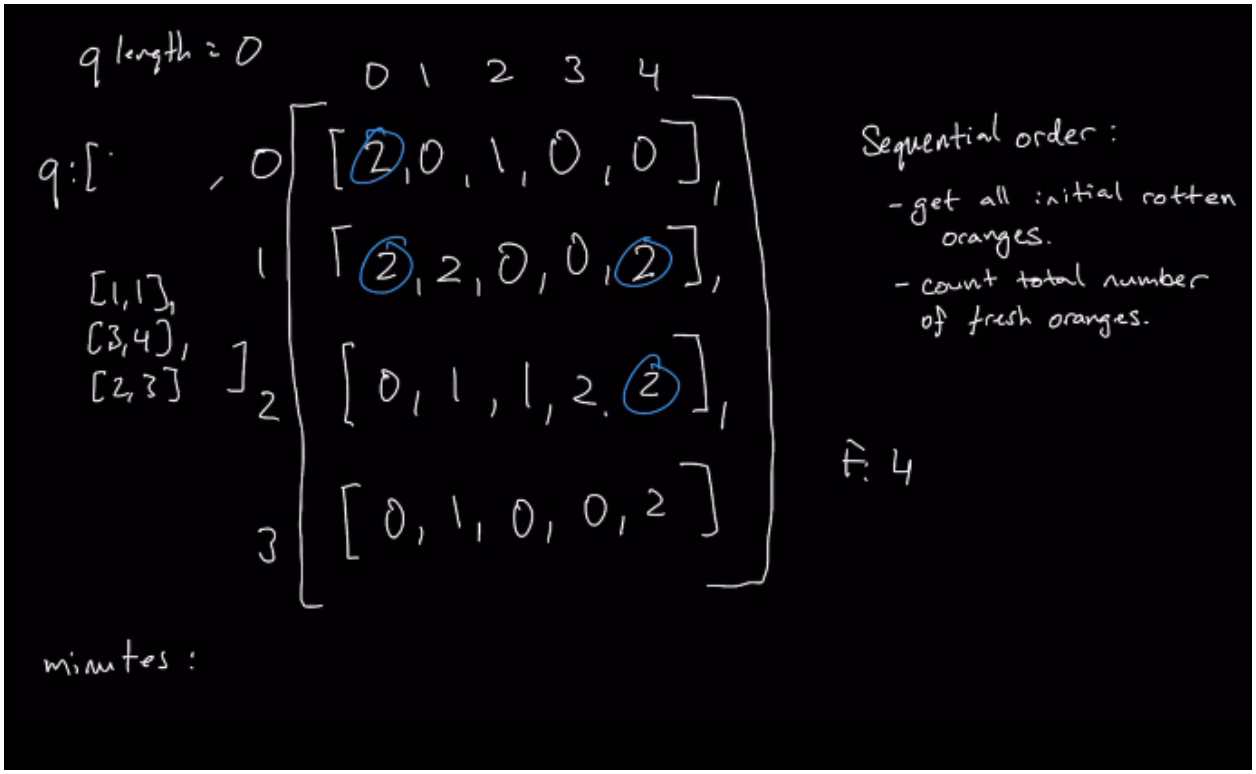
First explore the matrix



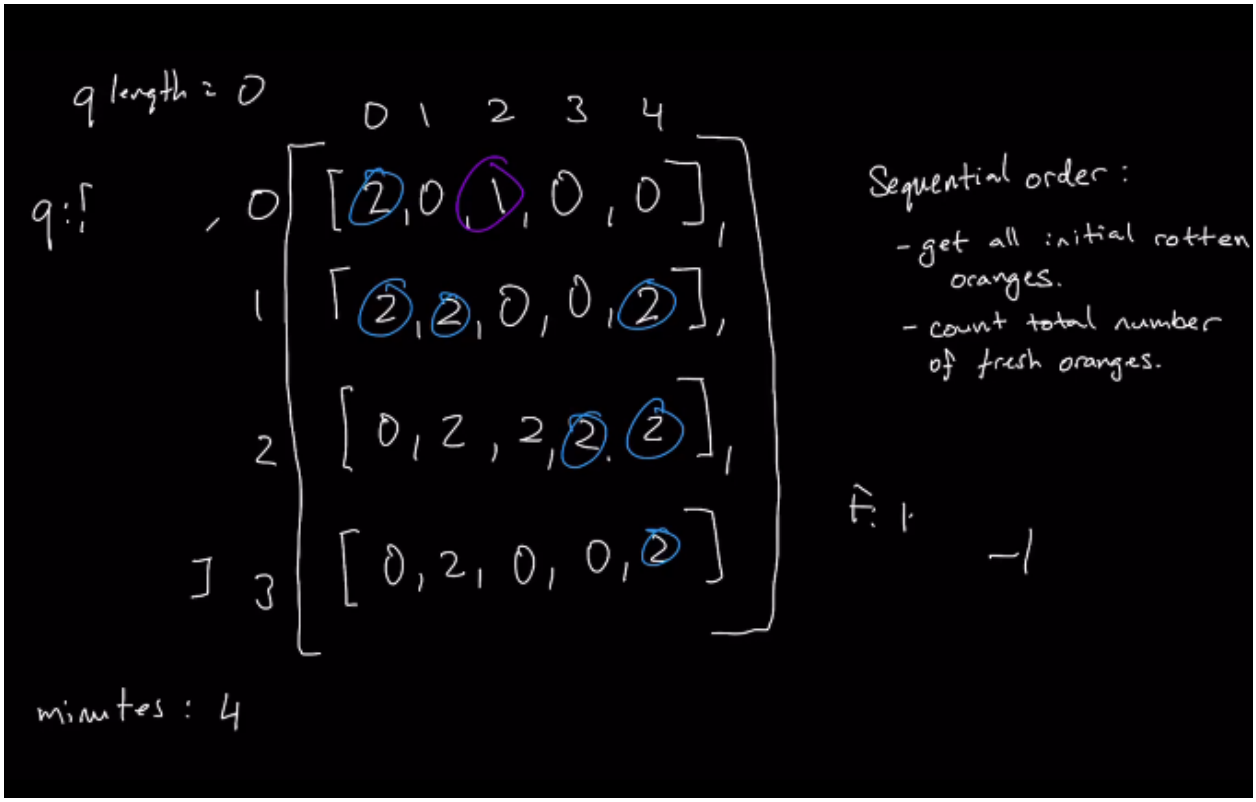
And among de exploration count the fresh oranges and notice other stuff.

We should keep track of each iteration





We are done with [2, 4]
And minutes are 2



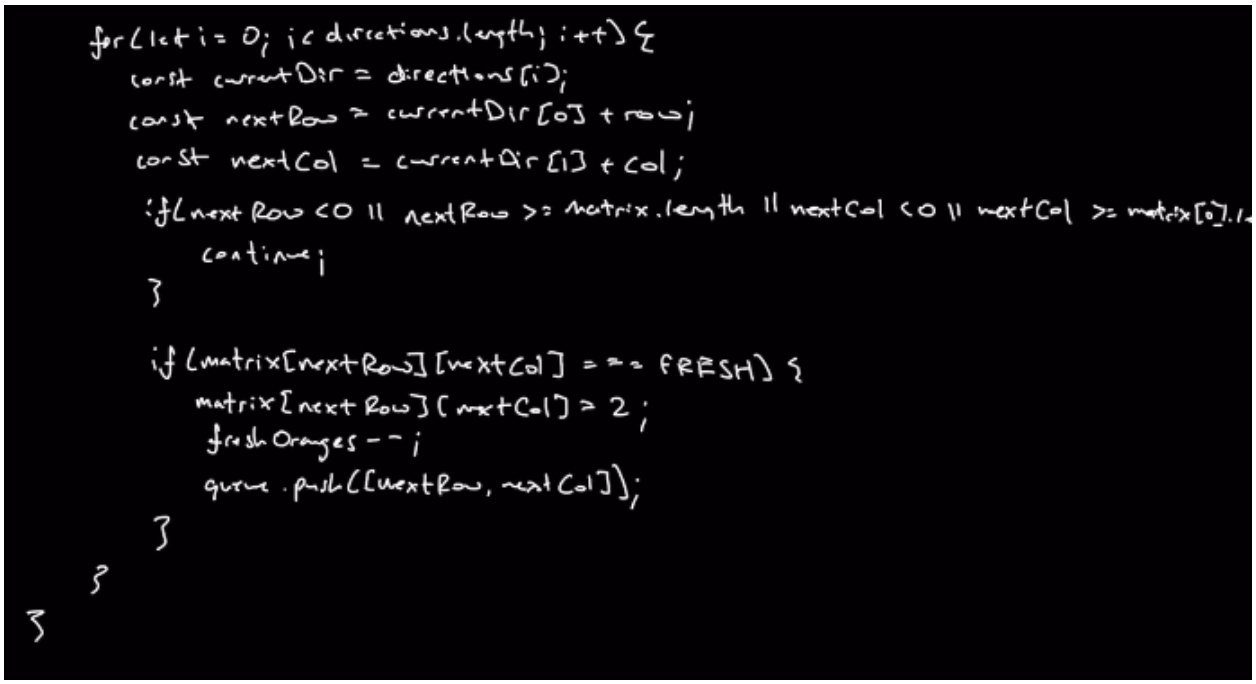
Sequential order :
- Count fresh oranges
- put rotten oranges into queue

BFS :
- use queue size to track minutes
- process rotting oranges
  - rot adjacent fresh oranges
  - push into queue
  - decrement fresh Oranges count

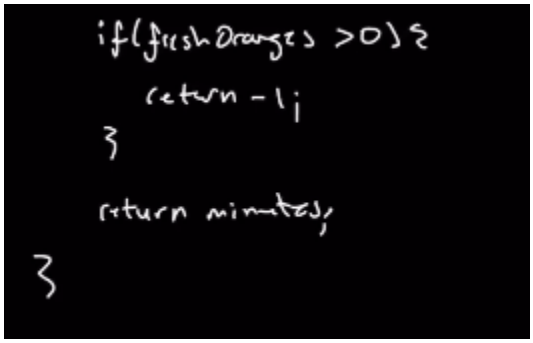
```
const directions = [[-1,0],[0,1],[1,0],[0,-1]];

const ROTTEN = 2;
const FRESH  = 1;
const EMPTY = 0;

const orangesRotting = function (matrix) {
    if(matrix.length === 0) return 0;
```

```
const queue = [];
let freshOranges = 0;
for(let row = 0; row < matrix.length; row++) {
  for (let col = 0; col < matrix[0].length; col++) {
    if(matrix[row][col] === ROTTEN) {
      queue.push([row, col]);
    }
    if(matrix[row][col] === FRESH) {
      freshOranges++;
    }
  }
}
```

```
let currentQueueSize = queue.length;
let minutes = 0;
while(queue.length > 0) {
  if(currentQueueSize === 0) {
    minutes++;
    currentQueueSize = queue.length;
  }
  const currentOrange = queue.shift();
  currentQueueSize--;
  const row = currentOrange[0];
  const col = currentOrange[1];
```



```
    for(let i = 0; i< directions.length; ++i) {
      const currentDir = directions[i];
      const nextRow = currentDir[0] + row;
      const nextCol = currentDir[1] + col;
      if(nextRow < 0 || nextRow >= matrix.length || nextCol < 0 || nextCol >= matrix[0].l
          continue;
      }

      if (matrix[nextRow][nextCol] === FRESH) {
         matrix[nextRow][nextCol] = 2;
         freshOranges--;
         queue.push([nextRow, nextCol]);
      }
    }
}
```



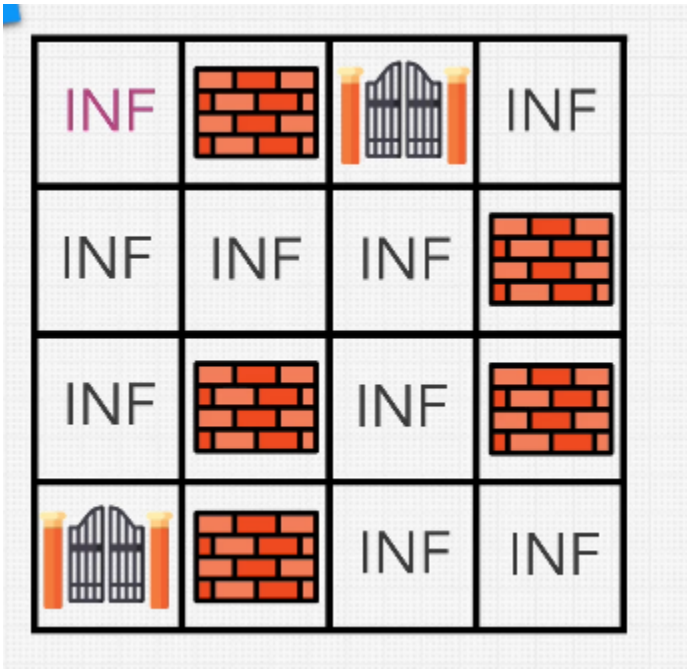
```
    if(freshOranges > 0) {
       return -1;
    }

    return minutes;
}
```

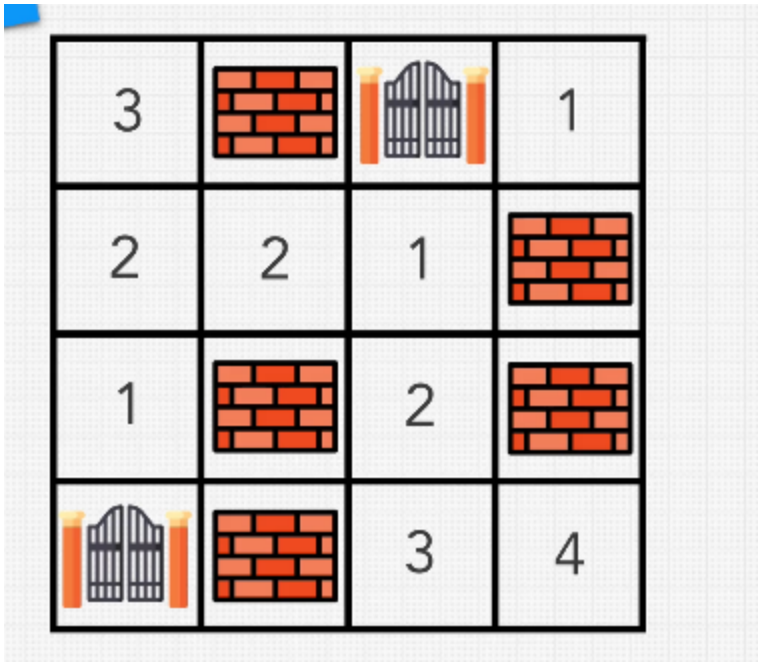
Walls and gates

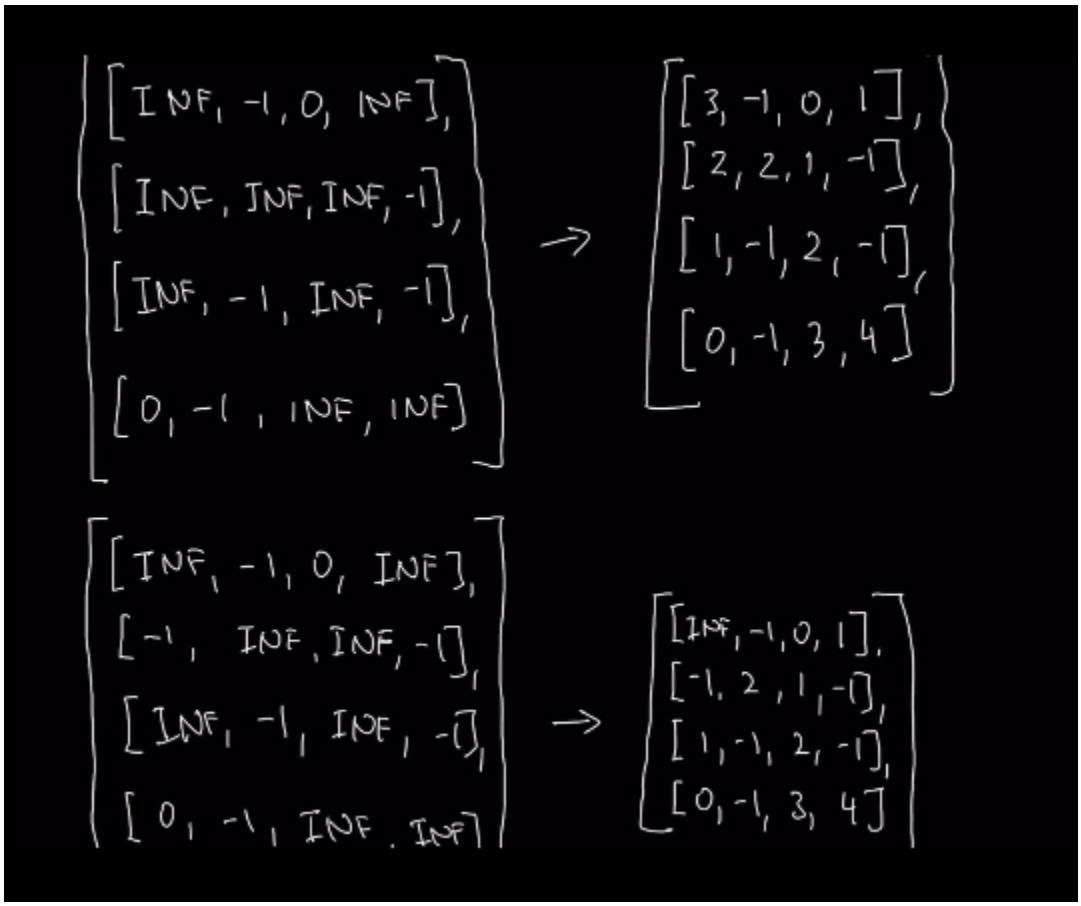
Problem

Given a 2D array containing -1's (walls), 0's (gates) and INF's (empty room). Fill each empty room with the number of steps to the nearest gate.

If it is impossible to reach a gate, leave INF as the value. INF is equal to 2147483647.

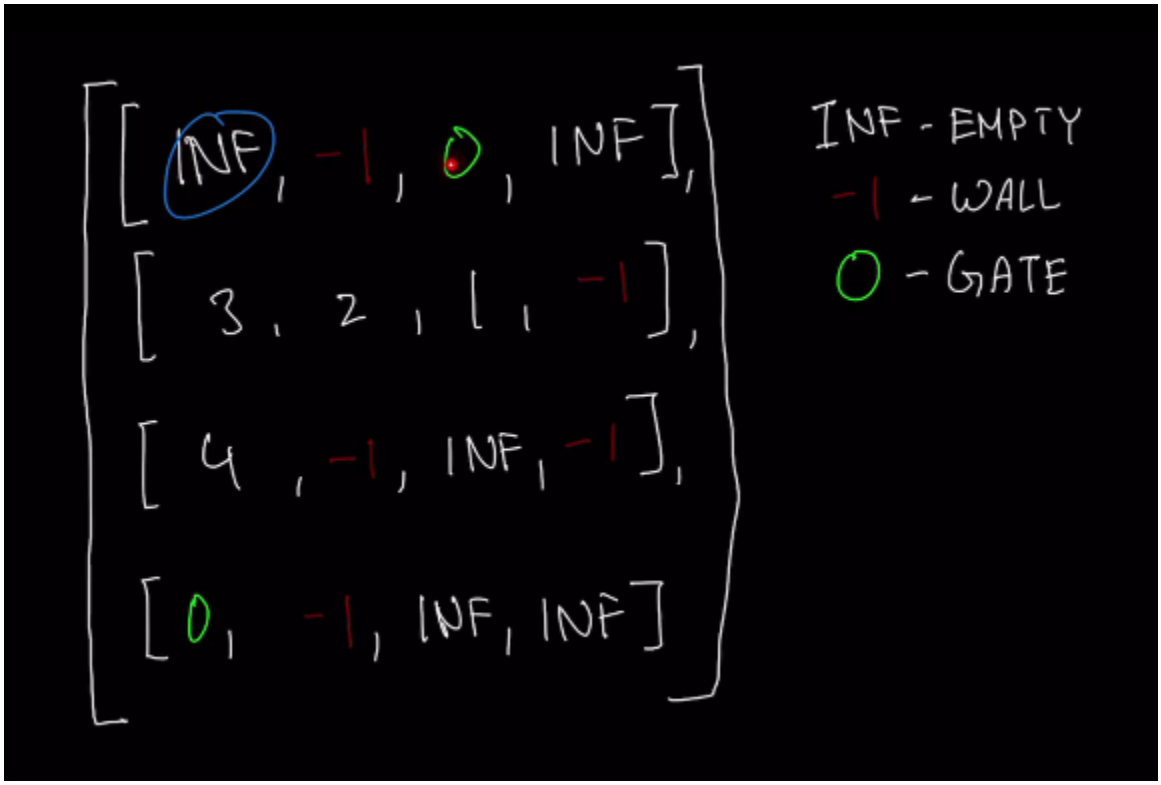



3
1
2
2
1
1
2
3
4


[INF, -1, 0, INF],
[INF, INF, INF, -1],
[INF, -1, INF, -1],
[0, -1, INF, INF]
→
[3, -1, 0, 1],
[2, 2, 1, -1],
[1, -1, 2, -1],
[0, -1, 3, 4]
[INF, -1, 0, INF],
[-1, INF, INF, -1],
[INF, -1, INF, -1],
[0, -1, INF, INF]
→
[INF, -1, 0, 1],
[-1, 2, 1, -1],
[1, -1, 2, -1],
[0, -1, 3, 4]

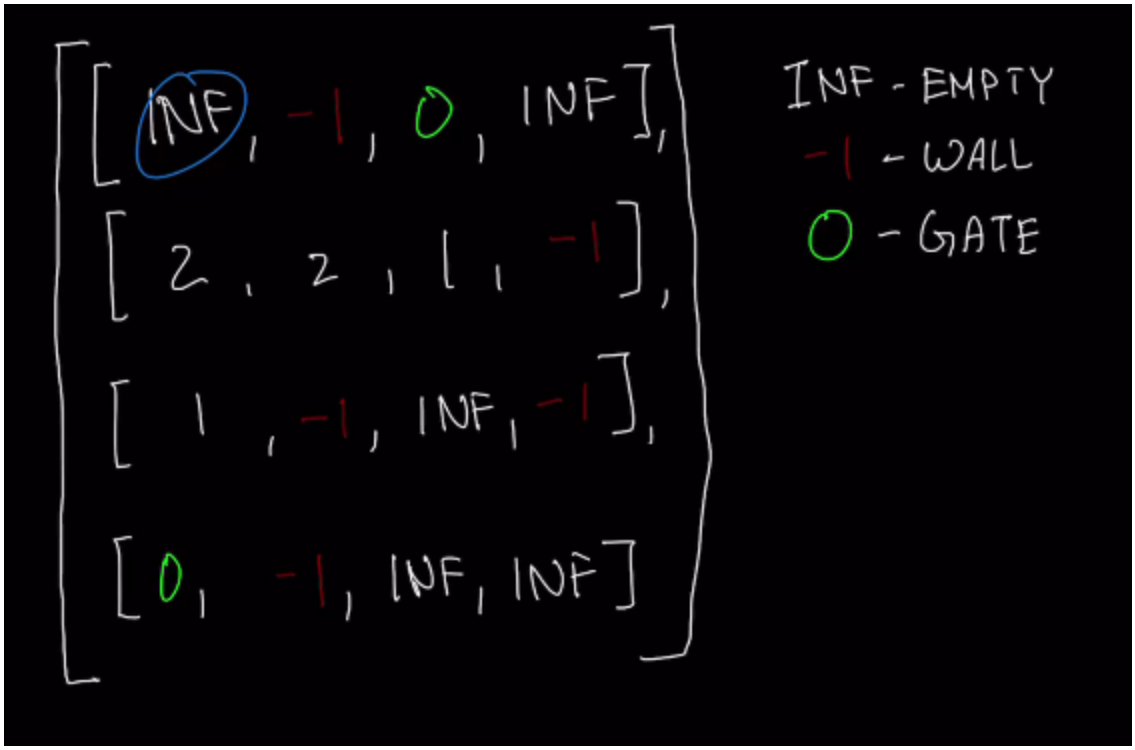
Method: 2D Array DFS or BFS

We chose the gates as starting point



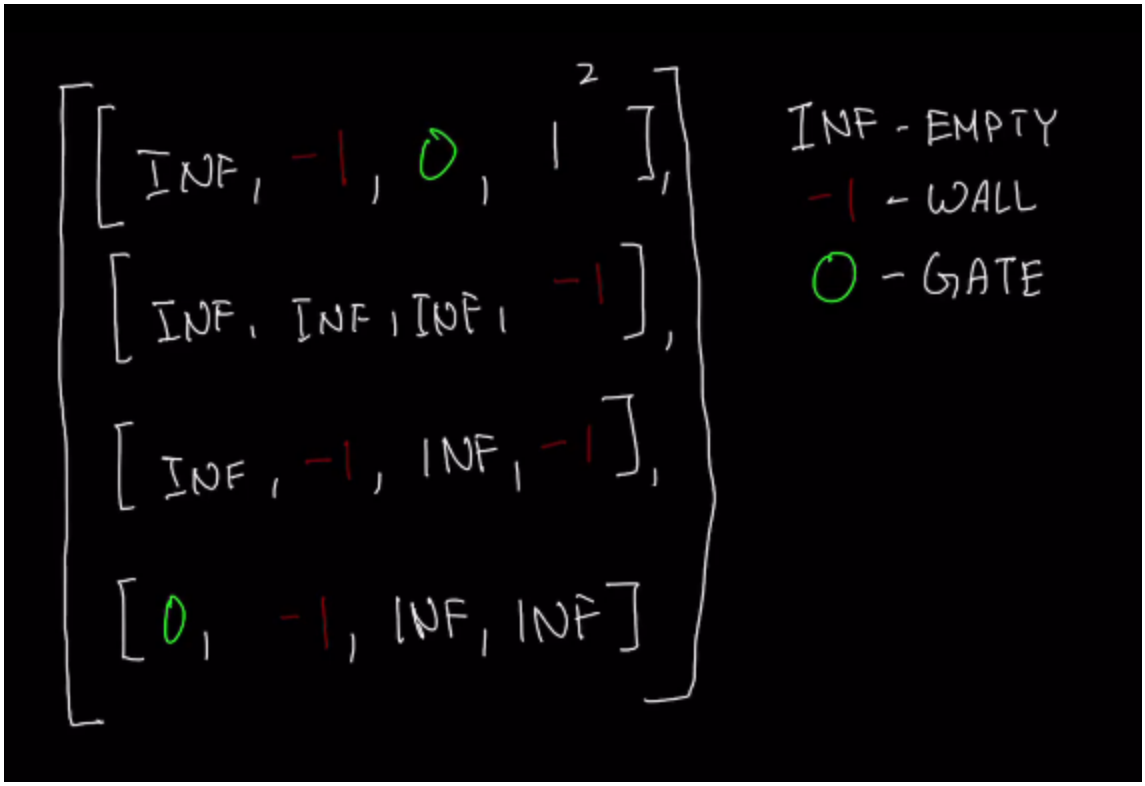
Then propagate the total steps

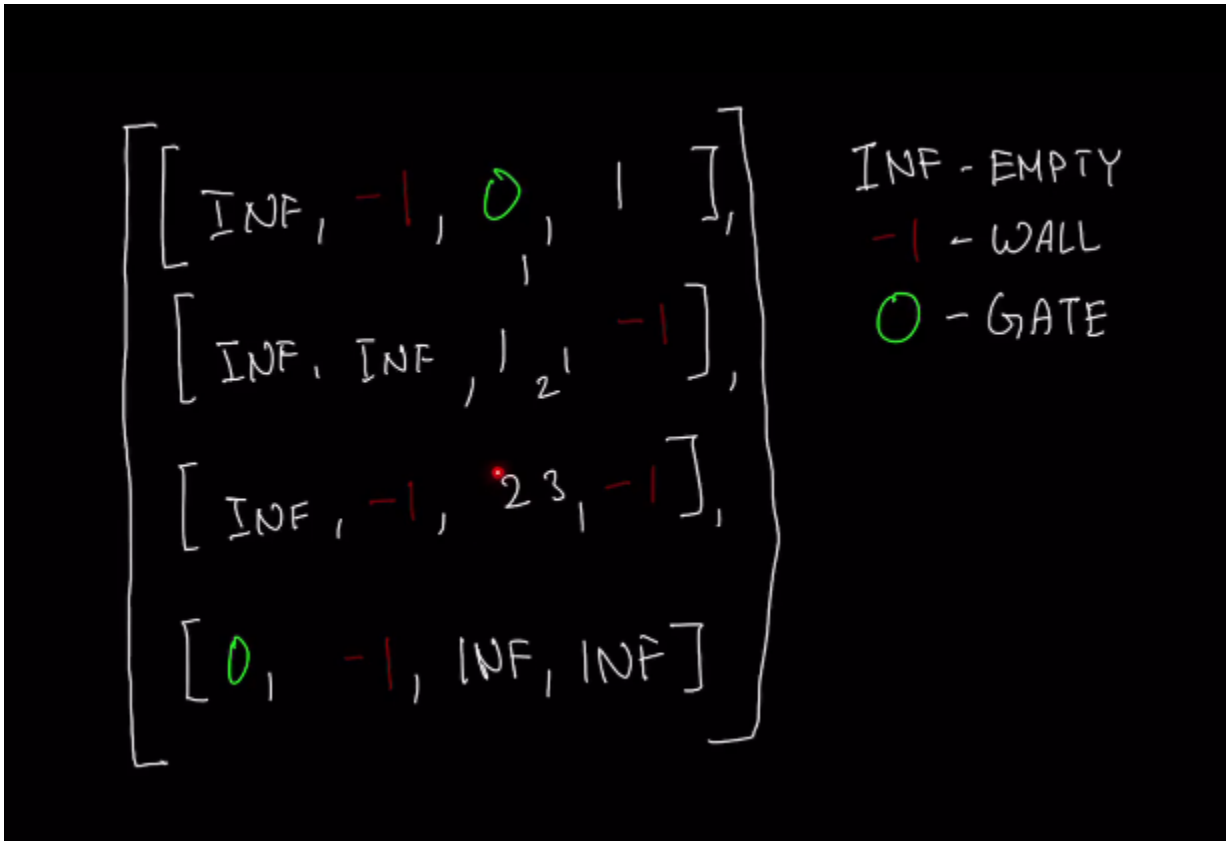
Then with the other gate we compare



If the steps as less or higher

First use sequential


[[INF, -1, 0, 1],
[INF, INF, INF, -1],
[INF, -1, INF, -1],
[0, -1, INF, INF]]
INF - EMPTY
-1 - WALL
0 - GATE

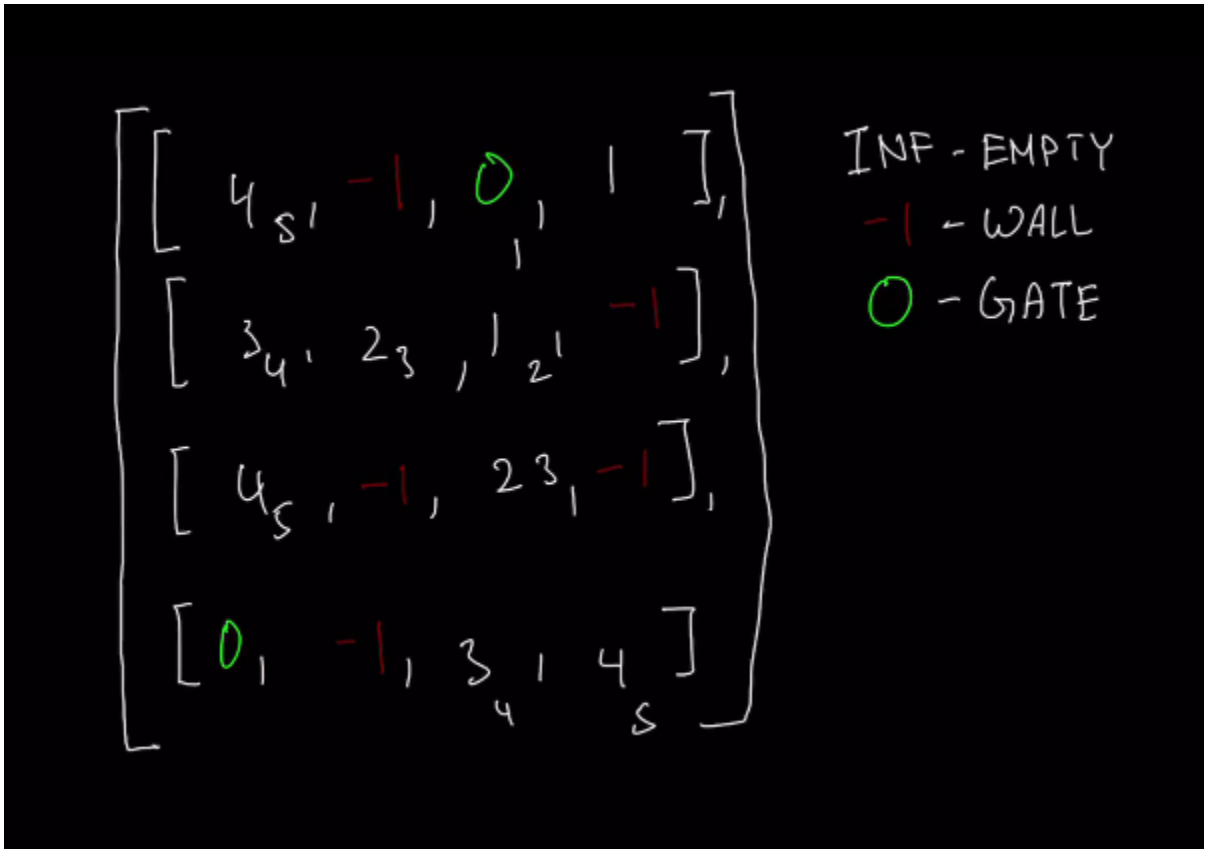

[[INF, -1, 0, 1],
[INF, INF, 1, -1],
[INF, -1, 2, -1],
[0, -1, INF, INF]]
INF - EMPTY
-1 - WALL
0 - GATE

We continue increasing

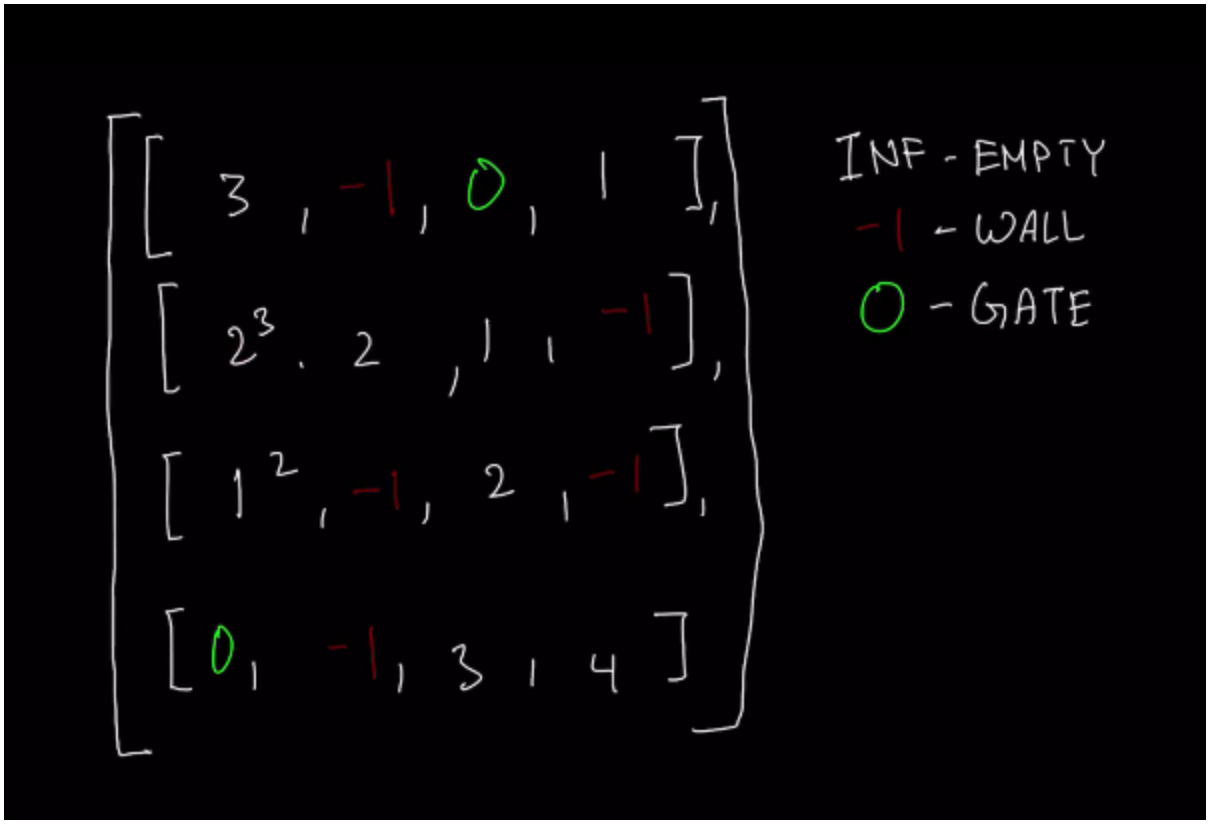
From the first gate

With DFS

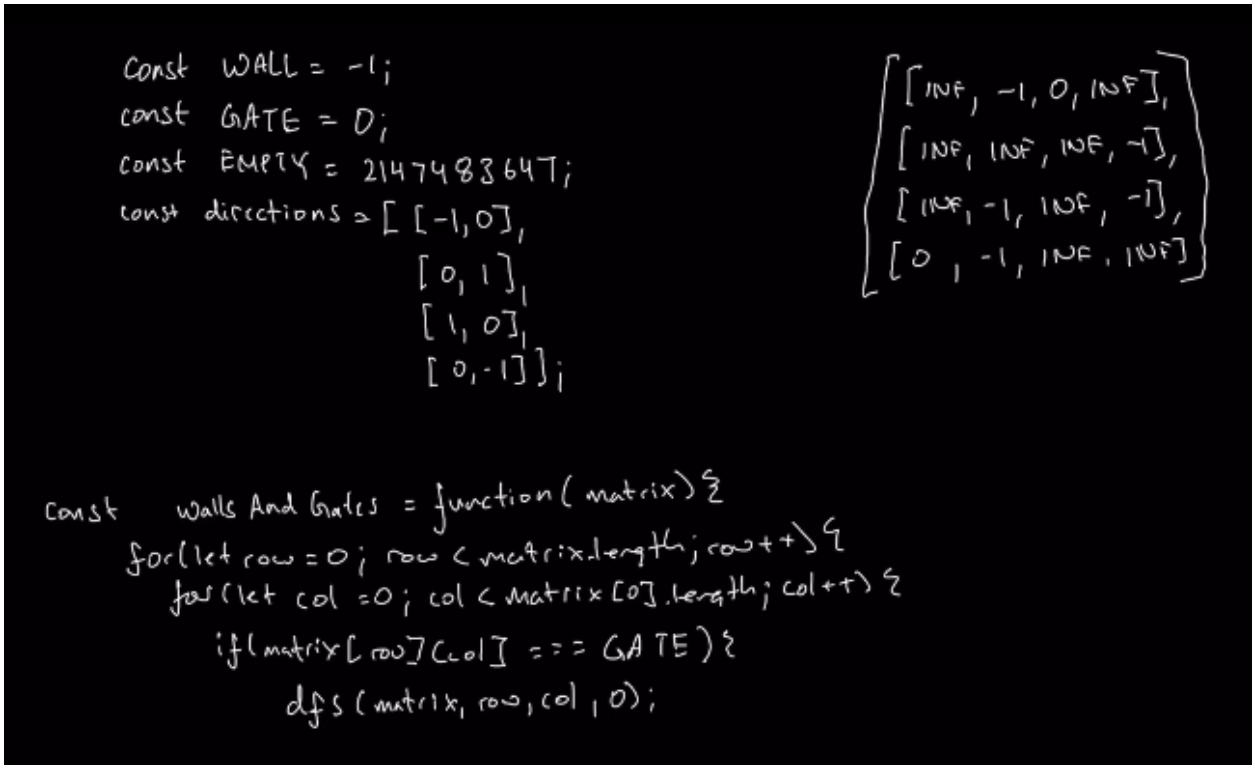
Generate this

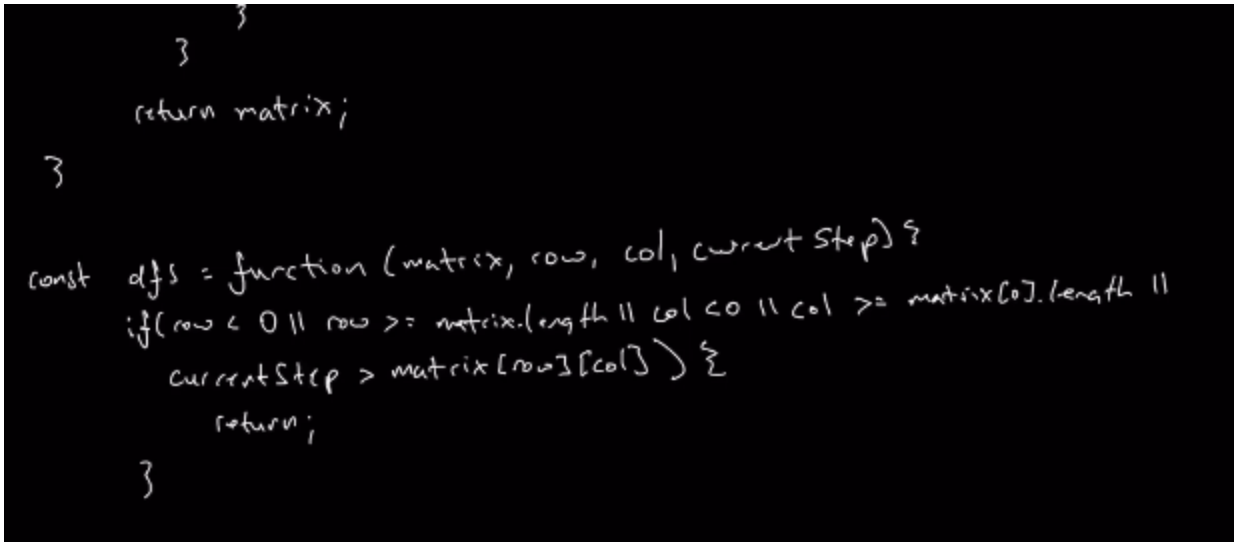


Then from the second gate

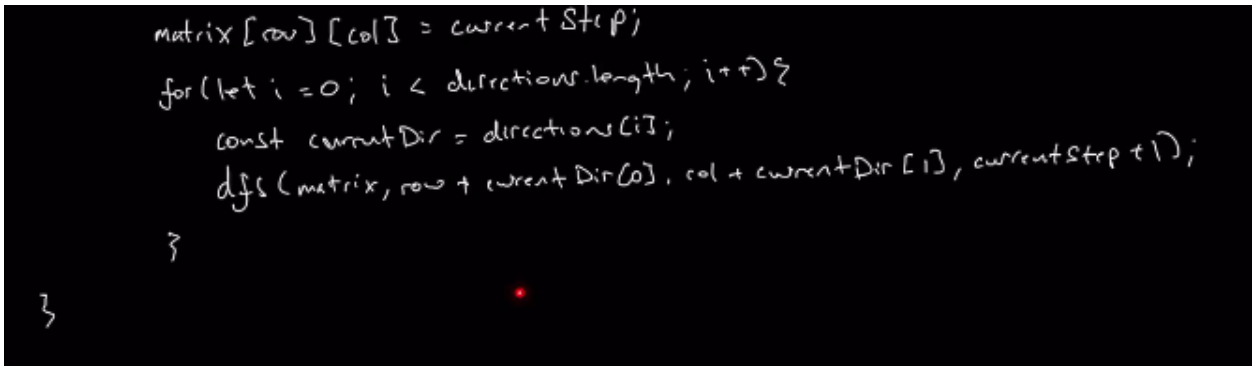


When get a higher value it’s not necessary to continue





```
        }
    }
    return matrix;
};

const dfs = function (matrix, row, col, currentStep) {
    if(row < 0 || row >= matrix.length || col < 0 || col >= matrix[0].length ||
      currentStep > matrix[row][col]) {
        return;
    }
```



```
    matrix[row][col] = currentStep;
    for(let i = 0; i < directions.length; i++) {
        const currentDir = directions[i];
        dfs(matrix, row + currentDir[0], col + currentDir[1], currentStep + 1);
    }
};
```

Important: here it was very important to begin the algorithm from 0s.

Graphs

In interview this questions are very abstract and we have to figure out what to do.

Time needed to inform all employee

A company has n employees with unique IDs from 0 to n-1. The head of the company has the ID *headID*.

head of company

n = 8 *8 employees: 0, 1, 2, 3, 4, 5, 6, 7*

headID = 4

You will receive a *managers* array where *managers[i]* is the ID of the manager for employee *i*. Each employee has one direct manager. The company head has no manager (*managers[headID]* = -1). It's guaranteed the subordination relationships will have a tree structure.

8 employees: 0, 1, 2, 3, 4, 5, 6, 7 *headID = 4*

0 1 2 3 4 5 6 7

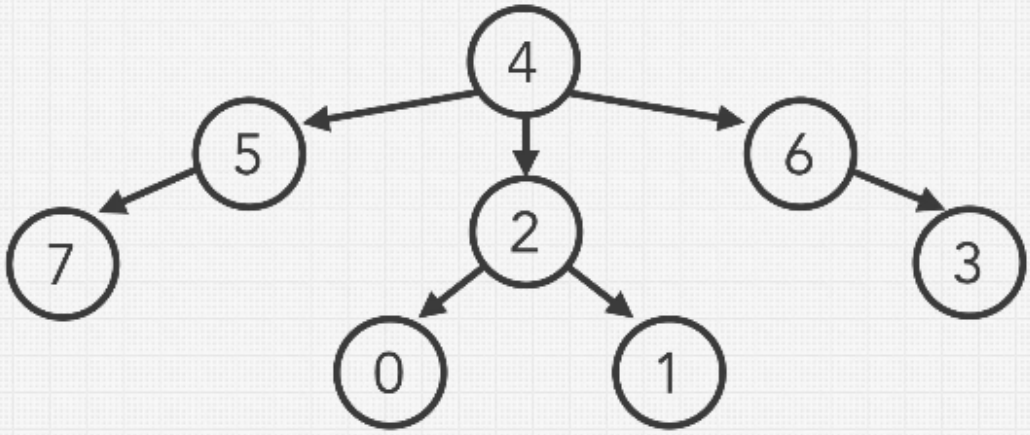
managers = [2, 2, 4, 6, -1, 4, 4, 5]

This is a N-Tree

8 employees: 0, 1, 2, 3, 4, 5, 6, 7 *headID = 4*

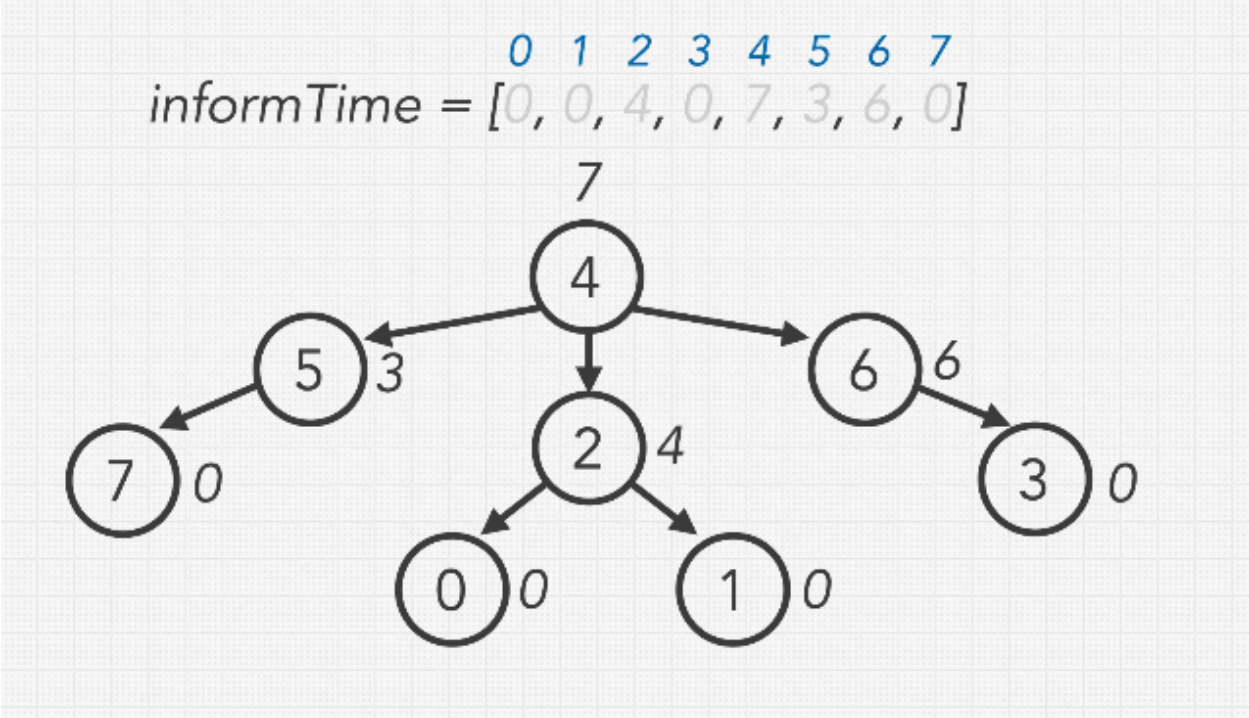
0 1 2 3 4 5 6 7

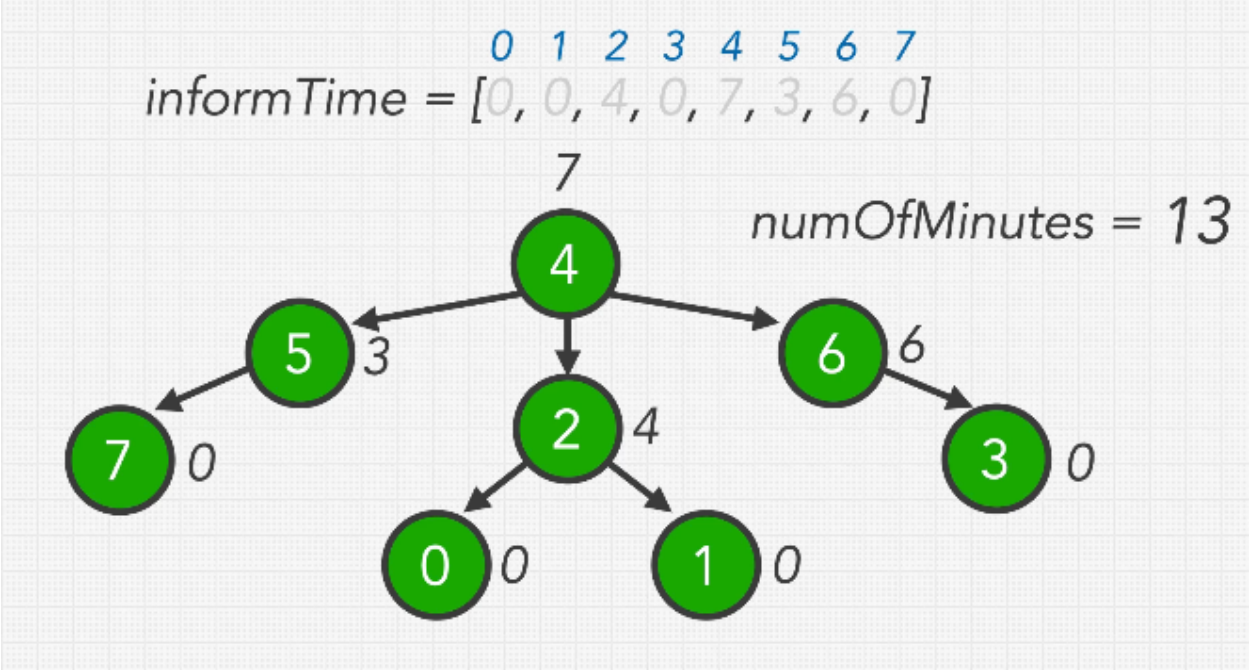
managers = [2, 2, 4, 6, -1, 4, 4, 5]



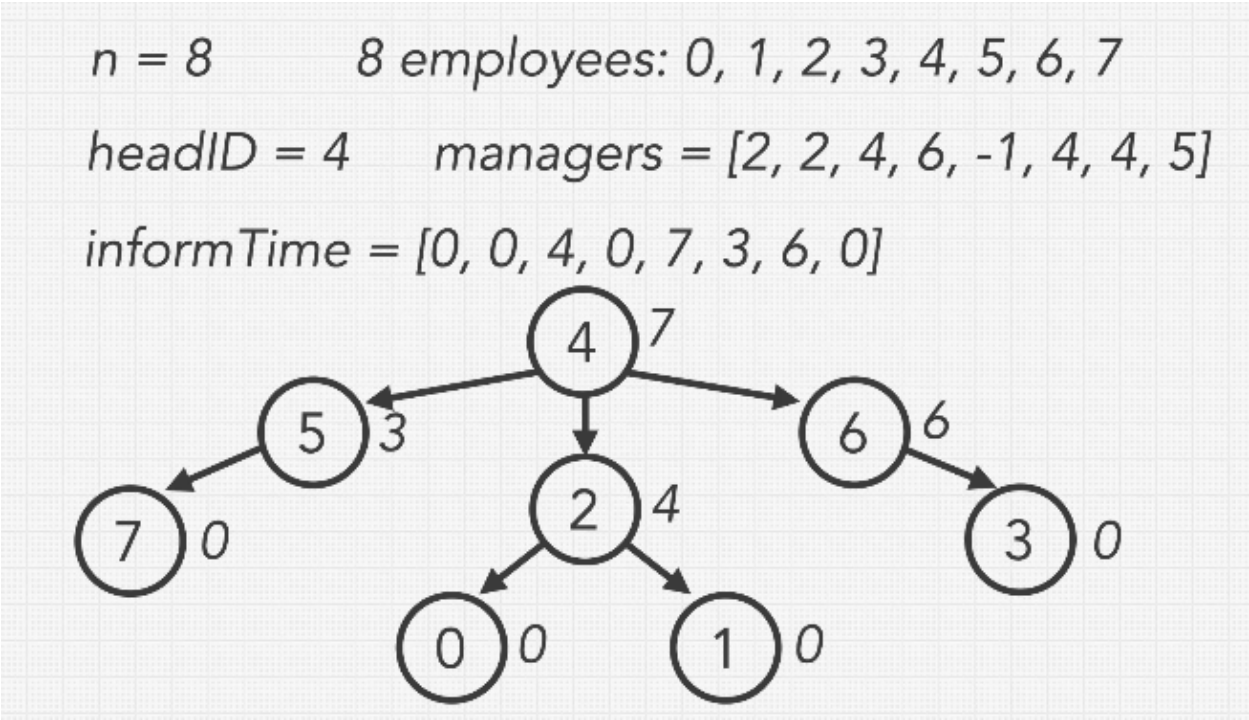
The head of the company wants to inform all employees of news. He will inform his direct subordinates who will inform their direct subordinates and so on until everyone knows the news.

You will receive an *informTime* array where *informTime[i]* is the time it takes for employee *i* to inform all their direct subordinates. Return the total number of minutes it takes to inform all employees of the news.





The information is made at the same time !!!



Verify constraints

It’s a tree but is easier to represent it as a graph

n = 8 headID = 4

managers = [2,2,4,6,-1,4,4,5]

informTime = [0,0,4,0,7,3,6,0]

13

n = 7 headID = 6

managers = [1,2,3,4,5,6,-1]

informTime = [0,6,5,4,3,2,1]

2(

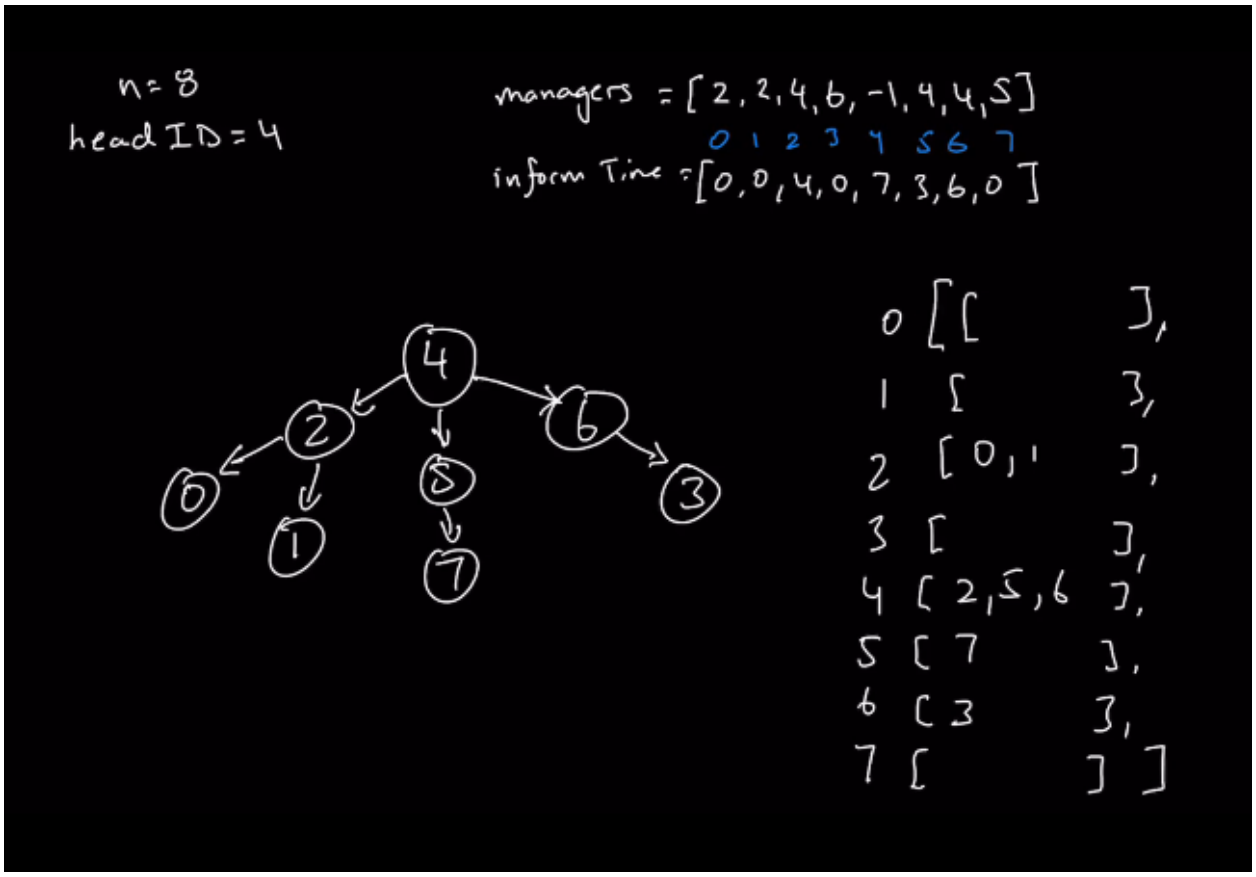
n = 1 headID = 0

managers = [-1]

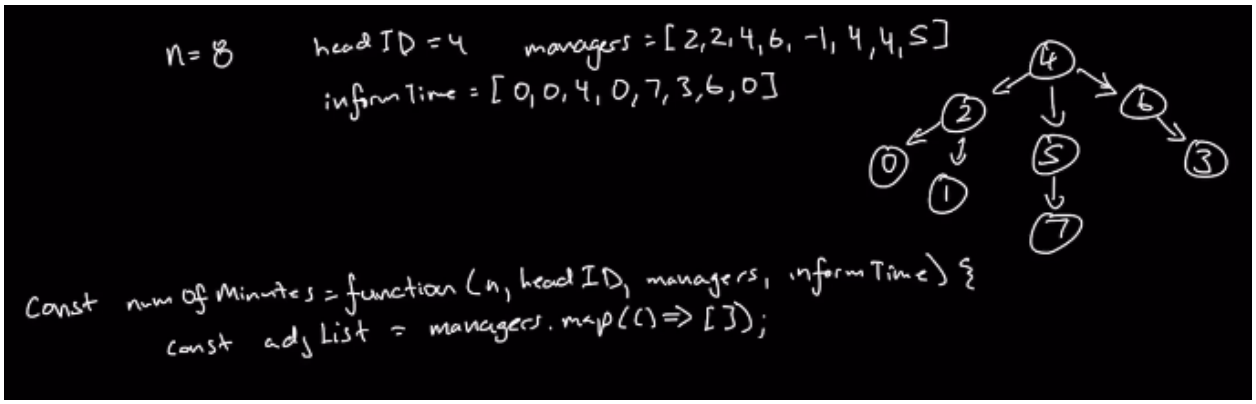
informTime = [0]

0

Method: DFS - Adjacency array Graph



We get the maximum value for each branch from the root



```
    for (let e = 0; e < n; e++) {
        const manager = managers[e];
        if (manager === -1) continue;
        adjList[manager].push(e);
    }
    return dfs(headID, adjList, informTime);
};
```

```
const dfs = function(currentID, adjList, informTime) {
    if (adjList[currentID].length === 0) {
        return 0;
    }

    let max = 0;
    const subordinates = adjList[currentID];
    for (let i = 0; i < subordinates.length; i++) {
        max = Math.max(max, dfs(subordinates[i], adjList, informTime));
    }
    return max + informTime[currentID];
};
```

T: O(N)
S: O(N)

Course Scheduler

Problem

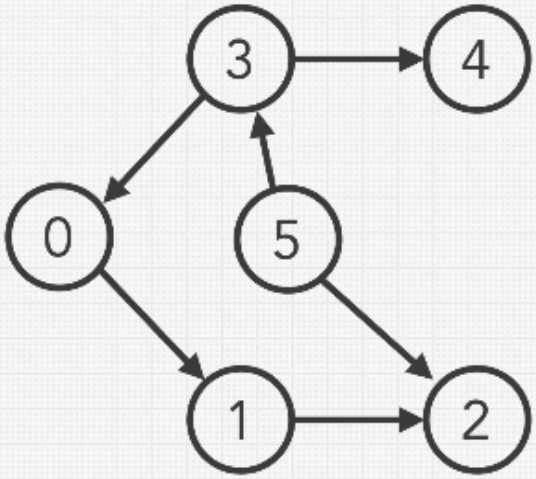
There are a total of n courses to take, labeled from 0 to n-1.

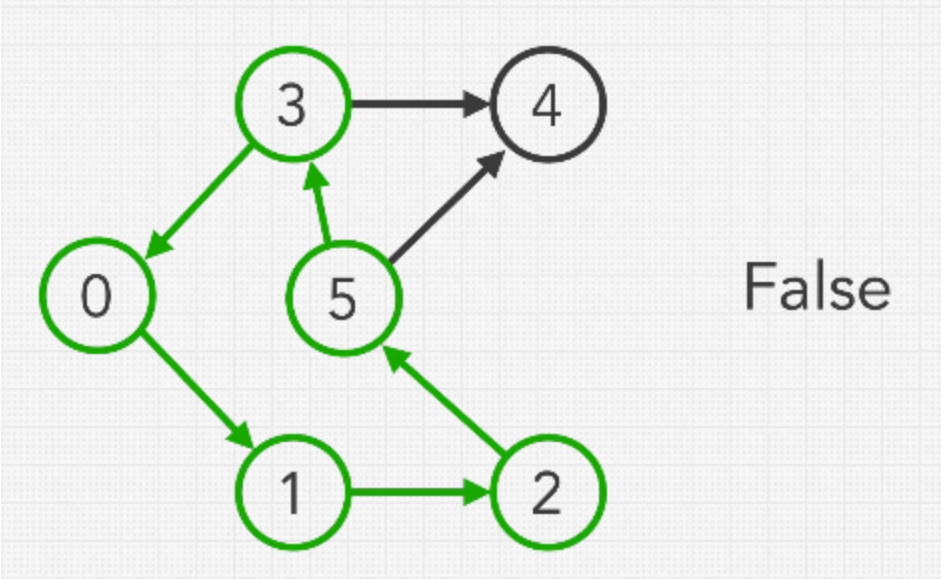
Some courses have prerequisite courses. This is expressed as a pair i.e. [1, 0] which indicates you must take course 0 before taking course 1.

Given the total number of courses and an array of prerequisite pairs, return if it is possible to finish all courses.

n = 6 6 courses: 0, 1, 2, 3, 4, 5

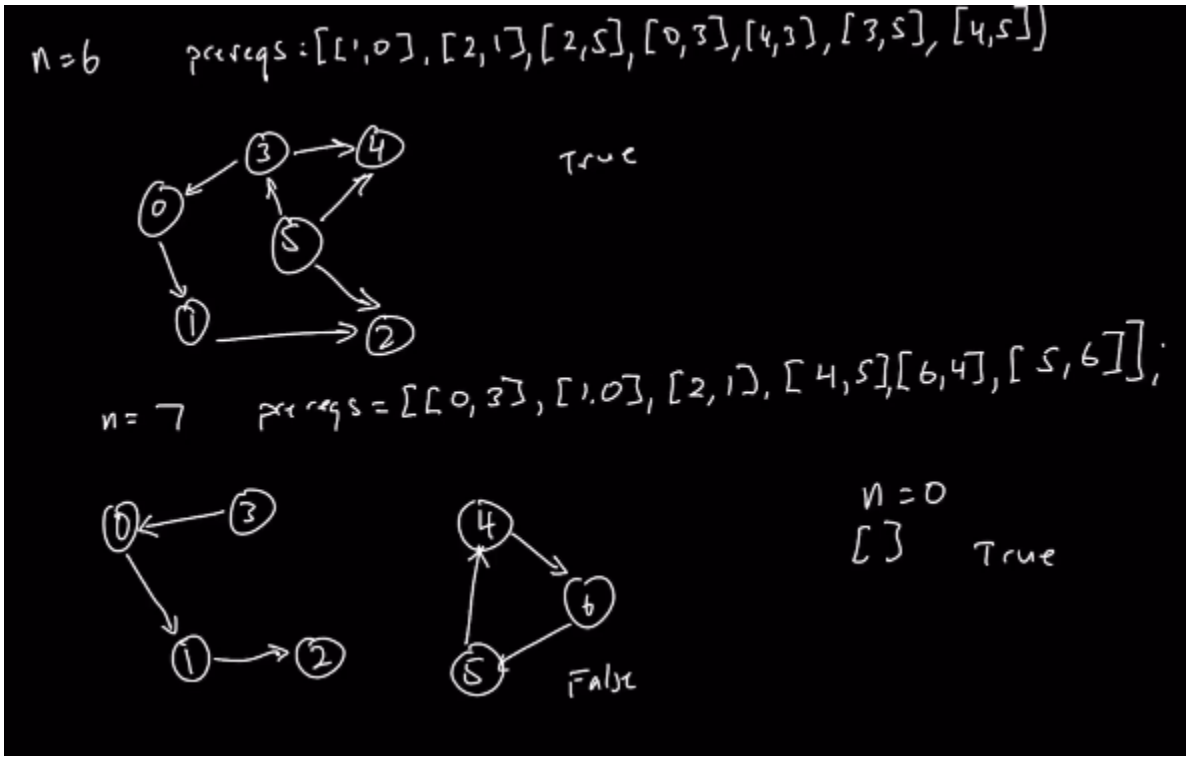
prerequisites: [[1, 0], [2, 1], [2, 5], [0, 3], [4, 3], [3, 5], [4,5]]





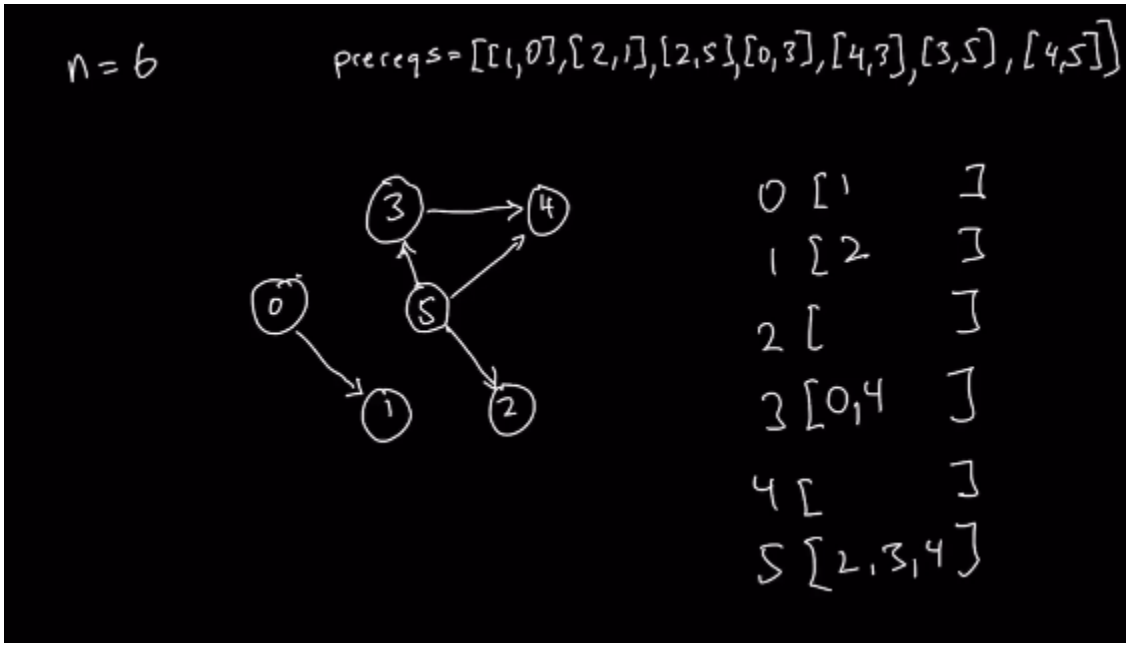
Method Inefficient: Brute force and adjacency list

Determine if a cycle exists. If exist a cycle then it's impossible complete the course



If a pair is defined in the problem determining a relationship, it's highly possible that graph is the best option to represent.

First represent the adjacency of the graph




```
const canFinish = function (n, prerequisites) {
    const adjList = new Array(n).fill(0).map(() => []);
    for(let i=0; i< prerequisites.length; i++) {
        const pair = prerequisites[i];
        adjList[pair[1]].push(pair[0]);
    }
    for(let v = 0; v< n; v++) {
```

```
const queue = [ ];
const seen = { };
for (let i = 0; i < adjList[v].length; i++) {
  queue.push(adjList[v][i];
}
```

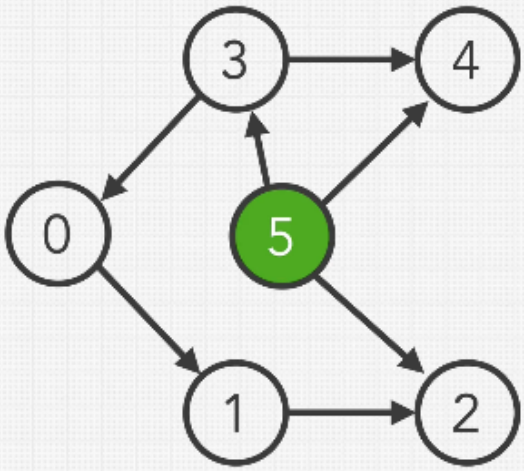
```
while (queue.length) {
  const current = queue.shift();
  seen[current] = true;
  if (current === v) return false;
```

```
  const adjacent = adjList[current];
  for (let i = 0; i < adjacent.length; i++) {
    const next = adjacent[i];
    if (!seen[next]) {
      queue.push(next);
    }
  }
}
```

```
T: O(P + n^3)
S: O(n)
```

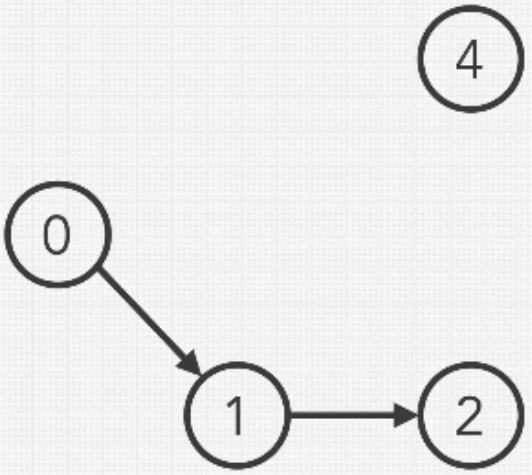
Method: Topological order

prerequisites: [[1, 0], [2, 1], [2, 5], [0, 3], [4, 3], [3, 5], [4,5]]



	inDegree:	adjList:
0	1,	[1],
1	1,	[2],
2	2,	[],
3	1,	[0, 4],
4	2,	[],
5	0	[2, 3, 4]

prerequisites: [[1, 0], [2, 1], [2, 5], [0, 3], [4, 3], [3, 5], [4,5]]



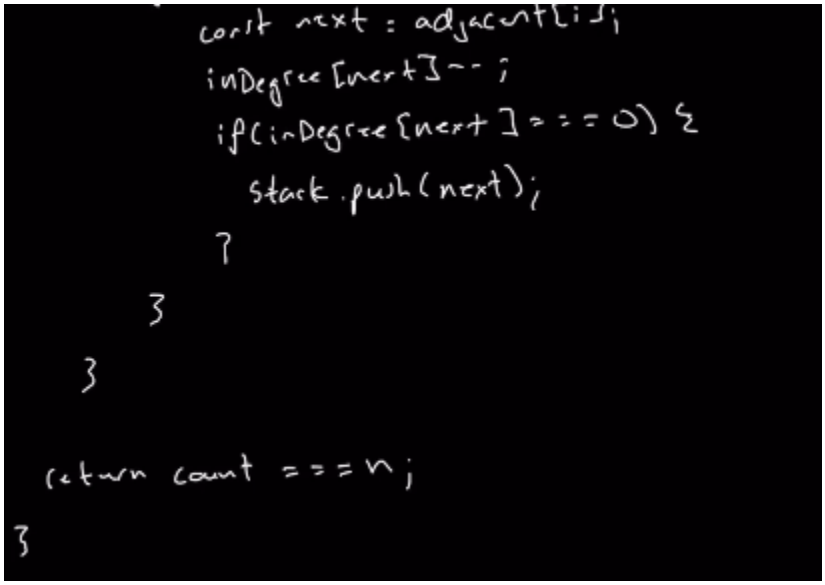
	inDegree:	adjList:
0	0,	[1],
1	1,	[2],
2	1,	[],
3	0,	[0, 4],
4	0,	[],
5	0	[2, 3, 4]

prerequisites: [[1, 0], [2, 1], [2, 5], [0, 3], [4, 3], [3, 5], [4,5]]

	inDegree:	adjList:
0	0,	[1],
1	0,	[2],
2	0,	[],
3	0,	[0, 4],
4	0,	[],
5	0	[2, 3, 4]

```
const canFinish = function(n, prerequisites) {
    const inDegree = new Array(n).fill(0);
    const adjList = inDegree.map(() => []);

    for (let i=0; i < prerequisites.length; i++) {
        const pair = prerequisites[i];
        inDegree[pair[0]]++;
        adjList[pair[1]].push(pair[0]);
```

```
  const stack = [];

  for( let i=0; i < inDegree.length; i++ ) {
    if (inDegree[i] === 0) {
      stack.push(i);
    }
  }

  let count = 0;

  while( stack.length ) {
    const current = stack.pop();
    count++;
    const adjacent = adjList[current];
    for( let i = 0; i < adjacent.length; i++) {
```



```
        const next = adjacent[i];
        inDegree[next]--;
        if(inDegree[next] === 0) {
          stack.push(next);
        }
      }
    }

  return count === n;
}
```

Network time delay

Problem

There are *n* network nodes labelled 1 to N.

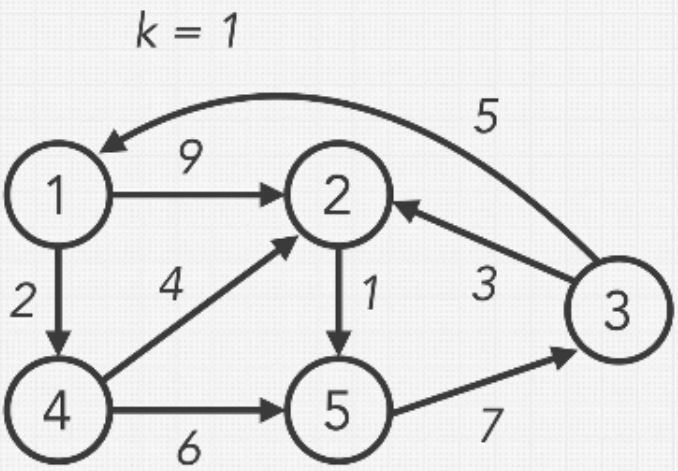
Given a *times* array, containing edges represented by arrays [*u*, *v*, *w*] where *u* is the source node, *v* is the target node, and *w* is the time taken to travel from the source node to the target node.

Send a signal from node *k*, return how long it takes for all nodes to receive the signal. Return -1 if it's impossible.

n = 5 *nodes* = 1, 2, 3, 4, 5

times = [[1, 2, 9], [1, 4, 2], [2, 5, 1], [4, 2, 4], [4, 5, 6], [3, 2, 3], [5, 3, 7], [3, 1, 5]]

Send a signal from node *k*, return how long it takes for all nodes to receive the signal. Return -1 if it's impossible.



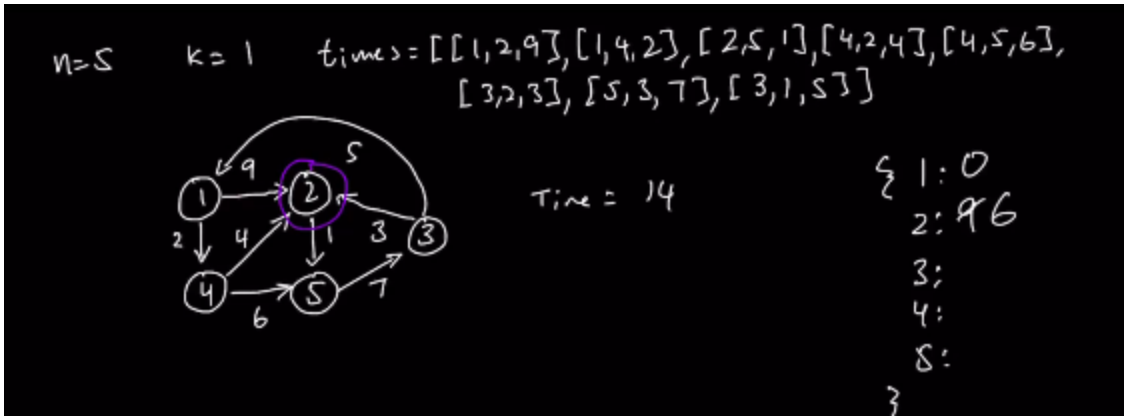
The signal wants to travel as fast as possible.

This is an optimization problem

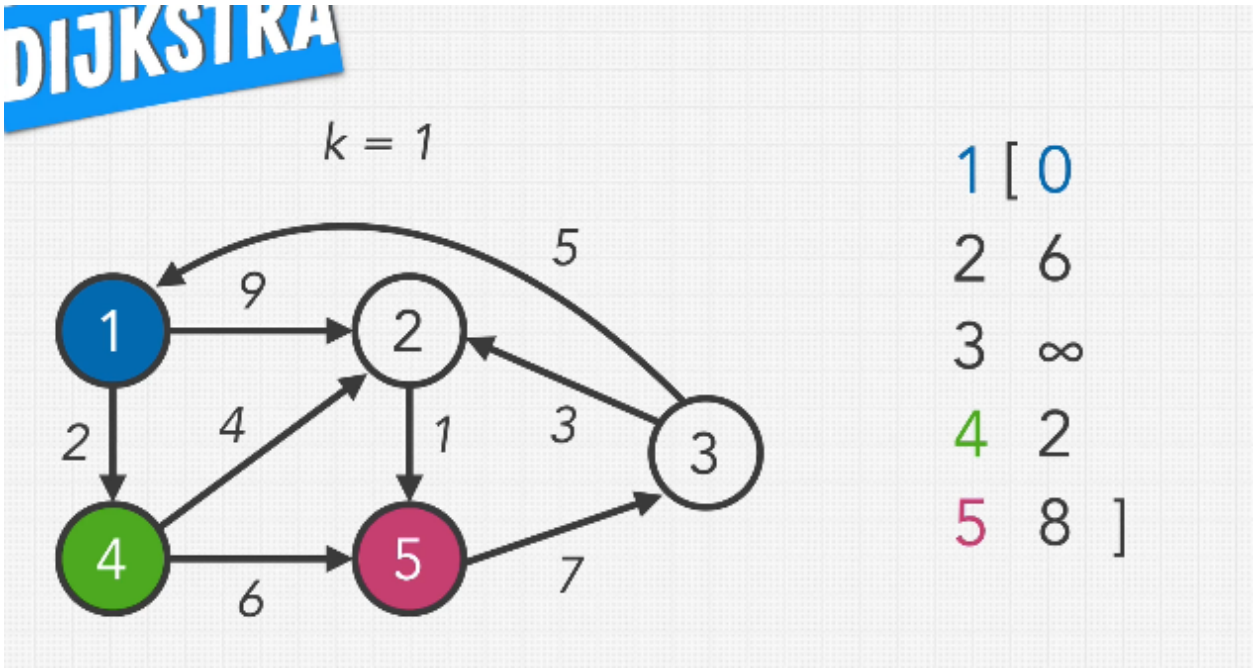
The optimal is search the minimum

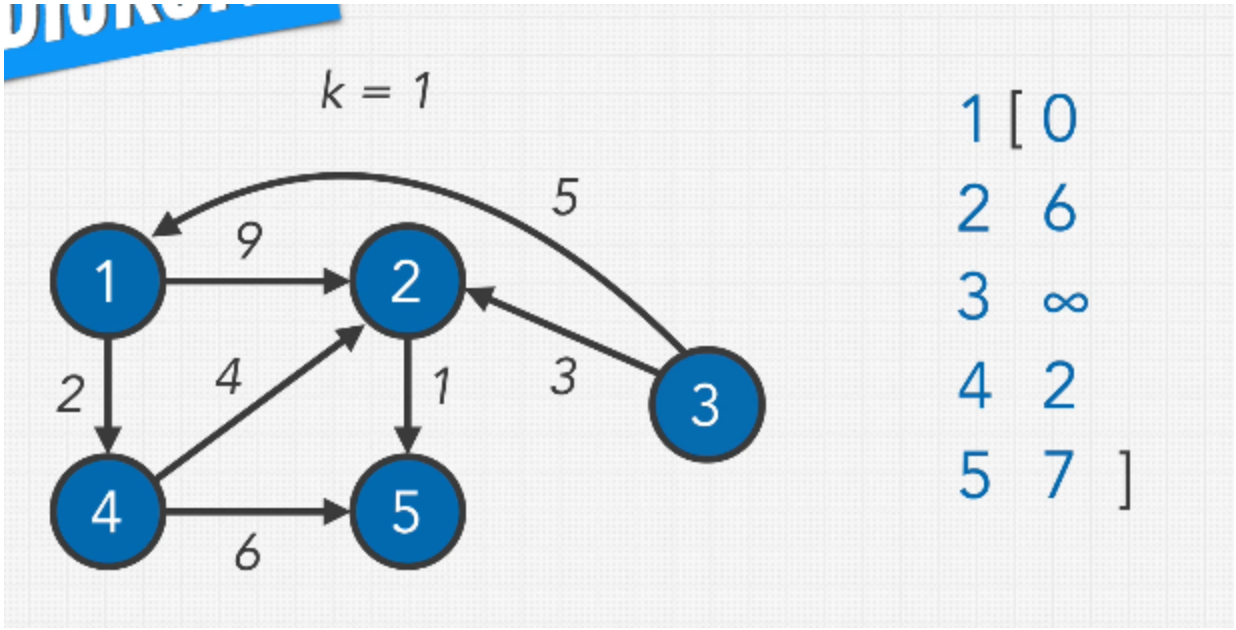
Method 1: DFS

We store the weights and keep track the smallest times



Method 2 (optimal): Dijkstra





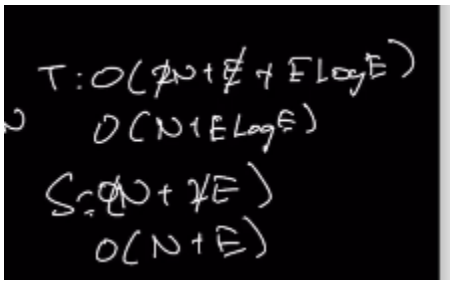
This is there is no answer

```
const networkDelayTime = function (times, N, k) {
    const distances = new Array(N).fill(Infinity);
    const adjList = distances.map(() => []);
    distances[k-1] = 0;

    const heap = new PriorityQueue((a, b) => distances[a] < distances[b]));
    heap.push(k-1);
```

```
for(let i=0; i<times.length; i++) {
    const source = times[i][0];
    const target = times[i][1];
    const weight = times[i][2];
    adjList[source-1].push([target-1, weight]);
}
```

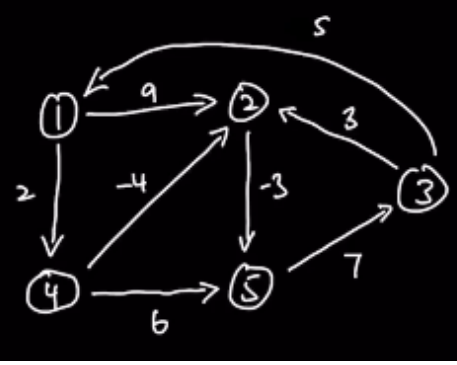
```
while (!heap.isEmpty()) {
    const currentVertex = heap.pop();
    const adjacent = adjList[currentVertex];
    for(let i=0; i< adjacent.length; i++){
        const neighboringVertex = adjacent[i][0];
        const weight = adjacent[i][1];
        if(distances[currentVertex] + weight < distances[neighboringVertex]){
```

```
            if(distances[currentVertex] + weight < distances[neighboringVertex]){
                distances[neighboringVertex] = distances[currentVertex] + weight;
                heap.push(neighboringVertex);
            }
        }
    }

    const ans = Math.max(...distances);
    return ans === Infinity ? -1 : ans;
}
```



Method 3: Bellman ford for negative

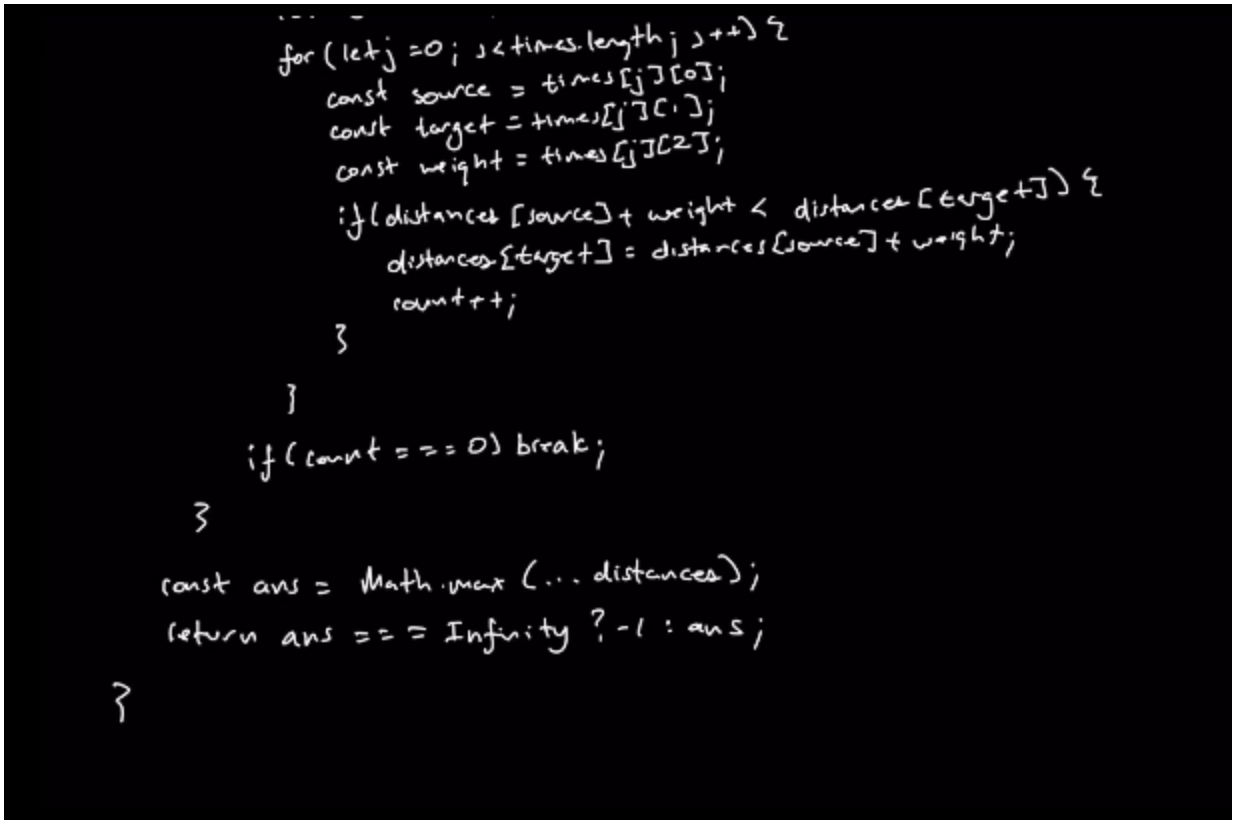
If negative cycles doesn't exist we can implement it



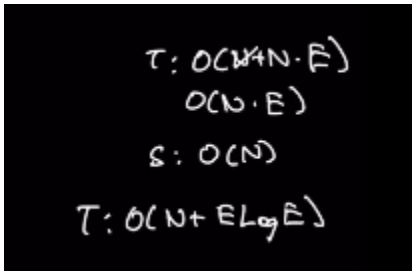
N=5 K=1

times = [[1,4,2], [1,2,9], [4,2,-4], [2,5,-3], [4,5,6], [5,3,7], [3,2,3], [3,1,5]]

```
Const networkDelayTime = function(times, N, k) {
    Const distances = new Array(N).fill(Infinity);
    distances[k-1] = 0;
    for(let i=0; i < N-1; i++) {
        let count = 0;
        for(let j=0; j<times.length; j++) {
```



```
        for (let j =0; j<times.length; j++) {
            const source = times[j][0];
            const target = times[j][1];
            const weight = times[j][2];
            if(distances[source] + weight < distances[target]) {
                distances[target] = distances[source] + weight;
                count++;
            }
        }
        if(count === 0) break;
    }
    const ans = Math.max(...distances);
    return ans === Infinity ? -1 : ans;
};
```



```
T: O(V*N·E)
   O(N·E)
S: O(N)
T: O(N+ELogE)
```

Dynamic programming

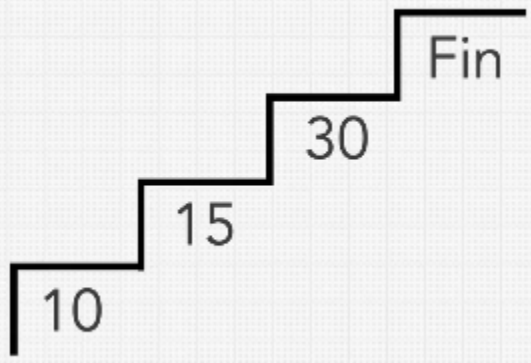
Minimum cost of climbing stairs

Problem

For a given staircase, the i-th step is assigned a non-negative cost indicated by a cost array.

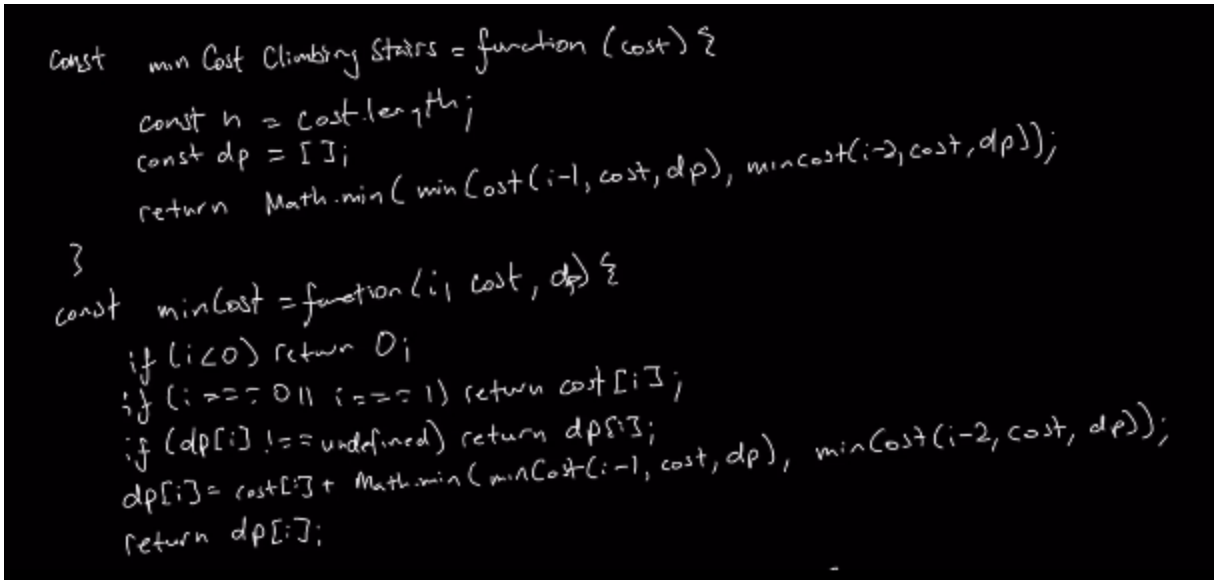
Once you pay the cost for a step, you can either climb one or two steps. Find the minimum cost to reach the top of the staircase. Your first step can either be the first or second step.

cost = [10, 15, 30]

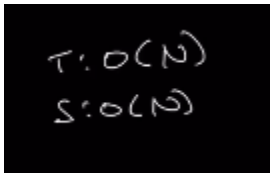


Decisions -> possible graph or tree

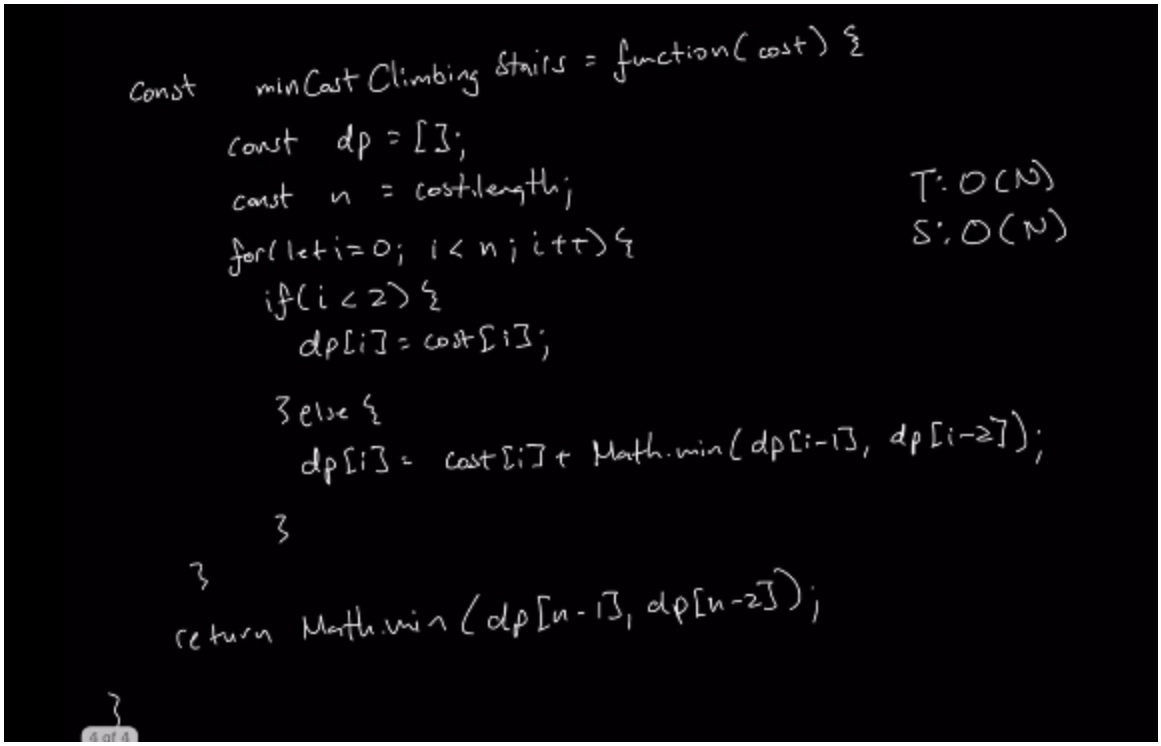
Method: DP and recursion



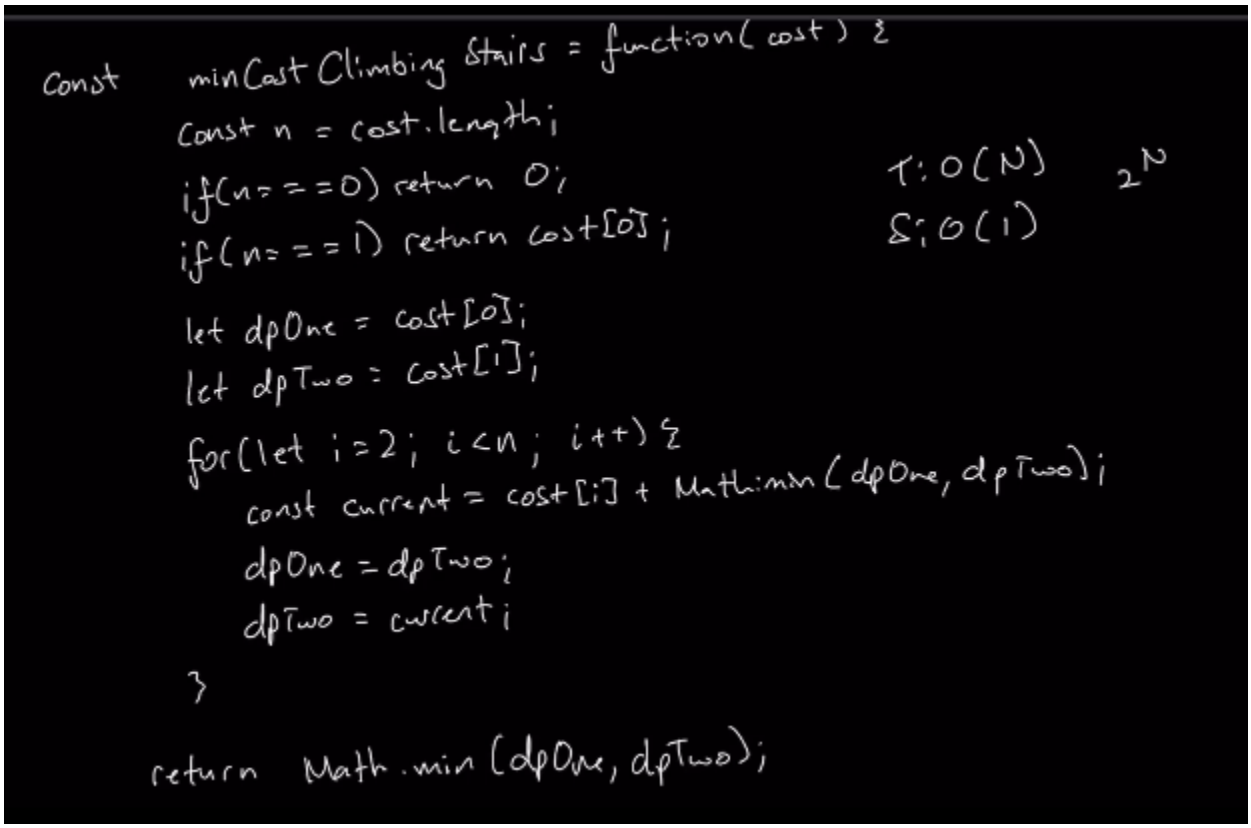
With DP we save a lot of calculations



Method 2: DP Tabulation Bottom up



Optimization



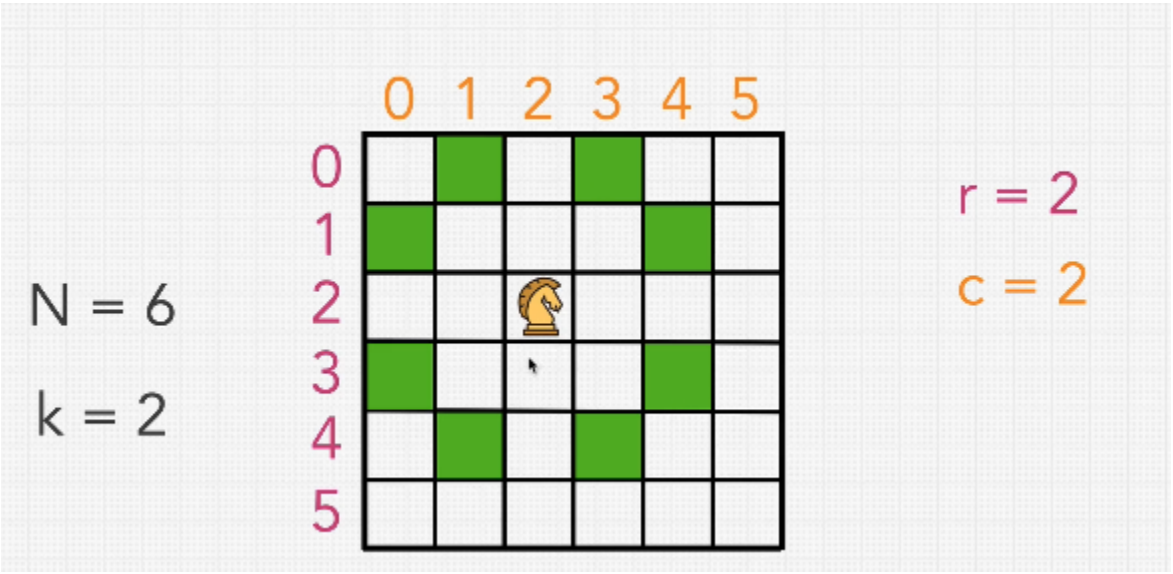
We can use just two variables instead of dp array

Knight probability in chessboard

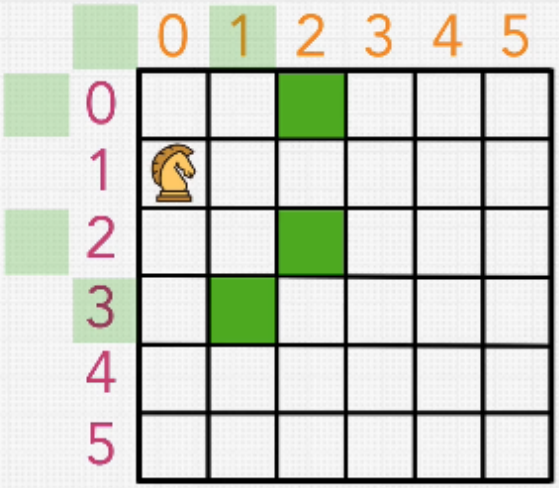
Problem

On a given *nxn* chessboard, a knight piece will start at the *r*-th row and *c*-th column. The knight will attempt to make *k* moves.

A knight can move in 8 possible ways. Each move will choose one of these 8 at random. The knight continues moving until it finishes *k* moves or it moves off the chessboard. Return the probability that the knight is on the chessboard after it finishes moving.



3 / 8



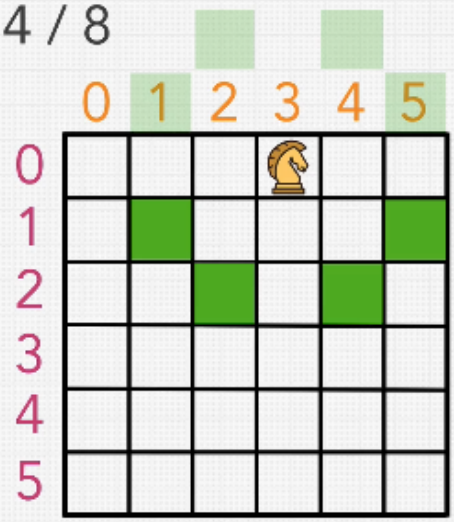
k = 2

r = 2

c = 2

Step 2

0.375 0.375 4 / 8

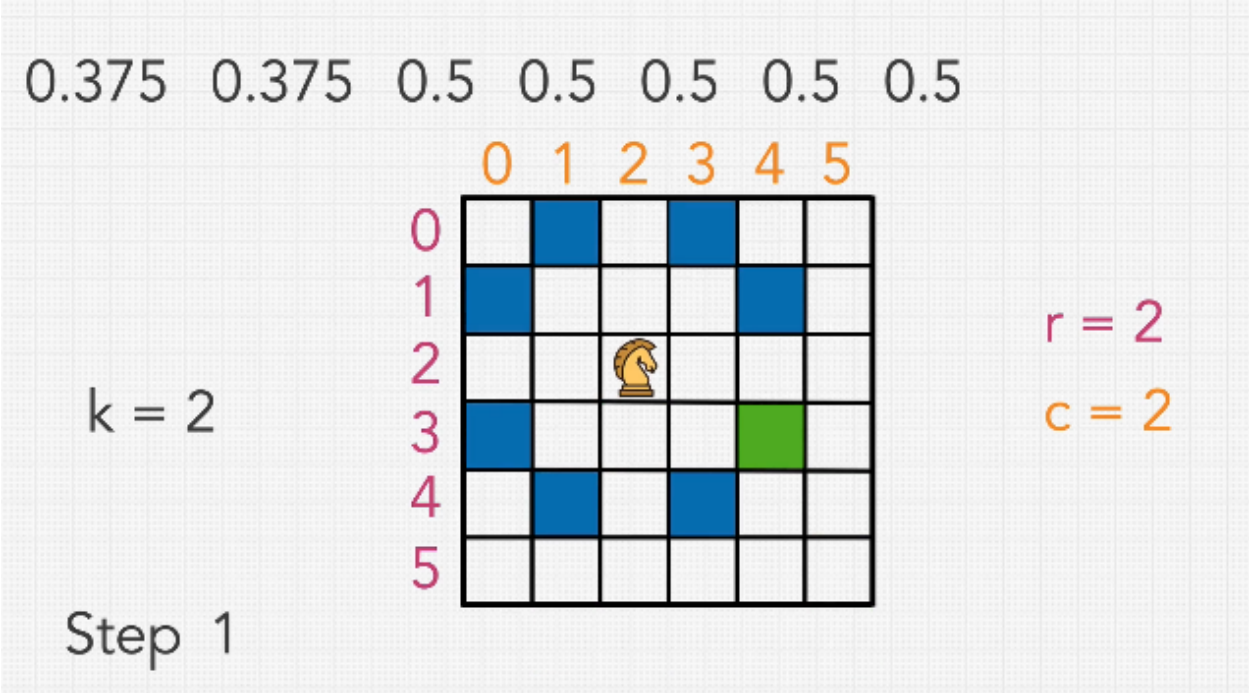


k = 2

r = 2

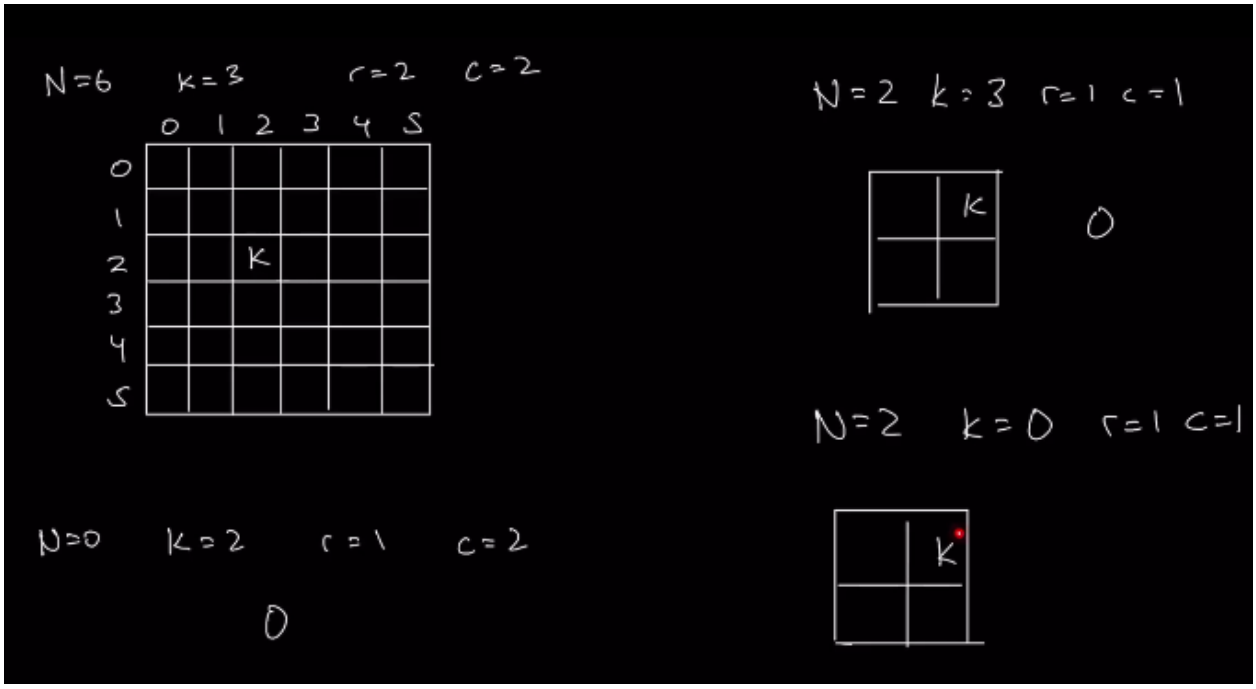
c = 2

Step 2



$$\frac{0.375}{8} + \frac{0.375}{8} + \frac{0.5}{8} + \frac{0.5}{8} + \frac{0.5}{8} + \frac{0.5}{8} + \frac{0.5}{8} + \frac{0.75}{8}$$

0.53125 -> answer

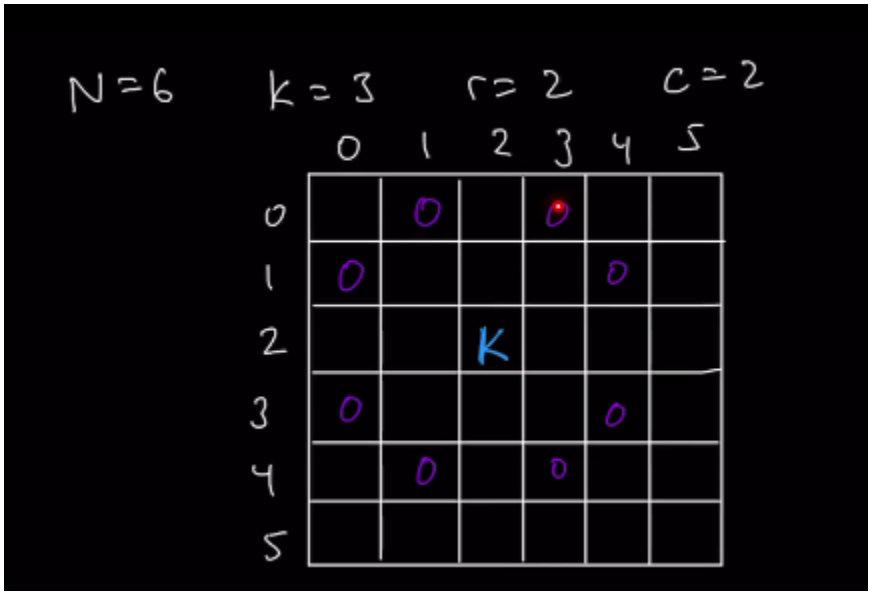


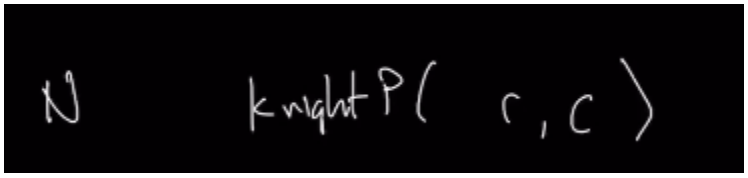
Method: Recurrence relation and DP

Example 2:

Reduce the problem to smaller but repetitive

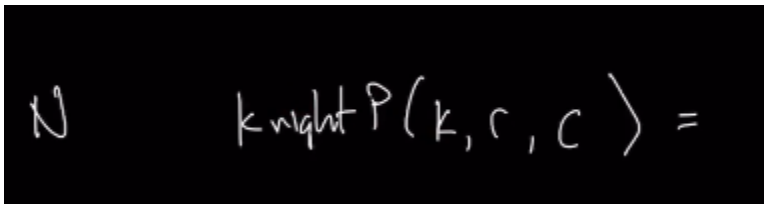
In the chessboard example the movement depend on the current (r,c) value



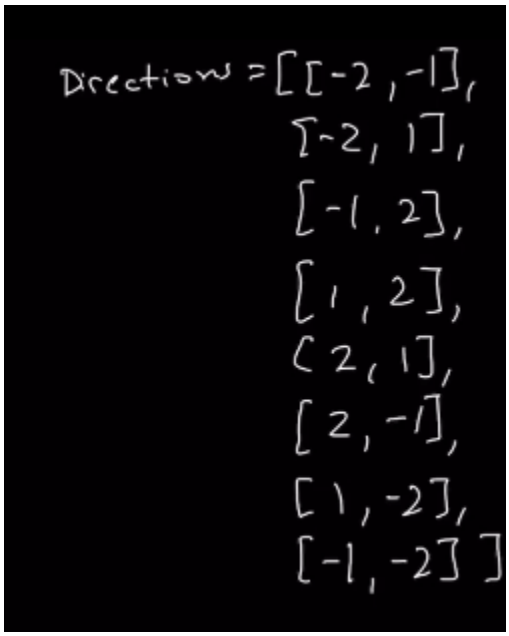


So r and c are definitely parameters

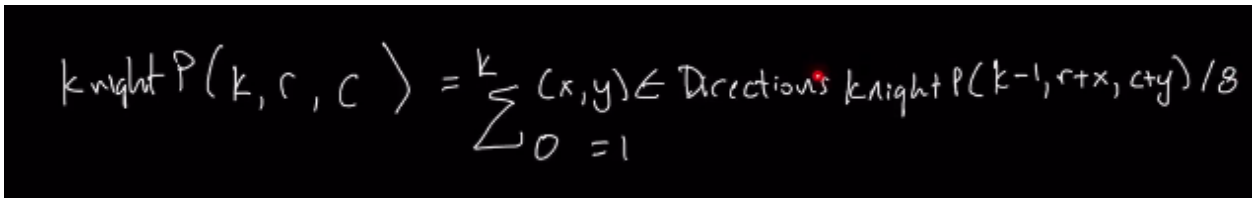
K also represent something in the steps



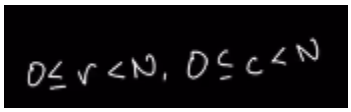
We can represent de directions



The number of recursion calls are

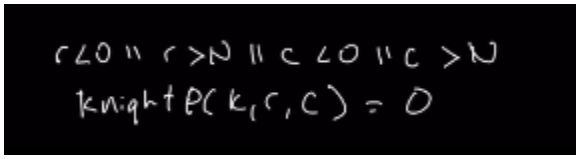


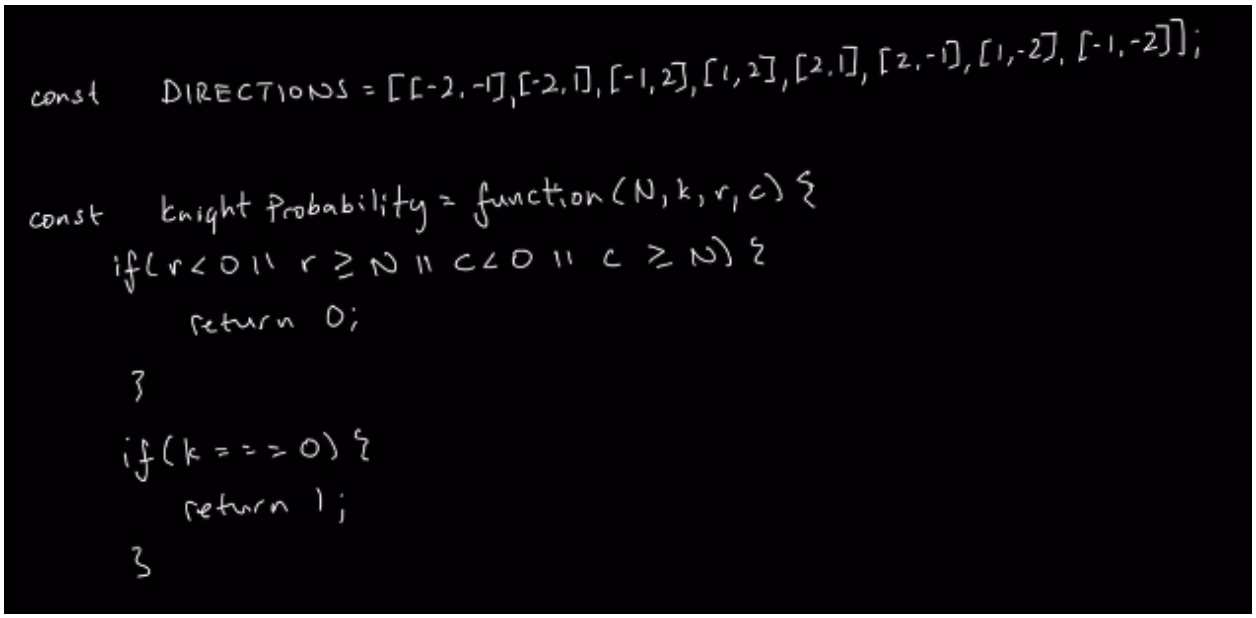
For the eight directions

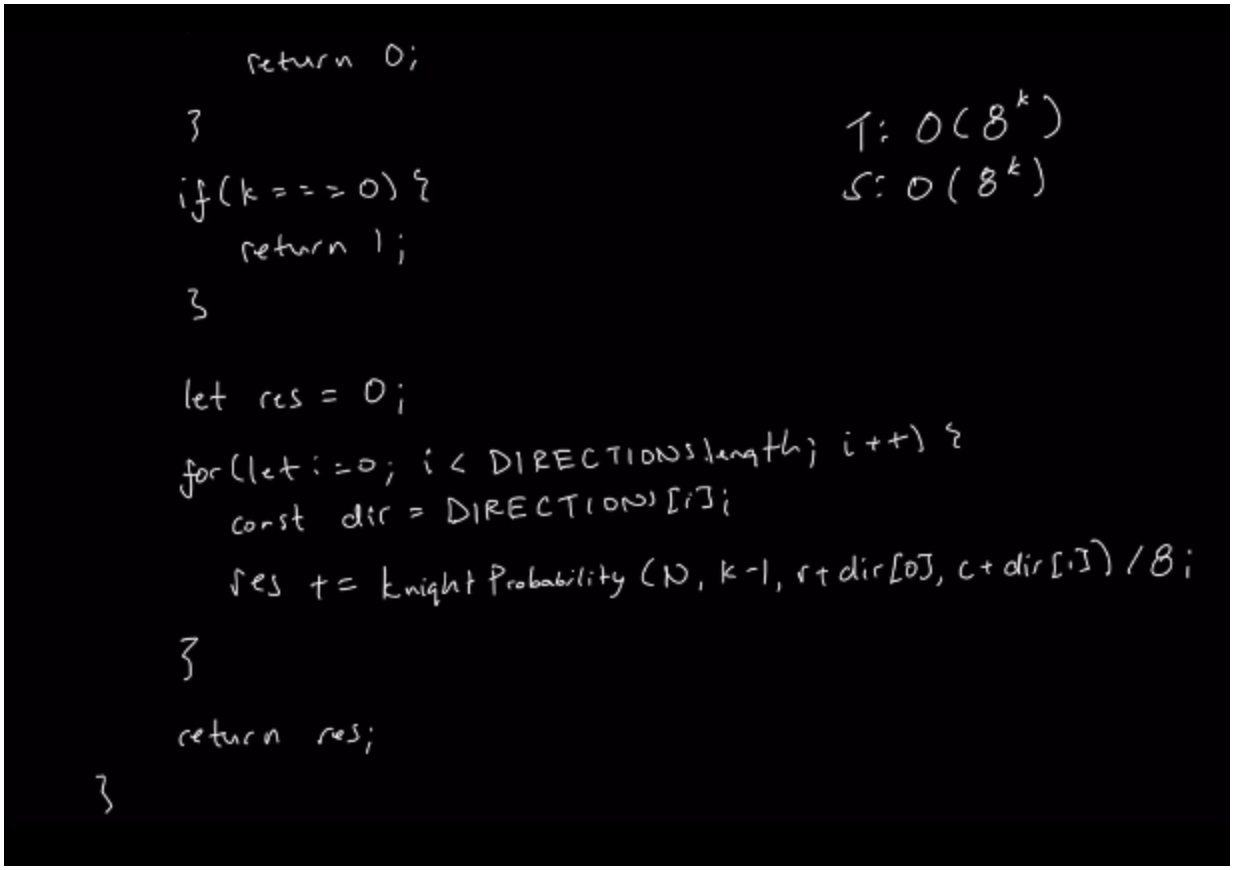


Only if we are inside the board

If we are out side







Now we have to memoize some parts

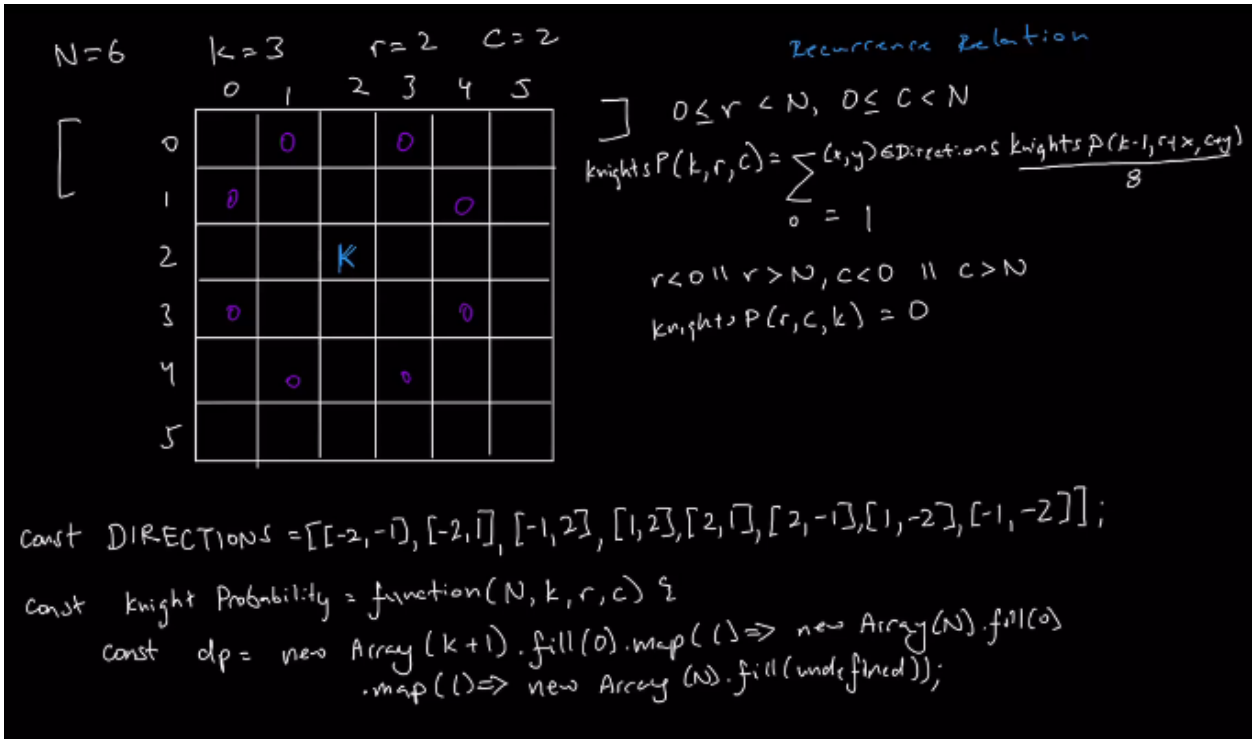
The previous steps made also determinate a factor:

```
const knightProbability = function(N, k, r, c) {
    const dp = new Array(k+1).fill(0).map(
        () => new Array(N).fill(0).map(() => new Array(N).fill(undefined))};

    return recurse(N, k, r, c, dp);
};

const recurse = function(N, k, r, c, dp) {
    if(r < 0 || c < 0 || r >= N || c >= N) {
        return 0;
    }
    if(k === 0) {
        return 1;
    }
    if(dp[k][r][c] !== undefined) return dp[k][r][c];
    let res = 0;
    for(let i = 0; i < DIRECTIONS.length; i++) {
        const dir = DIRECTIONS[i];
        res += recurse(N, k-1, r+dir[0], c+dir[1], dp) / 8;
    }
    dp[k][r][c] = res;
    return dp[k][r][c];
};
```

T: $O(N^2 \times k)$

S: $O(N^2 \times k)$

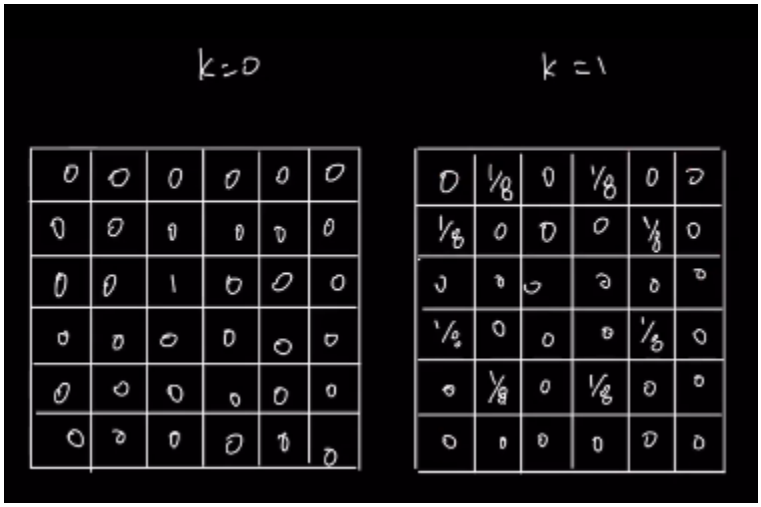
Method 2: Tabulation Bottom up

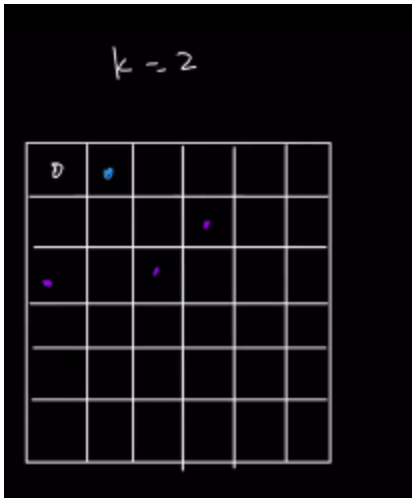


Here exist a depth of K

We generate all i need for this level of K, and fill up

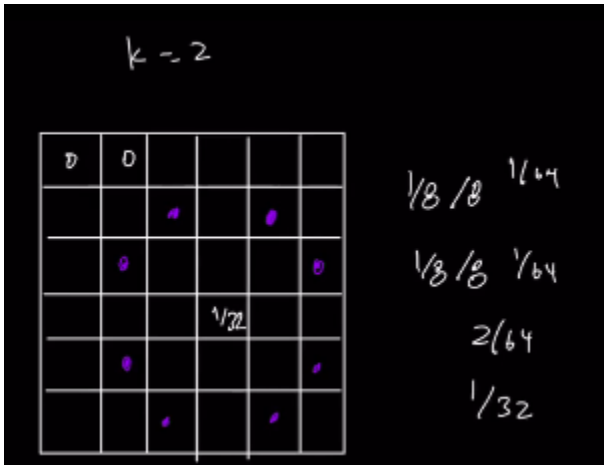
The problem want the probability so for 8 movements we fill with





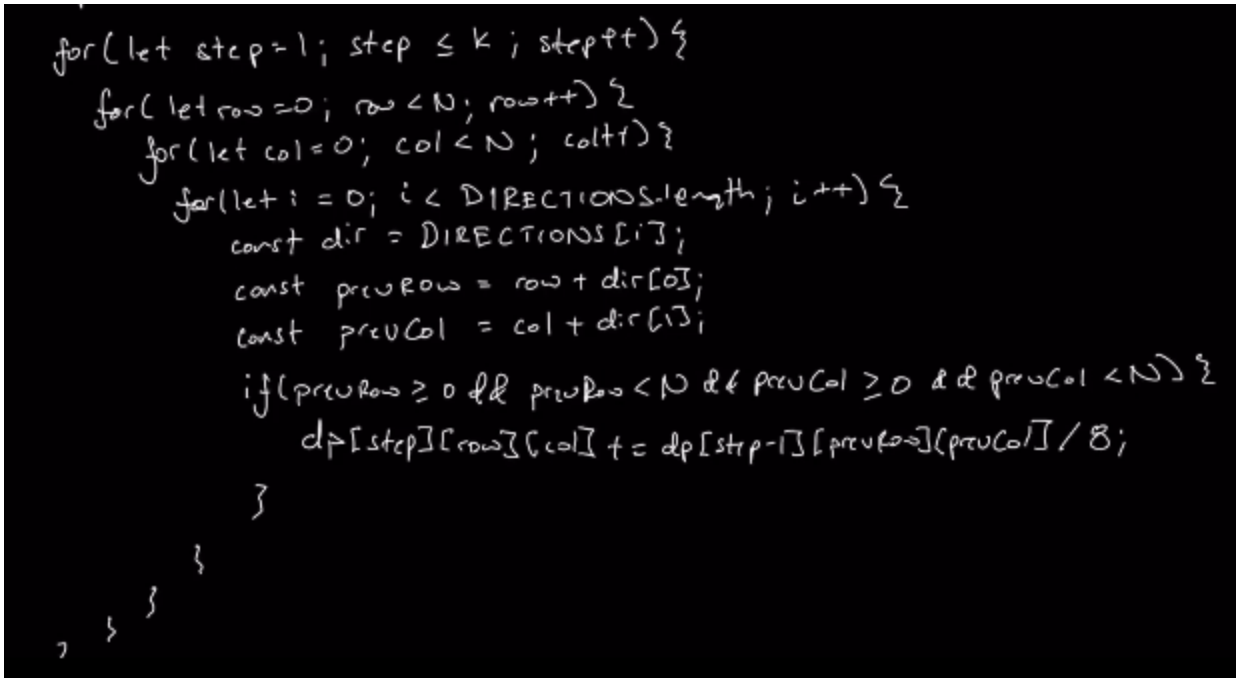
We check for each cell what is the probability of reach that part

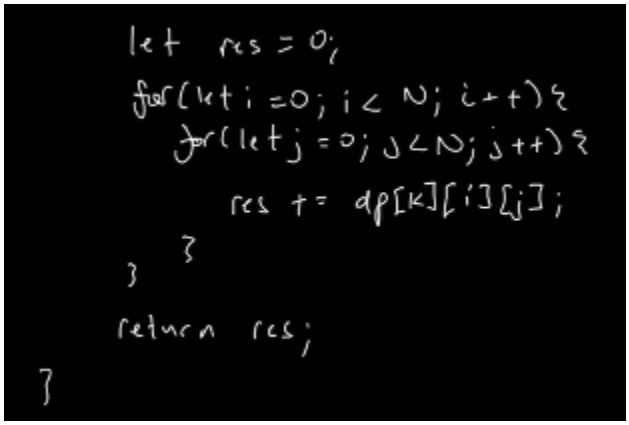
We divide k times the probability and sum

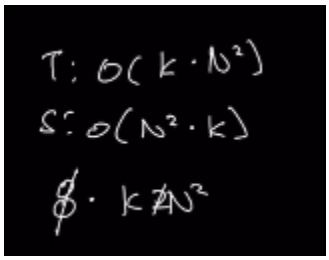


We try to drive the solution into table

```
const Directions = [[-2,-1],[-2,1],[-1,2],[1,2],[2,1],[2,-1],[1,-2],[-1,-2]];

const knightProbability = function(N, k, r, c) {
    const dp = new Array(k+1).fill(0).map(()=> new Array(N).map(()=>
                new Array(N).fill(0));
```







We also need to keep track of some parts

```
knightProbability = function(N, k, r, c) {
const prevDp = new Array(N).fill(0).map(()=> new Array(N).fill(0));
const currDp = new Array(N).fill(0).map(()=> new Array(N).fill(0));

prevDp[r][c] = 1;
```

```
if(prevRow >= 0 && prevRow < N && prevCol >= 0 && prevCol < N) {
    currDp[row][col] += prevDp[prevRow][prevCol] / 8;
}
```

And update the name of variables

```
prevDp = currDp;
currDp = new Array(N).fill(0).map(() => new Array(N).fill(0));
```

```
let res = 0
for(let i=0; i<N; i++) {
  for(let j=0; j<N; j++) {
      res += prevDp[i][j];
  }
}
return res;
}
```

```
T: O(K·N²)
S: O(2N²)
```

Backtracking

Sudoku solver

Problem

Create a function that solves for any 9x9 sudoku puzzle given.

Each of the digits 1-9 must occur exactly once in each row.

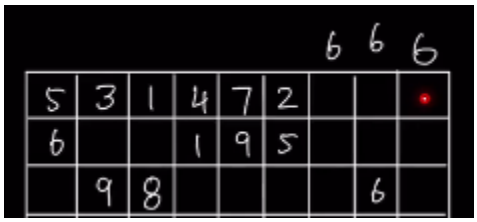
And for each roy

Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

We get a sample of board

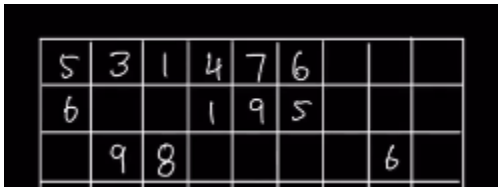
5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Method: Backtracking



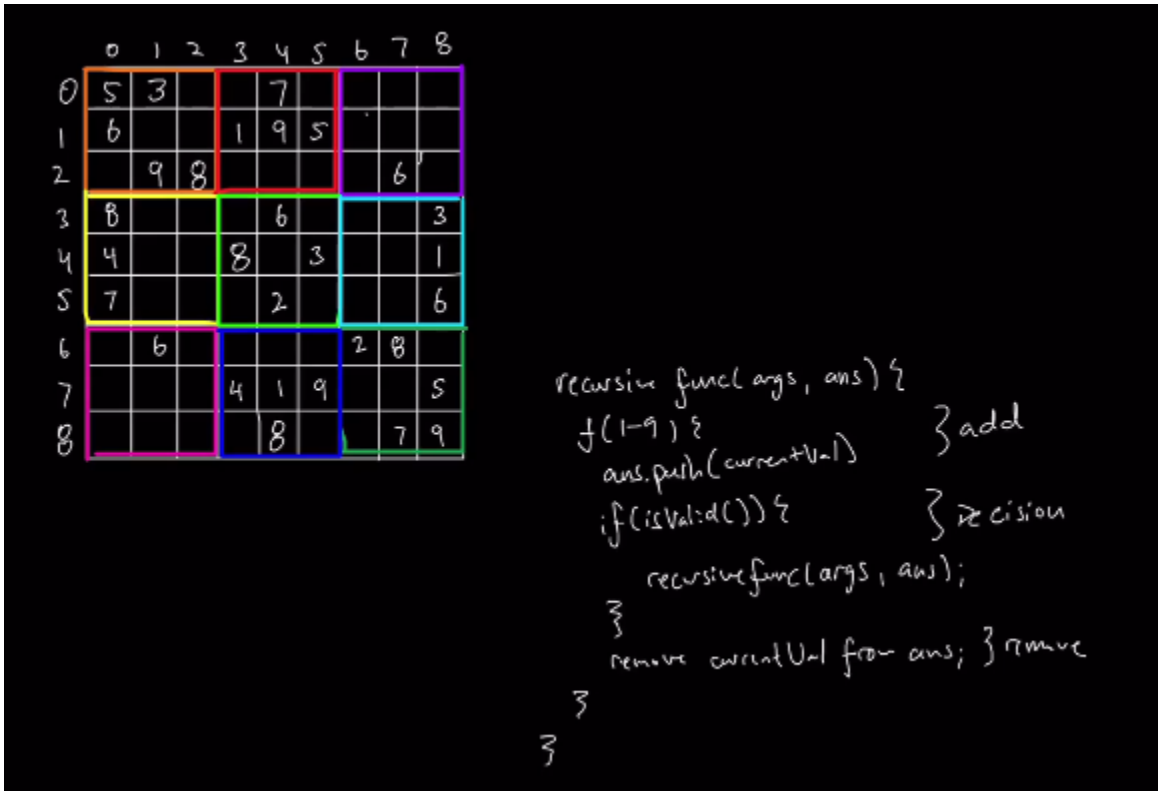
In this case exists an error

So we back track to another valid solution

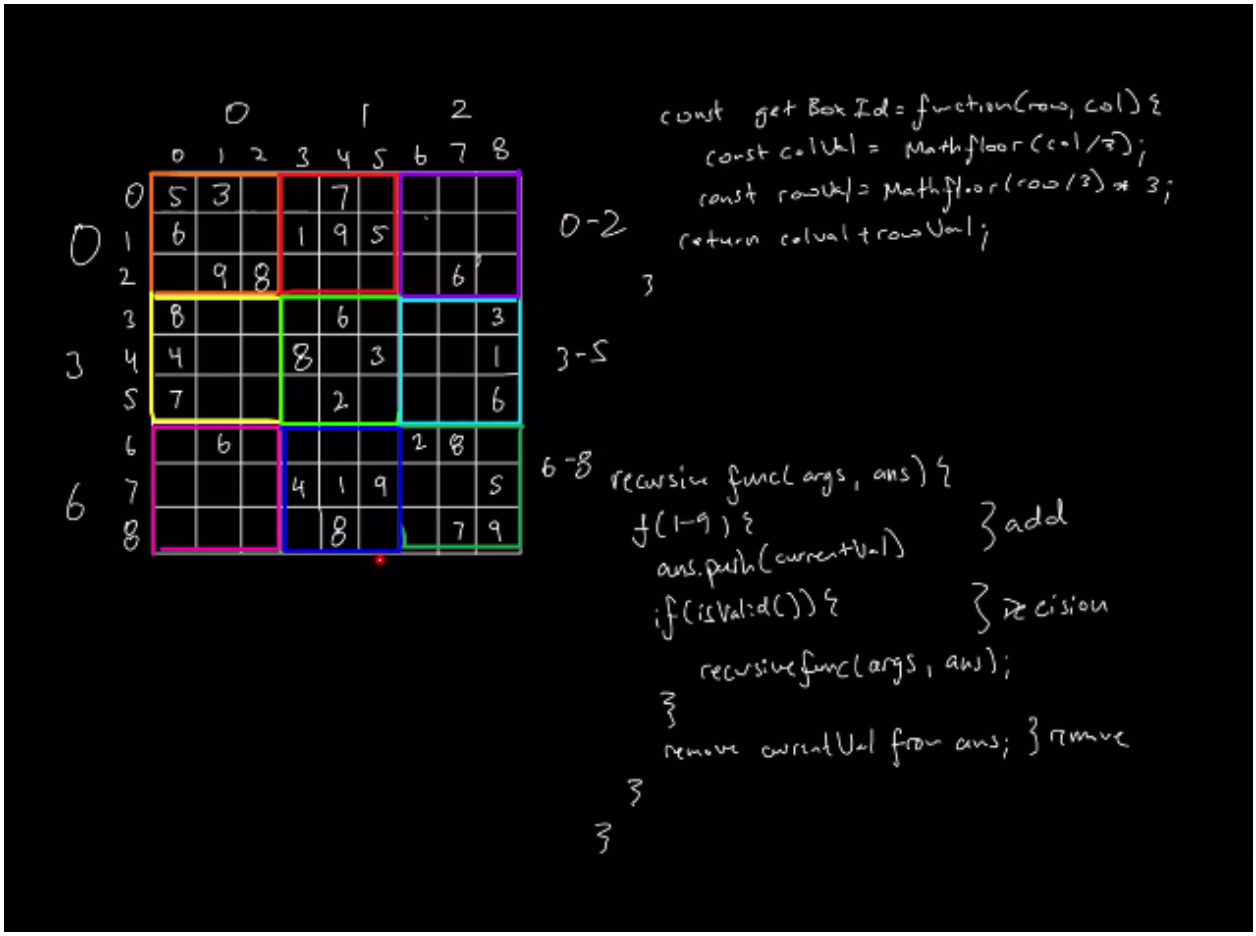


Sometimes the back track is more extensible not just one step

But backtracking doesn’’t cares



This is a general schema of backtracking



Keep track of all necessary

```
const solveSudoku = function(board) {
    const n = board.length;
    const boxes = new Array(n), rows = new Array(n), cols = new Array(n);
    for(let i=0; i<n; i++) {
        boxes[i] = {};
        rows[i] = {};
        cols[i] = {};
    }
    for(let r=0; r<n; r++) {
        for(let c=0; c<n; c++) {
```

```
        if (board[r][c] !== '.') {
          const val = board[r][c];
          const boxId = getBoxId(r, c);
          boxes[boxId][val] = true;
          rows[r][val] = true;
          cols[c][val] = true;
        }
      }
    }

    solveBacktrack(board, boxes, rows, cols, 0, 0);
};
```

```
const isValid = function(box, row, col, num) {
    if(box[num] || row[num] || col[num]) {
      return false;
    } else {
      return true;
    }
};
```

```
const solveBacktrack = function(board, boxes, rows, cols, r, c) {
    if(r === board.length || c === board[0].length) {
        return true;
    } else {
      if(board[r][c] === '.') {
```

```
for(let num=1; num<=9; num++) {
    const numVal = num.toString();
    board[r][c] = numVal;
    const boxId = getBoxId(r, c);
    const box = boxes[boxId];
    const row = rows[r];
    const col = cols[c];
    if(isValid(box, row, col, numVal)){
        box[numVal] = true;
        col[numVal] = true;
        row[numVal] = true;
```

```
        if(c === board[0].length -1) {
            if( solveBacktrack(board, boxes, rows, cols, r+1, 0) {
                return true;
            }
        } else {
            if( solveBacktrack( board, boxes, rows, cols, r, c+1) {
                return true;
            }
        }
        delete box[numVal];
        delete row[numVal];
        delete col[numVal];
    }
board[r][c] = '.';
```

```
} else {
    if(c === board[0].length) {
        if( solveBacktrack(board, boxes, rows, cols, r+1, 0) {
            return true;
        }
    } else {
        if( solveBacktrack( board, boxes, rows, r, c+1) {
    }
}
```

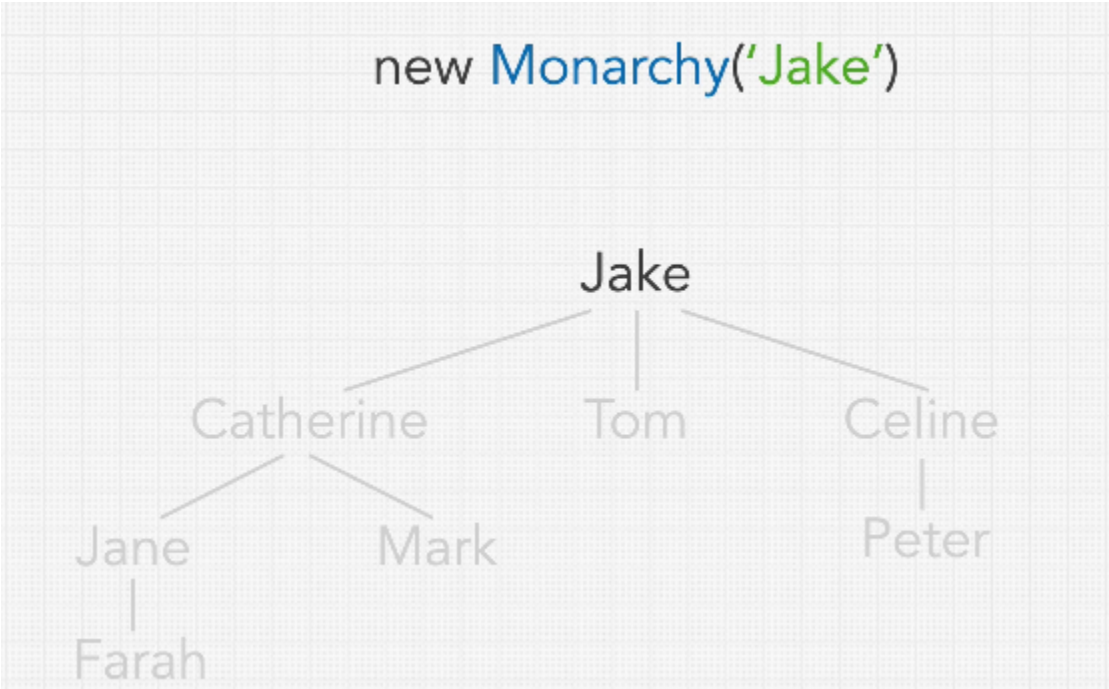
Interface design

Monarchy

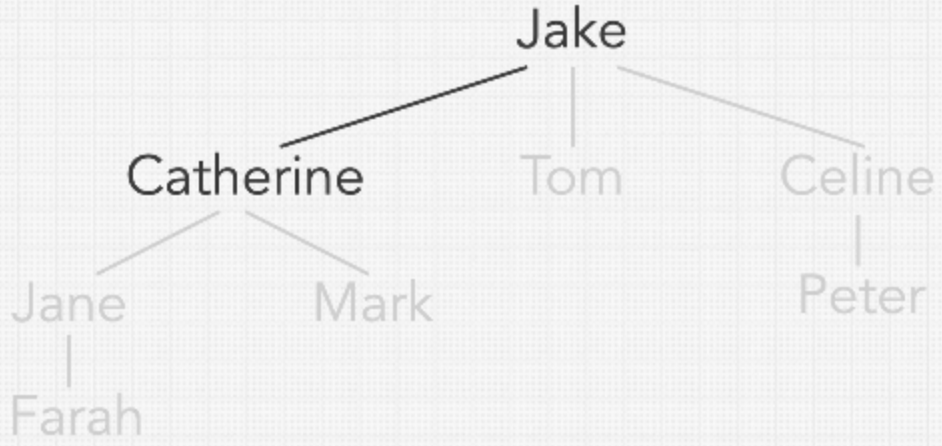
Problem

Given the following interface, implement its methods.

```
interface Monarchy {
  void birth(String child, String parent);
  void death(String name);
  List<String> getOrderOfSuccession();
}
```

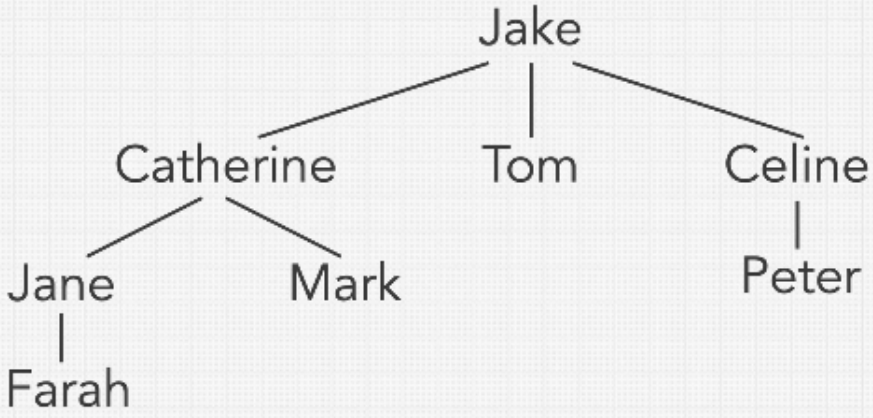


birth('Catherine', 'Jake')



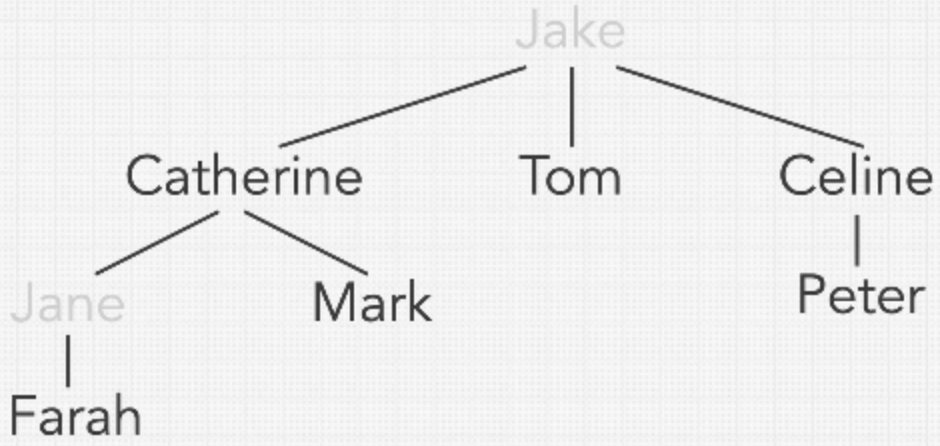
getOrderOfSuccession()

['Jake', 'Catherine', 'Jane', 'Farah', 'Mark', 'Tom', 'Celine', 'Peter']



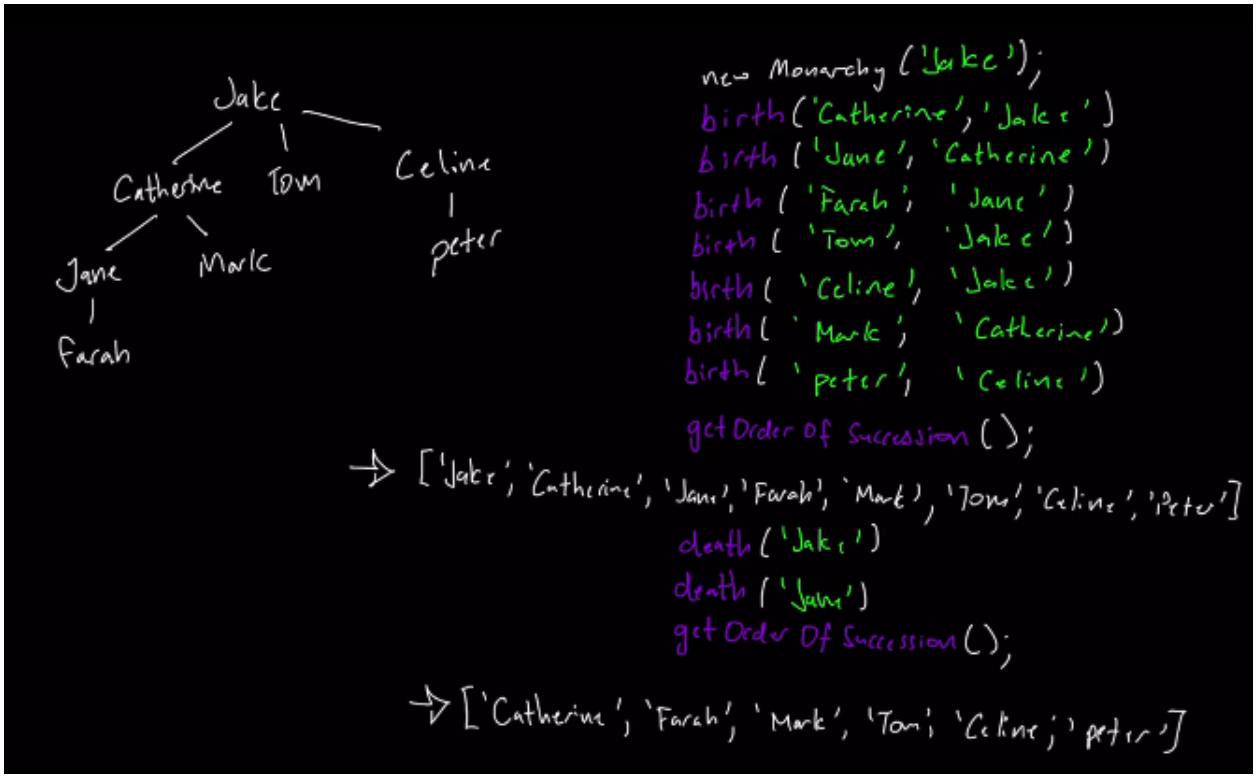
getOrderOfSuccession()

['Catherine', 'Farah', 'Mark', 'Tom', 'Celine', 'Peter']



Can we implement helper classes/objects?

Yes, you can use any features you see fit.


Jake
Catherine
Tom
Celine
Jane
Marc
peter
Farah
new Monarchy('Jake');
birth('Catherine', 'Jake')
birth('Jane', 'Catherine')
birth('Farah', 'Jane')
birth('Tom', 'Jake')
birth('Celine', 'Jake')
birth('Mark', 'Catherine')
birth('peter', 'Celine')
getOrderOfSuccession();
['Jake', 'Catherine', 'Jane', 'Farah', 'Mark', 'Tom', 'Celine', 'peter']
death('Jake')
death('Jane')
getOrderOfSuccession();
['Catherine', 'Farah', 'Mark', 'Tom', 'Celine', 'peter']

Solution

```
class Person {
    constructor(name) {
        this.name = name;
        this.isAlive = true;
        this.children = [];
    }
}

class Monarchy {
  constructor(king) {
    this.king = new Person(king);
    this._persons = {
      [this.king.name]: this.king
    };
  }
```

```
  birth(childName, parentName) {
    const parent = this._persons[parentName];
    const newChild = new Person(childName);
    parents.children.push(newChild);
    this._persons[childName] = newChild;
  }
```

```
  death(name) {
    const person = this._persons[name];

    if(person === undefined) {
      return null;
    }

    person.isAlive = false;
  }
```

```
getOrderOfSuccession() {
    const order = [];
    this._dfs(this.king, order);
    return order;
}
```

```
    _dfs(currentPerson, order) {
        if(currentPerson.isAlive) {
            order.push(currentPerson.name);
        }
        for(let i = 0; i < currentPerson.children.length; i++) {
            this._dfs(currentPerson.children[i], order);
        }
    }
}
```

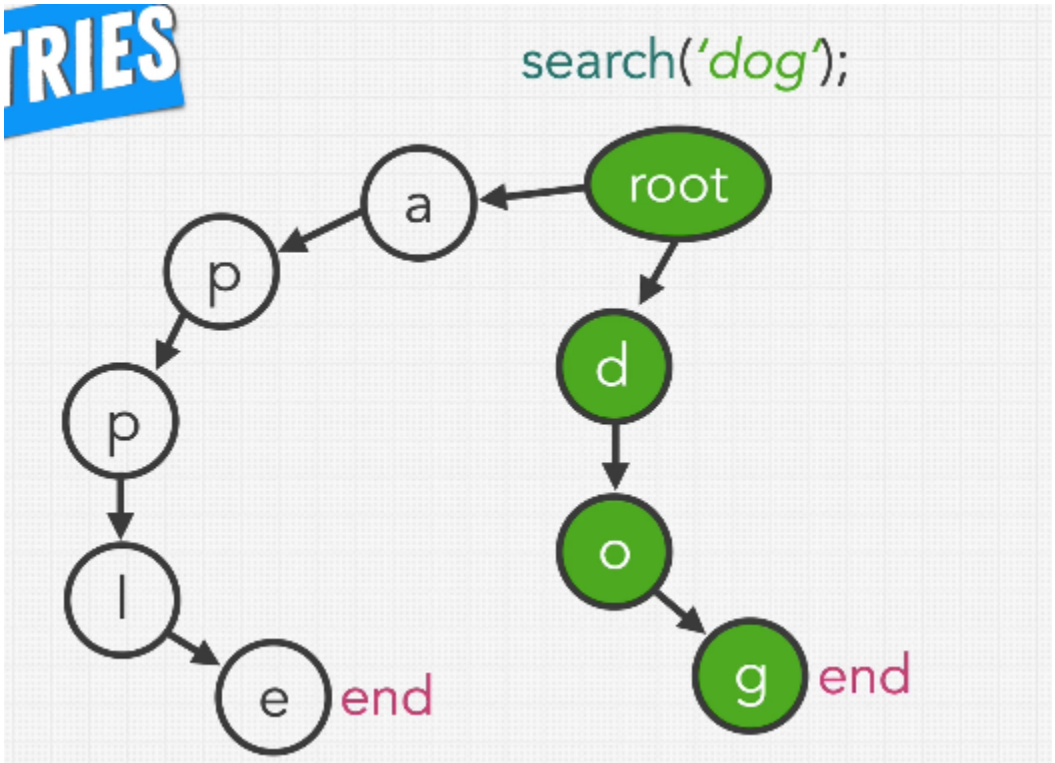
Tries

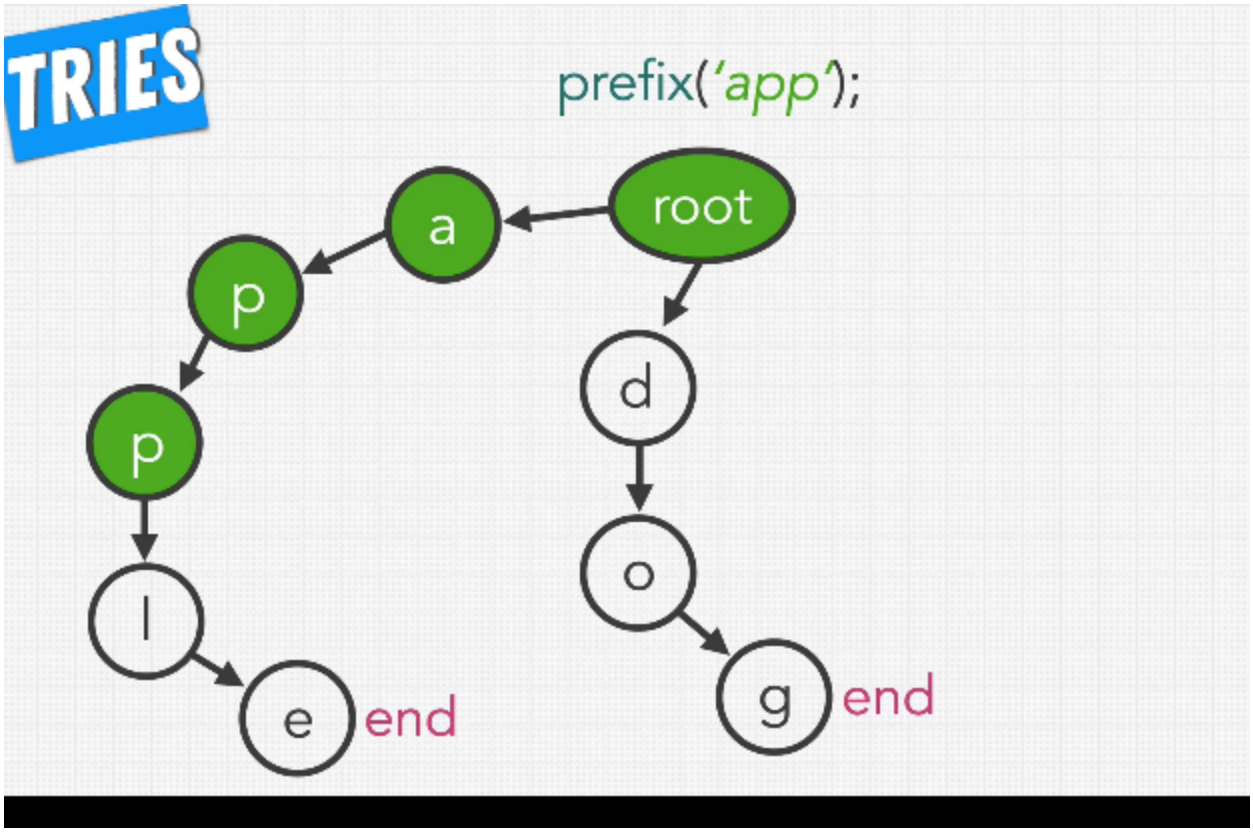
Implement prefix tries

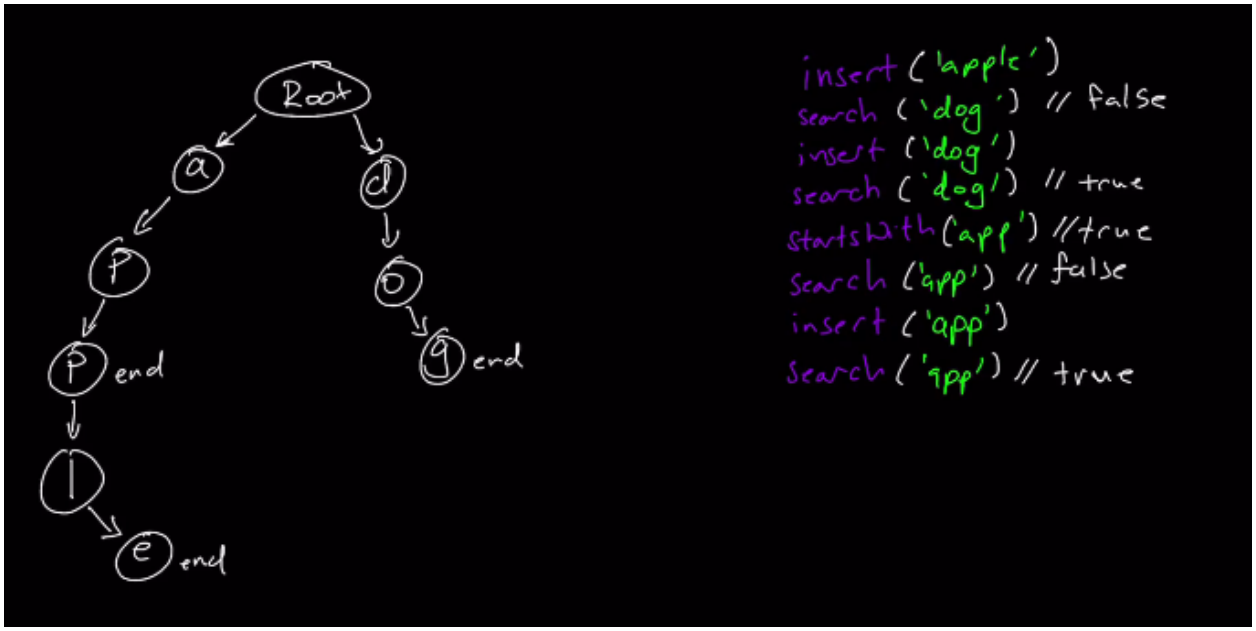
Problem

Implement a trie with *insert*, *search* and *startsWith* methods.

```
interface Trie {
  void insert(String word);
  Boolean search(String word);
  Boolean startsWith(String prefix);
}
```




TRIES
prefix('app');
root
a
p
p
l
e
end
d
o
g
end


Root
a
d
p
o
p
end
g
end
l
e
end
insert ('apple')
search ('dog') // false
insert ('dog')
search ('dog') // true
startsWith ('app') //true
search ('app') // false
insert ('app')
search ('app') // true

Solution

```
class TrieNode {
    constructor() {
        this.end = false;
        this.keys = {};
    }
}
```

```
class Trie {
    constructor() {
        this.root = new TrieNode();
    }
```

```
}

insert(word, node = this.root) {
    if(word.length === 0) {
        node.end = true;
        return;
    } else if(!node.keys[word[0]]) {
        node.keys[word[0]] = new TrieNode();
        this.insert(word.substring(1), node.keys[word[0]]);
    } else {
        this.insert(word.substring(1), node.keys[word[0]]);
    }
}
```

dog

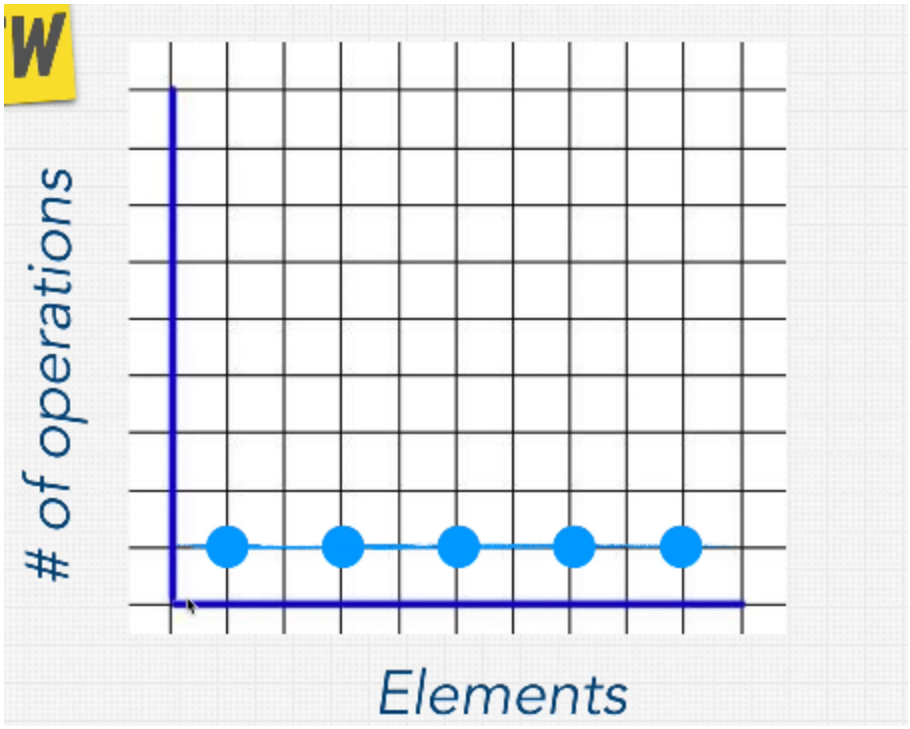
cat

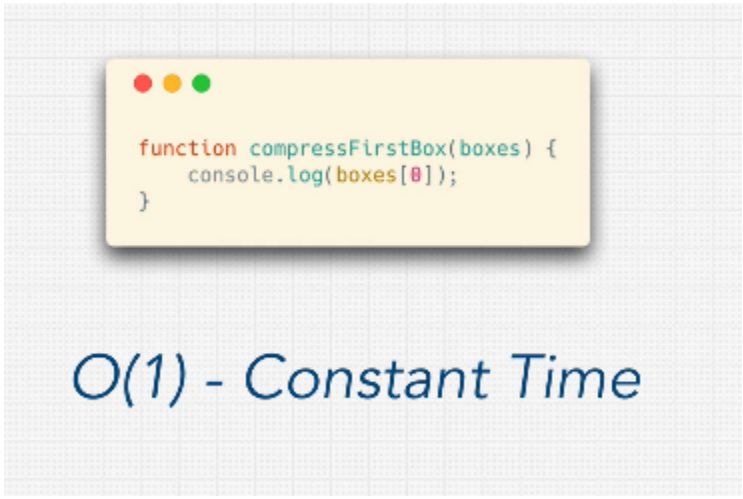
```
search ( word, node = this.root ) {
  if ( word.length === 0 && node.end ) {
      return true;
  } else if ( word.length === 0 ) {
      return false;
  } else if ( ! node.keys [word[0]] ) {
      return false;
  }else{
     return  this.search ( word.substring (1), node.keys [word[0]] );
  }
}
```

```
startsWith ( prefix, node = this.root ) {
    if ( prefix.length === 0 ) {
        return true;
    } else if ( ! node.keys [prefix[0]] ) {
        return false;
    }else{
        return  this.startsWith ( prefix.substring (1), node.keys [prefix[0]] );
    }
}
```

T: O(L)

S: O(N)
   O(L)


# of operations
Elements


function compressFirstBox(boxes) {
console.log(boxes[0]);
}
O(1) - Constant Time